Documentation of FullSWOF_2D

v1.10.00 (2023-04-06)

Generated by Doxygen 1.9.6
on Thu Apr 6 2023 17:15:37

Chapter 1

Todo List

Class Boundary_condition

Add time and space dependancy in the boundary conditions.

Improve boundary conditions at the second order for the wall and periodic conditions.

Take into account source terms (friction and topography) in the boundary conditions, see Le Roux [2001], Bristeau and Coussin [2001].

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 4

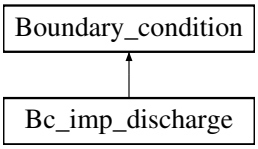
Class Documentation

4.1 Bc_imp_discharge Class Reference

Imposed discharge.

`#include <bc_imp_discharge.hpp>`

Inheritance diagram for Bc_imp_discharge:



Public Member Functions

- Bc_imp_discharge (Parameters &, TAB &, int, int)

  Constructor.
- SCALAR getValueOfPolynomial (const SCALAR, const SCALAR, const SCALAR, const SCALAR, const SCALAR, int, int) const

  Gives the value of the function that must vanish.
- SCALAR getValueofDerivativeOfPolynomial (const SCALAR, const SCALAR, const SCALAR, const SCALAR, int, int) const

  Gives the value of the derivative of the function that must vanish.
- SCALAR newtonSolver (const SCALAR, const SCALAR, const SCALAR, const SCALAR, const SCALAR, int, int) const

  Solves the equation with Newton iterative method.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)

  Calculates the boundary condition.
- virtual ∼Bc_imp_discharge ()

  Destructor.

Public Member Functions inherited from Boundary_condition

- Boundary_condition (Parameters &)

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)=0

  Function to be specified in each boundary condition.

- SCALAR get_hbound () const

  Gives the water height on the fictive cell.
- SCALAR get_unormbound () const

  Gives the normal velocity of the flow on the fictive cell.
- SCALAR get_utanbound () const

  Gives the tangential velocity of the flow on the fictive cell.
- virtual ~Boundary_condition ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Boundary_condition

- const int NXCELL
- const int NYCELL
- SCALAR hbound
- SCALAR unormbound
- SCALAR utanbound
- SCALAR unormfix
- map< int, int > Choice_Lbound
- map< int, int > Choice_Rbound
- map< int, int > Choice_Bbound
- map< int, int > Choice_Tbound

4.1.1 Detailed Description

Imposed discharge.

Class that computes the boundary condition where the discharge is imposed. For supercritical flows, the water height is imposed too.

Definition at line 73 of file bc_imp_discharge.hpp.

4.1.2 Constructor & Destructor Documentation

Bc_imp_discharge()

```
Bc_imp_discharge::Bc_imp_discharge (
            Parameters & par,
            TAB & z,
            int n1,
            int n2 )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.
in,out	*z*	vector that represents the topography with suitable values on the fictive cells.

Definition at line 60 of file bc_imp_discharge.cpp.

~Bc_imp_discharge()

```
Bc_imp_discharge::~Bc_imp_discharge ( )  [virtual]
```

Destructor.

Definition at line 256 of file bc_imp_discharge.cpp.

4.1.3 Member Function Documentation

calcul()

```
void Bc_imp_discharge::calcul (
            SCALAR hin,
            SCALAR unorm_in,
            SCALAR utan_in,
            SCALAR hfix,
            SCALAR qfix,
            SCALAR hin_oppbound,
            SCALAR unorm_in_oppbound,
            SCALAR utan_in_oppbound,
            SCALAR time,
            int n1,
            int n2 )  [virtual]
```

Calculates the boundary condition.

Two cases are considered: subcritical and supercritical flows.

Parameters

in	*hin*	water height of the first cell inside the domain.
in	*unorm_in*	normal velocity of the first cell inside the domain.
in	*utan_in*	tangential velocity of the first cell inside the domain.
in	*hfix*	fixed (imposed) value of the water height (only for the supercritical case).
in	*qfix*	fixed (imposed) value of the discharge.
in	*hin_oppbound*	value of the water height of the first cell inside the domain at the opposite bound (unused).
in	*unorm_in_oppbound*	value of the normal velocity of the first cell inside the domain at the opposite bound (unused).
in	*utan_in_oppbound*	value of the tangential velocity of the first cell inside the domain at the opposite bound (unused).
in	*time*	current time (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Warning

Warning in the method Bc_imp_discharge::calcul() The water height at the inflow is zero ... continuing!

Modifies

Boundary_condition::hbound water height on the fictive cell.
Boundary_condition::unormbound normal velocity on the fictive cell.
Boundary_condition::utanbound tangential velocity on the fictive cell.

Implements Boundary_condition.

Definition at line 191 of file bc_imp_discharge.cpp.

getValueofDerivativeOfPolynomial()

```
SCALAR Bc_imp_discharge::getValueofDerivativeOfPolynomial (
            const SCALAR HIN,
            const SCALAR UNORM_IN,
            const SCALAR UTAN_IN,
            const SCALAR H,
            int n1,
            int n2 ) const
```

Gives the value of the derivative of the function that must vanish.

Computes $3\sqrt{gH} - (n1+n2)(UNORM\_IN + 2(n1+n2)\sqrt{gHIN})$ where $n1, n2$ are the normals.

Parameters

in	*HIN*	water height of the first cell inside the domain.
in	*UNORM_IN*	normal velocity of the first cell inside the domain.
in	*UTAN_IN*	tangential velocity of the first cell inside the domain (unused).
in	*H*	value for the variable of the polynomial function.
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Returns

The value of derivative of the polynomial function defined in Bc_imp_discharge::getValueOfPolynomial().

Definition at line 133 of file bc_imp_discharge.cpp.

getValueOfPolynomial()

```
SCALAR Bc_imp_discharge::getValueOfPolynomial (
            const SCALAR HIN,
            const SCALAR UNORM_IN,
            const SCALAR UTAN_IN,
            const SCALAR QFIX,
            const SCALAR H,
            int n1,
            int n2 ) const
```

Gives the value of the function that must vanish.

Computes $2H\sqrt{gH} - (n1+n2)(UNORM\_IN + 2(n1+n2)\sqrt{gHIN})H - |QFIX|$ where $n1, n2$ are the normals.

Parameters

in	*HIN*	water height of the first cell inside the domain.
in	*UNORM_IN*	normal velocity of the first cell inside the domain.
in	*UTAN_IN*	tangential velocity of the first cell inside the domain (unused).
in	*QFIX*	fixed (imposed) value of the discharge.
in	*H*	value for the variable of the polynomial function.
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Returns

The value of the polynomial function.

Definition at line 113 of file bc_imp_discharge.cpp.

newtonSolver()

```
SCALAR Bc_imp_discharge::newtonSolver (
            const SCALAR HIN,
            const SCALAR UNORM_IN,
            const SCALAR UTAN_IN,
            const SCALAR QFIX,
            const SCALAR H_INIT,
            int n1,
            int n2 ) const
```

Solves the equation with Newton iterative method.

Finds the root of the polynomial function corresponding to the imposed discharge. Needs the evaluation of the function and of its derivative.

Parameters

in	*HIN*	water height of the first cell inside the domain.
in	*UNORM_IN*	normal velocity of the first cell inside the domain.
in	*UTAN_IN*	tangential velocity of the first cell inside the domain.
in	*QFIX*	fixed (imposed) value of the discharge.
in	*H_INIT*	initialization of the Newton solver.
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Warning

Warning: Newton bc did not converge.

Returns

h: water height that satifies Riemann invariants.

Definition at line 152 of file bc_imp_discharge.cpp.

The documentation for this class was generated from the following files:

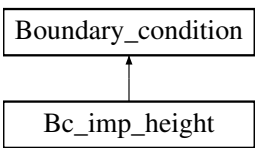
- Headers/libboundaryconditions/bc_imp_discharge.hpp
- Sources/libboundaryconditions/bc_imp_discharge.cpp

4.2 Bc_imp_height Class Reference

Imposed water height.

`#include <bc_imp_height.hpp>`

Inheritance diagram for Bc_imp_height:



Public Member Functions

- Bc_imp_height (Parameters &, TAB &, int, int)

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)

  Calculates the boundary condition.
- virtual ∼Bc_imp_height ()

  Destructor.

Public Member Functions inherited from Boundary_condition

- Boundary_condition (Parameters &)

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)=0

  Function to be specified in each boundary condition.
- SCALAR get_hbound () const

  Gives the water height on the fictive cell.
- SCALAR get_unormbound () const

  Gives the normal velocity of the flow on the fictive cell.
- SCALAR get_utanbound () const

  Gives the tangential velocity of the flow on the fictive cell.
- virtual ∼Boundary_condition ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Boundary_condition

- const int NXCELL
- const int NYCELL
- SCALAR hbound
- SCALAR unormbound
- SCALAR utanbound
- SCALAR unormfix
- map< int, int > Choice_Lbound
- map< int, int > Choice_Rbound
- map< int, int > Choice_Bbound
- map< int, int > Choice_Tbound

4.2.1 Detailed Description

Imposed water height.

Class that computes the boundary condition where the water height is imposed, thanks to the modified method of characteristics. For supercritical flows, the discharge is imposed too.

Definition at line 76 of file bc_imp_height.hpp.

4.2.2 Constructor & Destructor Documentation

Bc_imp_height()

```
Bc_imp_height::Bc_imp_height (
            Parameters & par,
            TAB & z,
            int n1,
            int n2 )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.
in,out	*z*	vector that represents the topography with suitable values on the fictive cells.

Definition at line 64 of file bc_imp_height.cpp.

~Bc_imp_height()

```
Bc_imp_height::~Bc_imp_height ( ) [virtual]
```

Destructor.

Definition at line 181 of file bc_imp_height.cpp.

4.2.3 Member Function Documentation

calcul()

```
void Bc_imp_height::calcul (
            SCALAR hin,
            SCALAR unorm_in,
            SCALAR utan_in,
            SCALAR hfix,
            SCALAR qfix,
            SCALAR hin_oppbound,
            SCALAR unorm_in_oppbound,
            SCALAR utan_in_oppbound,
            SCALAR time,
            int n1,
            int n2 ) [virtual]
```

Calculates the boundary condition.

Two cases are considered: subcritical and supercritical flows. In each case, the values to be imposed depend on the flow (inflow or outflow).

Parameters

in	*hin*	water height of the first cell inside the domain.
in	*unorm_in*	normal velocity of the first cell inside the domain.
in	*utan_in*	tangential velocity of the first cell inside the domain.
in	*hfix*	fixed (imposed) value of the water height.
in	*qfix*	fixed (imposed) value of the discharge.

Parameters

in	*hin_oppbound*	value of the water height of the first cell inside the domain at the opposite bound (unused).
in	*unorm_in_oppbound*	value of the normal velocity of the first cell inside the domain at the opposite bound (unused).
in	*utan_in_oppbound*	value of the tangential velocity of the first cell inside the domain at the opposite bound (unused).
in	*time*	current time (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Modifies

Boundary_condition::hbound water height on the fictive cell.
Boundary_condition::unormbound normal velocity on the fictive cell.
Boundary_condition::utanbound tangential velocity on the fictive cell.

Implements Boundary_condition.
Definition at line 112 of file bc_imp_height.cpp.
The documentation for this class was generated from the following files:

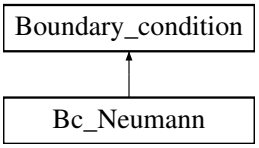
- Headers/libboundaryconditions/bc_imp_height.hpp
- Sources/libboundaryconditions/bc_imp_height.cpp

4.3 Bc_Neumann Class Reference

Neumann condition.

`#include <bc_neumann.hpp>`

Inheritance diagram for Bc_Neumann:



Public Member Functions

- Bc_Neumann (Parameters &, TAB &, int, int)

  Constructor.

- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)

  Calculates the boundary condition.

- virtual ~Bc_Neumann ()

  Destructor.

Public Member Functions inherited from Boundary_condition

- Boundary_condition (Parameters &)

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)=0

  Function to be specified in each boundary condition.
- SCALAR get_hbound () const

  Gives the water height on the fictive cell.
- SCALAR get_unormbound () const

  Gives the normal velocity of the flow on the fictive cell.
- SCALAR get_utanbound () const

  Gives the tangential velocity of the flow on the fictive cell.
- virtual ~Boundary_condition ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Boundary_condition

- const int NXCELL
- const int NYCELL
- SCALAR hbound
- SCALAR unormbound
- SCALAR utanbound
- SCALAR unormfix
- map< int, int > Choice_Lbound
- map< int, int > Choice_Rbound
- map< int, int > Choice_Bbound
- map< int, int > Choice_Tbound

4.3.1 Detailed Description

Neumann condition.

Class that computes the boundary condition with Neumann condition (the normal derivative is null).

Definition at line 73 of file bc_neumann.hpp.

4.3.2 Constructor & Destructor Documentation

Bc_Neumann()

```
Bc_Neumann::Bc_Neumann (
            Parameters & par,
            TAB & z,
            int n1,
            int n2 )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Parameters

in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.
in,out	*z*	vector that represents the topography with suitable values on the fictive cells.

Definition at line 62 of file bc_neumann.cpp.

~Bc_Neumann()

```
Bc_Neumann::~Bc_Neumann ( )  [virtual]
```

Destructor.

Definition at line 143 of file bc_neumann.cpp.

4.3.3 Member Function Documentation

calcul()

```
void Bc_Neumann::calcul (
            SCALAR hin,
            SCALAR unorm_in,
            SCALAR utan_in,
            SCALAR hfix,
            SCALAR qfix,
            SCALAR hin_oppbound,
            SCALAR unorm_in_oppbound,
            SCALAR utan_in_oppbound,
            SCALAR time,
            int n1,
            int n2 )  [virtual]
```

Calculates the boundary condition.

Parameters

in	*hin*	water height of the first cell inside the domain.
in	*unorm_in*	normal velocity of the first cell inside the domain.
in	*utan_in*	tangential velocity of the first cell inside the domain.
in	*hfix*	fixed (imposed) value of the water height (unused).
in	*qfix*	fixed (imposed) value of the discharge (unused).
in	*hin_oppbound*	value of the water height of the first cell inside the domain at the opposite bound (unused).
in	*unorm_in_oppbound*	value of the normal velocity of the first cell inside the domain at the opposite bound (unused).
in	*utan_in_oppbound*	value of the tangential velocity of the first cell inside the domain at the opposite bound (unused).
in	*time*	current time (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary (unused).
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary (unused).

Modifies

Boundary_condition::hbound water height on the fictive cell.
Boundary_condition::unormbound normal velocity on the fictive cell.
Boundary_condition::utanbound tangential velocity on the fictive cell.

Implements Boundary_condition.
Definition at line 108 of file bc_neumann.cpp.
The documentation for this class was generated from the following files:

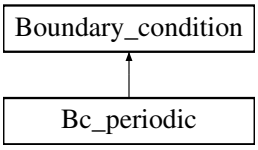
- Headers/libboundaryconditions/bc_neumann.hpp
- Sources/libboundaryconditions/bc_neumann.cpp

4.4 Bc_periodic Class Reference

Periodic condition.

`#include <bc_periodic.hpp>`

Inheritance diagram for Bc_periodic:



Public Member Functions

- Bc_periodic (Parameters &, TAB &, int, int)

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)

  Calculates boundary condition.
- virtual ~Bc_periodic ()

  Destructor.

Public Member Functions inherited from Boundary_condition

- Boundary_condition (Parameters &)

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)=0

  Function to be specified in each boundary condition.
- SCALAR get_hbound () const

  Gives the water height on the fictive cell.
- SCALAR get_unormbound () const

  Gives the normal velocity of the flow on the fictive cell.
- SCALAR get_utanbound () const

  Gives the tangential velocity of the flow on the fictive cell.
- virtual ~Boundary_condition ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Boundary_condition

- const int NXCELL
- const int NYCELL
- SCALAR hbound
- SCALAR unormbound
- SCALAR utanbound
- SCALAR unormfix
- map< int, int > Choice_Lbound
- map< int, int > Choice_Rbound
- map< int, int > Choice_Bbound
- map< int, int > Choice_Tbound

4.4.1 Detailed Description

Periodic condition.

Class that computes the periodic boundary condition

Definition at line 74 of file bc_periodic.hpp.

4.4.2 Constructor & Destructor Documentation

Bc_periodic()

```
Bc_periodic::Bc_periodic (
            Parameters & par,
            TAB & z,
            int n1,
            int n2 )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.
in,out	*z*	vector that represents the topography with suitable values on the fictive cells.

Definition at line 61 of file bc_periodic.cpp.

~Bc_periodic()

```
Bc_periodic::~Bc_periodic ( )  [virtual]
```

Destructor.

Definition at line 144 of file bc_periodic.cpp.

4.4.3 Member Function Documentation

calcul()

```
void Bc_periodic::calcul (
            SCALAR hin,
            SCALAR unorm_in,
            SCALAR utan_in,
            SCALAR hfix,
            SCALAR qfix,
            SCALAR hin_oppbound,
            SCALAR unorm_in_oppbound,
            SCALAR utan_in_oppbound,
            SCALAR time,
            int n1,
            int n2 ) [virtual]
```

Calculates boundary condition.
The velocity and water height are fixed to have the same behavior at each bound of the domain.

Parameters

in	*hin*	water height of the first cell inside the domain (unused).
in	*unorm_in*	normal velocity of the first cell inside the domain (unused).
in	*utan_in*	tangential velocity of the first cell inside the domain (unused).
in	*hfix*	fixed (imposed) value of the water height (unused).
in	*qfix*	fixed (imposed) value of the discharge (unused).
in	*hin_oppbound*	value of the water height of the first cell inside the domain at the opposite bound.
in	*unorm_in_oppbound*	value of the normal velocity of the first cell inside the domain at the opposite bound.
in	*utan_in_oppbound*	value of the tangential velocity of the first cell inside the domain at the opposite bound.
in	*time*	current time (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary (unused).
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary (unused).

Modifies

Boundary_condition::hbound water height on the fictive cell.
Boundary_condition::unormbound normal velocity on the fictive cell.
Boundary_condition::utanbound tangential velocity on the fictive cell.

Implements Boundary_condition.
Definition at line 108 of file bc_periodic.cpp.
The documentation for this class was generated from the following files:

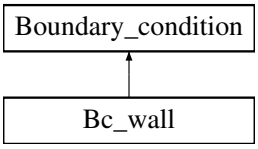
- Headers/libboundaryconditions/bc_periodic.hpp
- Sources/libboundaryconditions/bc_periodic.cpp

4.5 Bc_wall Class Reference

Wall condition.

```
#include <bc_wall.hpp>
```

Inheritance diagram for Bc_wall:



Public Member Functions

- Bc_wall (Parameters &, TAB &, int, int)

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)

  Calculates the boundary condition.
- virtual ∼Bc_wall ()

  Destructor.

Public Member Functions inherited from Boundary_condition

- Boundary_condition (Parameters &)

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)=0

  Function to be specified in each boundary condition.
- SCALAR get_hbound () const

  Gives the water height on the fictive cell.
- SCALAR get_unormbound () const

  Gives the normal velocity of the flow on the fictive cell.
- SCALAR get_utanbound () const

  Gives the tangential velocity of the flow on the fictive cell.
- virtual ∼Boundary_condition ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Boundary_condition

- const int NXCELL
- const int NYCELL
- SCALAR hbound
- SCALAR unormbound
- SCALAR utanbound
- SCALAR unormfix
- map< int, int > Choice_Lbound
- map< int, int > Choice_Rbound
- map< int, int > Choice_Bbound
- map< int, int > Choice_Tbound

4.5.1 Detailed Description

Wall condition.

Class that computes the wall boundary condition.

Definition at line 71 of file bc_wall.hpp.

4.5.2 Constructor & Destructor Documentation

Bc_wall()

```
Bc_wall::Bc_wall (
            Parameters & par,
            TAB & z,
            int n1,
            int n2 )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.
in,out	*z*	vector that represents the topography with suitable values on the fictive cells.

Definition at line 61 of file bc_wall.cpp.

~Bc_wall()

```
Bc_wall::~Bc_wall ( )  [virtual]
```

Destructor.

Definition at line 141 of file bc_wall.cpp.

4.5.3 Member Function Documentation

calcul()

```
void Bc_wall::calcul (
            SCALAR hin,
            SCALAR unorm_in,
            SCALAR utan_in,
            SCALAR hfix,
            SCALAR qfix,
            SCALAR hin_oppbound,
            SCALAR unorm_in_oppbound,
            SCALAR utan_in_oppbound,
            SCALAR time,
            int n1,
            int n2 )  [virtual]
```

Calculates the boundary condition.

Parameters

in	*hin*	water height of the first cell inside the domain.
in	*unorm_in*	normal velocity of the first cell inside the domain.
in	*utan_in*	tangential velocity of the first cell inside the domain.
in	*hfix*	fixed (imposed) value of the water height (unused).
in	*qfix*	fixed (imposed) value of the discharge (unused).
in	*hin_oppbound*	value of the water height of the first cell inside the domain at the opposite bound (unused).
in	*unorm_in_oppbound*	value of the normal velocity of the first cell inside the domain at the opposite bound (unused).
in	*utan_in_oppbound*	value of the tangential velocity of the first cell inside the domain at the opposite bound (unused).
in	*time*	current time (unused).
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary (unused).
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary (unused).

Modifies

Boundary_condition::hbound water height on the fictive cell.
Boundary_condition::unormbound normal velocity on the fictive cell.
Boundary_condition::utanbound tangential velocity on the fictive cell.

Implements Boundary_condition.
Definition at line 106 of file bc_wall.cpp.
The documentation for this class was generated from the following files:

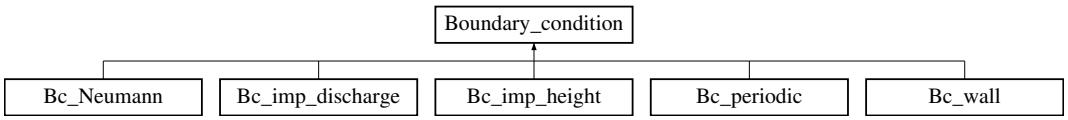
- Headers/libboundaryconditions/bc_wall.hpp
- Sources/libboundaryconditions/bc_wall.cpp

4.6 Boundary_condition Class Reference

Boundary condition.

`#include <boundary_condition.hpp>`

Inheritance diagram for Boundary_condition:



Public Member Functions

- Boundary_condition (Parameters &)

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)=0

  Function to be specified in each boundary condition.
- SCALAR get_hbound () const

Gives the water height on the fictive cell.

- SCALAR get_unormbound () const

  Gives the normal velocity of the flow on the fictive cell.

- SCALAR get_utanbound () const

  Gives the tangential velocity of the flow on the fictive cell.

- virtual ~Boundary_condition ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- SCALAR hbound
- SCALAR unormbound
- SCALAR utanbound
- SCALAR unormfix
- map< int, int > Choice_Lbound
- map< int, int > Choice_Rbound
- map< int, int > Choice_Bbound
- map< int, int > Choice_Tbound

4.6.1 Detailed Description

Boundary condition.

Class that contains all the common declarations for the boundary conditions.

Todo Add time and space dependancy in the boundary conditions.

Improve boundary conditions at the second order for the wall and periodic conditions.

Take into account source terms (friction and topography) in the boundary conditions, see Le Roux [2001], Bristeau and Coussin [2001].

Definition at line 74 of file boundary_condition.hpp.

4.6.2 Constructor & Destructor Documentation

Boundary_condition()

```
Boundary_condition::Boundary_condition (
            Parameters & par )
```

Constructor.

Defines the number of cells.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file boundary_condition.cpp.

~Boundary_condition()

```
Boundary_condition::~Boundary_condition ( )  [virtual]
```

Destructor.

Definition at line 106 of file boundary_condition.cpp.

4.6.3 Member Function Documentation

calcul()

```
virtual void Boundary_condition::calcul (
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            int ,
            int  )  [pure virtual]
```

Function to be specified in each boundary condition.

Implemented in Bc_imp_discharge, Bc_imp_height, Bc_Neumann, Bc_periodic, and Bc_wall.

get_hbound()

```
SCALAR Boundary_condition::get_hbound ( ) const
```

Gives the water height on the fictive cell.

Returns

Boundary_condition::hbound water height on the fictive cell.

Definition at line 75 of file boundary_condition.cpp.

get_unormbound()

```
SCALAR Boundary_condition::get_unormbound ( ) const
```

Gives the normal velocity of the flow on the fictive cell.

Returns

Boundary_condition::unormbound normal velocity on the fictive cell.

Definition at line 85 of file boundary_condition.cpp.

get_utanbound()

```
SCALAR Boundary_condition::get_utanbound ( ) const
```

Gives the tangential velocity of the flow on the fictive cell.

Returns

Boundary_condition::utanbound tangential velocity on the fictive cell.

Definition at line 95 of file boundary_condition.cpp.

4.6.4 Member Data Documentation

Choice_Bbound

```
map<int,int> Boundary_condition::Choice_Bbound [protected]
```

Definition at line 112 of file boundary_condition.hpp.

Choice_Lbound

```
map<int,int> Boundary_condition::Choice_Lbound [protected]
```

Definition at line 110 of file boundary_condition.hpp.

Choice_Rbound

```
map<int,int> Boundary_condition::Choice_Rbound [protected]
```

Definition at line 111 of file boundary_condition.hpp.

Choice_Tbound

```
map<int,int> Boundary_condition::Choice_Tbound [protected]
```

Definition at line 113 of file boundary_condition.hpp.

hbound

```
SCALAR Boundary_condition::hbound [protected]
```

Water height on the fictive cell, to be specified in each boundary condition.

Definition at line 102 of file boundary_condition.hpp.

NXCELL

```
const int Boundary_condition::NXCELL [protected]
```

Number of cells (in space) in the x direction.

Definition at line 98 of file boundary_condition.hpp.

NYCELL

```
const int Boundary_condition::NYCELL [protected]
```

Number of cells (in space) in the y direction.

Definition at line 100 of file boundary_condition.hpp.

unormbound

`SCALAR Boundary_condition::unormbound [protected]`

Normal velocity on the fictive cell, to be specified in each boundary condition.

Definition at line 104 of file boundary_condition.hpp.

unormfix

`SCALAR Boundary_condition::unormfix [protected]`

Imposed value of the velocity from Parameters::left_imp_discharge (or Parameters::right_imp_discharge, Parameters::bottom_imp_discharge, Parameters::top_imp_discharge) and Parameters::left_imp_h (or Parameters::right_imp_h, Parameters::bottom_imp_h, Parameters::top_imp_h).

Definition at line 108 of file boundary_condition.hpp.

utanbound

`SCALAR Boundary_condition::utanbound [protected]`

Tangential velocity on the fictive cell, to be specified in each boundary condition.

Definition at line 106 of file boundary_condition.hpp.

The documentation for this class was generated from the following files:

- Headers/libboundaryconditions/boundary_condition.hpp
- Sources/libboundaryconditions/boundary_condition.cpp

4.7 Choice_condition Class Reference

Choice of boundary condition.

`#include <choice_condition.hpp>`

Public Member Functions

- Choice_condition (map< int, int > &, Parameters &, TAB &, int, int)

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, int, int)

  Calculates the boundary condition.
- SCALAR get_hbound ()

  Gives the water height on the fictive cell.
- SCALAR get_unormbound ()

  Gives the normal velocity of the flow on the fictive cell.
- SCALAR get_utanbound ()

  Gives the tangential velocity of the flow on the fictive cell.
- virtual ~Choice_condition ()

  Destructor.
- void setXY (const int, const int)

  set index (i,j) of mesh to compute boundary condition
- void setChoice (map< int, int > &)

  set the type of boundary condition

4.7.1 Detailed Description

Choice of boundary condition.

Class that calls the boundary condition chosen in the parameters file.

Definition at line 95 of file choice_condition.hpp.

4.7.2 Constructor & Destructor Documentation

Choice_condition()

```
Choice_condition::Choice_condition (
            map< int, int > & choice,
            Parameters & par,
            TAB & z,
            int n1,
            int n2 )
```

Constructor.

Defines the boundary condition from the value given in the parameters file.

Parameters

in	*choice*	integer that correspond to the chosen boundary condition.
in	*par*	parameter, contains all the values from the parameters file.
in	*z*	array that represents the topography.
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Definition at line 62 of file choice_condition.cpp.

~Choice_condition()

```
Choice_condition::~Choice_condition ( ) [virtual]
```

Destructor.

Definition at line 240 of file choice_condition.cpp.

4.7.3 Member Function Documentation

calcul()

```
void Choice_condition::calcul (
            SCALAR hin,
            SCALAR unorm_in,
            SCALAR utan_in,
            SCALAR hfix,
            SCALAR qfix,
            SCALAR hin_oppbound,
            SCALAR unorm_in_oppbound,
            SCALAR utan_in_oppbound,
            SCALAR time,
```

```
int n1,
int n2 )
```

Calculates the boundary condition.
Calls the calculation of the boundary condition.

Parameters

in	*hin*	water height of the first cell inside the domain.
in	*unorm_in*	normal velocity of the first cell inside the domain.
in	*utan_in*	tangential velocity of the first cell inside the domain.
in	*hfix*	fixed (imposed) value of the water height.
in	*qfix*	fixed (imposed) value of the discharge.
in	*hin_oppbound*	value of the water height of the first cell inside the domain at the opposite bound.
in	*unorm_in_oppbound*	value of the normal velocity of the first cell inside the domain at the opposite bound.
in	*utan_in_oppbound*	value of the tangential velocity of the first cell inside the domain at the opposite bound.
in	*time*	current time.
in	*n1*	integer to specify whether it is the left (-1) or the right (1) boundary.
in	*n2*	integer to specify whether it is the bottom (-1) or the top (1) boundary.

Definition at line 86 of file choice_condition.cpp.

get_hbound()

```
SCALAR Choice_condition::get_hbound ( )
```

Gives the water height on the fictive cell.
Calls the function to get the water height on the fictive cell.

Returns

Boundary_condition::hbound water height on the fictive cell for the chosen boundary condition.

Definition at line 166 of file choice_condition.cpp.

get_unormbound()

```
SCALAR Choice_condition::get_unormbound ( )
```

Gives the normal velocity of the flow on the fictive cell.
Calls the function to get the normal velocity on the fictive cell.

Returns

Boundary_condition::unormbound normal velocity on the fictive cell for the chosen boundary condition.

Definition at line 177 of file choice_condition.cpp.

get_utanbound()

```
SCALAR Choice_condition::get_utanbound ( )
```

Gives the tangential velocity of the flow on the fictive cell.
Calls the function to get the tangential velocity on the fictive cell.

Returns

Boundary_condition::utanbound tangential velocity on the fictive cell for the chosen boundary condition.

Definition at line 191 of file choice_condition.cpp.

setChoice()

```
void Choice_condition::setChoice (
            map< int, int > & vChoice_bound )
```

set the type of boundary condition
Calls the function to define the container containing the set of choices

Parameters

in,out	*vChoice_bound*	container containing the choice of boundary conditions

Definition at line 229 of file choice_condition.cpp.

setXY()

```
void Choice_condition::setXY (
            const int i,
            const int j )
```

set index (i,j) of mesh to compute boundary condition
Calls the function to define the point indices where the boundary condition is evaluated

Parameters

in	*i*	index of the point in the x direction
in	*j*	index of the point in the y direction

Warning

***: ERROR: the indexes i and j are too big/small.

Definition at line 203 of file choice_condition.cpp.
The documentation for this class was generated from the following files:

- Headers/libboundaryconditions/choice_condition.hpp
- Sources/libboundaryconditions/choice_condition.cpp

4.8 Choice_flux Class Reference

Choice of numerical flux.

```
#include <choice_flux.hpp>
```

Public Member Functions

- Choice_flux (int)
  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)
  Calculates the numerical flux.

- void set_tx (SCALAR)

  Sets the variable Flux::tx.
- SCALAR get_f1 ()

  Gives the first component of the numerical flux.
- SCALAR get_f2 ()

  Gives the second component of the numerical flux.
- SCALAR get_f3 ()

  Gives the third component of the numerical flux.
- SCALAR get_cfl ()

  Gives the CFL value.
- virtual ~Choice_flux ()

  Destructor.

4.8.1 Detailed Description

Choice of numerical flux.

Class that calls the numerical flux chosen in the parameters file.

Definition at line 93 of file choice_flux.hpp.

4.8.2 Constructor & Destructor Documentation

Choice_flux()

```
Choice_flux::Choice_flux (
            int choice )
```

Constructor.

Defines the numerical flux from the value given in the parameters file.

Parameters

in	*choice*	integer that correspond to the chosen numerical flux.

Definition at line 61 of file choice_flux.cpp.

~Choice_flux()

```
Choice_flux::~Choice_flux ( )  [virtual]
```

Destructor.

Definition at line 161 of file choice_flux.cpp.

4.8.3 Member Function Documentation

calcul()

```
void Choice_flux::calcul (
            SCALAR h_L,
            SCALAR u_L,
            SCALAR v_L,
```

```
SCALAR h_R,
SCALAR u_R,
SCALAR v_R )
```

Calculates the numerical flux.
Calls the calculation of the numerical flux.

Parameters

in	*h_L*	water height at the left of the interface where the flux is calculated.
in	*u_L*	velocity (in the x direction) at the left of the interface where the flux is calculated.
in	*v_L*	velocity (in the y direction) at the left of the interface where the flux is calculated.
in	*h_R*	water height at the right of the interface where the flux is calculated.
in	*u_R*	velocity (in the x direction) at the right of the interface where the flux is calculated.
in	*v_R*	velocity (in the y direction) at the right of the interface where the flux is calculated.

Definition at line 90 of file choice_flux.cpp.

get_cfl()

```
SCALAR Choice_flux::get_cfl ( )
```

Gives the CFL value.
Calls the function to get the value of the CFL.

Returns

Flux::cfl value of the CFL.

Definition at line 150 of file choice_flux.cpp.

get_f1()

```
SCALAR Choice_flux::get_f1 ( )
```

Gives the first component of the numerical flux.
Calls the function to get the first component of the numerical flux.

Returns

Flux::f1 first component of the numerical flux.

Definition at line 117 of file choice_flux.cpp.

get_f2()

```
SCALAR Choice_flux::get_f2 ( )
```

Gives the second component of the numerical flux.
Calls the function to get the second component of the numerical flux.

Returns

Flux::f2 second component of the numerical flux.

Definition at line 128 of file choice_flux.cpp.

get_f3()

```
SCALAR Choice_flux::get_f3 ( )
```

Gives the third component of the numerical flux.
Calls the function to get the third component of the numerical flux.

Returns

Flux::f3 third component of the numerical flux.

Definition at line 139 of file choice_flux.cpp.

set_tx()

```
void Choice_flux::set_tx (
            SCALAR tx )
```

Sets the variable Flux::tx.
Calls the setting of the value given in parameter to the variable ***tx***.

Parameters

in	*tx*	value of dt/dx.

Definition at line 106 of file choice_flux.cpp.
The documentation for this class was generated from the following files:

- Headers/libflux/choice_flux.hpp
- Sources/libflux/choice_flux.cpp

4.9 Choice_friction Class Reference

Choice of friction law.

```
#include <choice_friction.hpp>
```

Public Member Functions

- Choice_friction (Parameters &)

  Constructor.
- void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)

  Calculates the friction term.
- TAB get_q1mod ()

  Gives the discharge in the first direction modified by the friction term.
- TAB get_q2mod ()

  Gives the discharge in the second direction modified by the friction term.
- void calculSf (const TAB &, const TAB &, const TAB &)

  Calculates the explicit friction term. It will be used for computations with erosion.

- TAB get_Sf1 ()

  Gives the explicit friction term in the first direction.
- TAB get_Sf2 ()

  Gives the explicit friction term in the second direction.
- virtual ~Choice_friction ()

  Destructor.

4.9.1 Detailed Description

Choice of friction law.

Class that calls the fricition law chosen in the parameters file.

Definition at line 89 of file choice_friction.hpp.

4.9.2 Constructor & Destructor Documentation

Choice_friction()

```
Choice_friction::Choice_friction (
            Parameters & par )
```

Constructor.

Defines the friction law from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file choice_friction.cpp.

~Choice_friction()

```
Choice_friction::~Choice_friction ( )  [virtual]
```

Destructor.

Definition at line 161 of file choice_friction.cpp.

4.9.3 Member Function Documentation

calcul()

```
void Choice_friction::calcul (
            const TAB & uold,
            const TAB & vold,
            const TAB & hnew,
            const TAB & q1new,
            const TAB & q2new,
            SCALAR dt )
```

Calculates the friction term.

Calls the calculation of the friction law.

Parameters

in	*uold*	velocity in the first direction at the previous time (n if you are calculating the $n+1$th time step).
in	*vold*	velocity in the second direction at the previous time (n if you are calculating the $n+1$th time step).
in	*hnew*	water height after the Shallow-Water computation (without friction).
in	*q1new*	discharge in the first direction after the Shallow-Water computation (without friction).
in	*q2new*	discharge in the second direction after the Shallow-Water computation (without friction).
in	*dt*	time step.

Note

The friction only affects the discharge.

Definition at line 85 of file choice_friction.cpp.

calculSf()

```
void Choice_friction::calculSf (
            const TAB & h,
            const TAB & u,
            const TAB & v )
```

Calculates the explicit friction term. It will be used for computations with erosion.
Calls the calculation of the explicit friction law.

Parameters

in	*h*	water height.
in	*u*	velocity in the first direction.
in	*v*	velocity in the second direction.

Note

This term will be used to compute erosion.

Definition at line 125 of file choice_friction.cpp.

get_q1mod()

```
TAB Choice_friction::get_q1mod ( )
```

Gives the discharge in the first direction modified by the friction term.
Calls the function to get the discharge in the first direction modified by the friction term.

Returns

Friction::q1mod discharge in the first direction modified by the friction term.

Definition at line 102 of file choice_friction.cpp.

get_q2mod()

`TAB Choice_friction::get_q2mod ( )`

Gives the discharge in the second direction modified by the friction term.

Calls the function to get the discharge in the second direction modified by the friction term.

Returns

Friction::q2mod discharge in the second direction modified by the friction term.

Definition at line 113 of file choice_friction.cpp.

get_Sf1()

`TAB Choice_friction::get_Sf1 ( )`

Gives the explicit friction term in the first direction.

Calls the function to get the explicit friction term in the first direction.

Returns

Friction::Sf1 explicit friction term in the first direction.

Definition at line 139 of file choice_friction.cpp.

get_Sf2()

`TAB Choice_friction::get_Sf2 ( )`

Gives the explicit friction term in the second direction.

Calls the function to get the explicit friction term in the second direction.

Returns

Friction::Sf2 explicit friction term in the second direction.

Definition at line 150 of file choice_friction.cpp.

The documentation for this class was generated from the following files:

- Headers/libfrictions/choice_friction.hpp
- Sources/libfrictions/choice_friction.cpp

4.10 Choice_infiltration Class Reference

Choice of infiltration law.

`#include <choice_infiltration.hpp>`

Public Member Functions

- Choice_infiltration (Parameters &)
  Constructor.
- void calcul (const TAB &, const TAB &, const SCALAR)
  Performs the computation of the modified water height and the infiltrated volume.
- virtual ~Choice_infiltration ()
  Destructor.
- TAB get_hmod ()
  Gives the value of the modified water height.
- TAB get_Vin ()
  Gives the value of the infiltrated volume.

4.10.1 Detailed Description

Choice of infiltration law.

Class that calls the infiltration chosen in the parameters file.

Definition at line 82 of file choice_infiltration.hpp.

4.10.2 Constructor & Destructor Documentation

Choice_infiltration()

```
Choice_infiltration::Choice_infiltration (
            Parameters & par )
```

Constructor.

Defines the friction law from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 61 of file choice_infiltration.cpp.

~Choice_infiltration()

```
Choice_infiltration::~Choice_infiltration ( )  [virtual]
```

Destructor.

Definition at line 112 of file choice_infiltration.cpp.

4.10.3 Member Function Documentation

calcul()

```
void Choice_infiltration::calcul (
            const TAB & h,
            const TAB & Vin,
            const SCALAR dt )
```

Performs the computation of the modified water height and the infiltrated volume.

Calls the computation of infiltration.

Parameters

in	*h*	water height.
in	*Vin*	infiltrated volume.
in	*dt*	time step.

Definition at line 80 of file choice_infiltration.cpp.

get_hmod()

```
TAB Choice_infiltration::get_hmod ( )
```

Gives the value of the modified water height.

Returns

The value hmod Infiltration::hmod.

Definition at line 92 of file choice_infiltration.cpp.

get_Vin()

```
TAB Choice_infiltration::get_Vin ( )
```

Gives the value of the infiltrated volume.

Returns

The value Vin Infiltration::Vin.

Definition at line 102 of file choice_infiltration.cpp.

The documentation for this class was generated from the following files:

- Headers/librain_infiltration/choice_infiltration.hpp
- Sources/librain_infiltration/choice_infiltration.cpp

4.11 Choice_init_huv Class Reference

Choice of initialization for h and U=(u,v)

```
#include <choice_init_huv.hpp>
```

Public Member Functions

- Choice_init_huv (Parameters &)
  Constructor.
- void initialization (TAB &, TAB &, TAB &)
  Performs the initialization.
- virtual ~Choice_init_huv ()
  Destructor.

4.11.1 Detailed Description

Choice of initialization for h and U=(u,v)

Class that calls the initialization of the water height and of the velocity chosen in the parameters file.

Definition at line 93 of file choice_init_huv.hpp.

4.11.2 Constructor & Destructor Documentation

Choice_init_huv()

```
Choice_init_huv::Choice_init_huv (
            Parameters & par )
```

Constructor.

Defines the initialization of the water height and of the velocity from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file choice_init_huv.cpp.

~Choice_init_huv()

```
Choice_init_huv::~Choice_init_huv ( )  [virtual]
```

Destructor.
Definition at line 101 of file choice_init_huv.cpp.

4.11.3 Member Function Documentation

initialization()

```
void Choice_init_huv::initialization (
            TAB & h,
            TAB & u,
            TAB & v )
```

Performs the initialization.
Calls the initialization of the water height and of the velocity.

Parameters

in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.

Definition at line 88 of file choice_init_huv.cpp.
The documentation for this class was generated from the following files:

- Headers/libinitializations/choice_init_huv.hpp
- Sources/libinitializations/choice_init_huv.cpp

4.12 Choice_init_topo Class Reference

Choice of initialization for the topography.

```
#include <choice_init_topo.hpp>
```

Public Member Functions

- Choice_init_topo (Parameters &)

  Constructor.
- void initialization (TAB &)

  Performs the initialization.
- virtual ~Choice_init_topo ()

  Destructor.

4.12.1 Detailed Description

Choice of initialization for the topography.

Class that calls the initialization of the topography chosen in the parameters file.

Definition at line 84 of file choice_init_topo.hpp.

4.12.2 Constructor & Destructor Documentation

Choice_init_topo()

```
Choice_init_topo::Choice_init_topo (
            Parameters & par )
```

Constructor.

Defines the initialization of the topography from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file choice_init_topo.cpp.

~Choice_init_topo()

```
Choice_init_topo::~Choice_init_topo ( )  [virtual]
```

Destructor.

Definition at line 93 of file choice_init_topo.cpp.

4.12.3 Member Function Documentation

initialization()

```
void Choice_init_topo::initialization (
            TAB & topo )
```

Performs the initialization.

Calls the initialization of the topography.

Parameters

in	*topo*	topography.

Definition at line 82 of file choice_init_topo.cpp.

The documentation for this class was generated from the following files:

- Headers/libinitializations/choice_init_topo.hpp
- Sources/libinitializations/choice_init_topo.cpp

4.13 Choice_limiter Class Reference

Choice of slope limiter.

```
#include <choice_limiter.hpp>
```

Public Member Functions

- Choice_limiter (int)

  Constructor.
- void calcul (SCALAR, SCALAR)

  Calculates the slope limiter.
- SCALAR get_rec () const

  Gives the reconstructed value.
- virtual ∼Choice_limiter ()

  Destructor.

4.13.1 Detailed Description

Choice of slope limiter.

Class that calls the slope limiter chosen in the parameters file.

Definition at line 84 of file choice_limiter.hpp.

4.13.2 Constructor & Destructor Documentation

Choice_limiter()

```
Choice_limiter::Choice_limiter (
            int choice )
```

Constructor.

Defines the slope limiter from the value given in the parameters file.

Parameters

in	*choice*	integer that corresponds to the chosen slope limiter.

Definition at line 60 of file choice_limiter.cpp.

∼Choice_limiter()

```
Choice_limiter::∼Choice_limiter ( ) [virtual]
```

Destructor.

Definition at line 104 of file choice_limiter.cpp.

4.13.3 Member Function Documentation

calcul()

```
void Choice_limiter::calcul (
            SCALAR a,
            SCALAR b )
```

Calculates the slope limiter.

Calls the calculation of the slope limiter.

Parameters

in	*a*	slope on the left of the cell.
in	*b*	slope on the right of the cell.

Definition at line 81 of file choice_limiter.cpp.

get_rec()

`SCALAR Choice_limiter::get_rec ( ) const`

Gives the reconstructed value.

Calls the function to get the reconstructed value.

Returns

Limiter::rec reconstructed value for the chosen slope limiter.

Definition at line 93 of file choice_limiter.cpp.

The documentation for this class was generated from the following files:

- Headers/liblimitations/choice_limiter.hpp
- Sources/liblimitations/choice_limiter.cpp

4.14 Choice_output Class Reference

Choice of output format.

`#include <choice_output.hpp>`

Public Member Functions

- Choice_output (Parameters &)
  Constructor.
- void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)
  Save the current time.
- void check_vol (const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &) const
  Saves the infiltrated and rain volumes.
- void result (const SCALAR &, const clock_t &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const int &, const SCALAR &) const
  Saves global values.
- void initial (const TAB &, const TAB &, const TAB &, const TAB &) const
  Saves the initial time.
- void final (const TAB &, const TAB &, const TAB &, const TAB &) const
  Saves the final time.
- SCALAR boundaries_flux (const SCALAR &, const TAB &, const TAB &, const SCALAR &, const SCALAR &, const int &, const int &) const
  Saves the cumulated fluxes on the boundaries.
- void boundaries_flux_LR (const SCALAR &, const TAB &) const
  Saves the fluxes on the left and right boundaries.
- void boundaries_flux_BT (const SCALAR &, const TAB &) const
  Saves the fluxes on the bottom and top boundaries.
- virtual ~Choice_output ()
  Destructor.

4.14.1 Detailed Description

Choice of output format.

From the value of the corresponding parameter, calls the savings in the chosen format.

Definition at line 84 of file choice_output.hpp.

4.14.2 Constructor & Destructor Documentation

Choice_output()

```
Choice_output::Choice_output (
            Parameters & par )
```

Constructor.

Defines the output format from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file choice_output.cpp.

~Choice_output()

```
Choice_output::~Choice_output ( )  [virtual]
```

Destructor.

Definition at line 202 of file choice_output.cpp.

4.14.3 Member Function Documentation

boundaries_flux()

```
SCALAR Choice_output::boundaries_flux (
            const SCALAR & time,
            const TAB & flux_u,
            const TAB & flux_v,
            const SCALAR & dt,
            const SCALAR & dt_first,
            const int & ORDER,
            const int & verif ) const
```

Saves the cumulated fluxes on the boundaries.

Calls the saving of the cumulative flux on the boundaries.

Parameters

in	*time*	current time.
in	*flux_u*	flux on the left and right boundaries (m^2/s).
in	*flux_v*	flux on the bottom and top boundaries (m^2/s).
in	*dt*	current time step.
in	*dt_first*	previous time step.
in	*ORDER*	order of scheme.

Parameters

in	*verif*	parameter to know if we removed the computation with the previous time step (dt_first).

Definition at line 161 of file choice_output.cpp.

boundaries_flux_BT()

```
void Choice_output::boundaries_flux_BT (
            const SCALAR & time,
            const TAB & BT_flux ) const
```

Saves the fluxes on the bottom and top boundaries.
Calls the saving of the fluxes on the top and bottom boundaries.

Parameters

in	*time*	current time.
in	*BT_flux*	flux on the bottom and tom boundaries (m^2/s).

Definition at line 190 of file choice_output.cpp.

boundaries_flux_LR()

```
void Choice_output::boundaries_flux_LR (
            const SCALAR & time,
            const TAB & LR_flux ) const
```

Saves the fluxes on the left and right boundaries.
Calls the saving of the fluxes on the left and right boundaries.

Parameters

in	*time*	current time.
in	*LR_flux*	flux on the left and right boundaries (m^2/s).

Definition at line 178 of file choice_output.cpp.

check_vol()

```
void Choice_output::check_vol (
            const SCALAR & time,
            const SCALAR & dt,
            const SCALAR & Vol_rain_tot,
            const SCALAR & Vol_inf,
            const SCALAR & Vol_of,
            const SCALAR & Vol_bound_tot ) const
```

Saves the infiltrated and rain volumes.
Calls the saving of the infiltrated and rain volumes.

Parameters

in	*time*	current time.
in	*dt*	time step.

Parameters

in	*Vol_rain_tot*	total rain volume.
in	*Vol_inf*	volume of infiltrated water.
in	*Vol_of*	volume of overland flow.
in	*Vol_bound_tot*	total volume of water at the boundary.

Definition at line 97 of file choice_output.cpp.

final()

```
void Choice_output::final (
            const TAB & z,
            const TAB & h,
            const TAB & u,
            const TAB & v ) const
```

Saves the final time.

Calls the saving of the final time.

Parameters

in	*z*	topography.
in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.

Definition at line 146 of file choice_output.cpp.

initial()

```
void Choice_output::initial (
            const TAB & z,
            const TAB & h,
            const TAB & u,
            const TAB & v ) const
```

Saves the initial time.

Calls the saving of the inital time.

Parameters

in	*z*	topography.
in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.

Definition at line 131 of file choice_output.cpp.

result()

```
void Choice_output::result (
            const SCALAR & time,
```

```
            const clock_t & cpu,
            const SCALAR & Vol_rain,
            const SCALAR & Vol_inf,
            const SCALAR & Vol_of,
            const SCALAR & FROUDE,
            const int & NBITER,
            const SCALAR & vol_output ) const
```

Saves global values.
Calls the saving of the global values.

Parameters

in	*time*	elapsed time.
in	*cpu*	CPU time.
in	*Vol_rain*	total rain volume.
in	*Vol_inf*	total volume of infiltrated water.
in	*Vol_of*	total volume of overland flow.
in	*FROUDE*	mean Froude number (in space) at the final time.
in	*NBITER*	number of time steps.
in	*vol_output*	total outflow volume at the boundary.

Definition at line 113 of file choice_output.cpp.

write()

```
void Choice_output::write (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const TAB & z,
            const SCALAR & time )
```

Save the current time.
Calls the saving of the current time.

Parameters

in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.
in	*z*	topography.
in	*time*	value of the current time.

Definition at line 82 of file choice_output.cpp.
The documentation for this class was generated from the following files:

- Headers/libsave/choice_output.hpp
- Sources/libsave/choice_output.cpp

4.15 Choice_rain Class Reference

Choice of initialization for the rain.

```
#include <choice_rain.hpp>
```

Public Member Functions

- Choice_rain (Parameters &)

  Constructor.
- void rain_func (SCALAR, TAB &)

  Performs the initialization filling up the table of the rain intensity.
- virtual ~Choice_rain ()

  Destructor.

4.15.1 Detailed Description

Choice of initialization for the rain.

Class that calls the initialization of the rain chosen in the parameters file.

Definition at line 84 of file choice_rain.hpp.

4.15.2 Constructor & Destructor Documentation

Choice_rain()

```
Choice_rain::Choice_rain (
            Parameters & par )
```

Constructor.

Defines the initialization of the rain from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file choice_rain.cpp.

~Choice_rain()

```
Choice_rain::~Choice_rain ( )  [virtual]
```

Destructor.

Definition at line 94 of file choice_rain.cpp.

4.15.3 Member Function Documentation

rain_func()

```
void Choice_rain::rain_func (
            SCALAR time,
            TAB & Tab_rain )
```

Performs the initialization filling up the table of the rain intensity.

Calls the initialization of the rain.

Parameters

in	*time*	current time.
in	*Tab_rain*	rain.

Definition at line 83 of file choice_rain.cpp.
The documentation for this class was generated from the following files:

- Headers/librain_infiltration/choice_rain.hpp
- Sources/librain_infiltration/choice_rain.cpp

4.16 Choice_reconstruction Class Reference

Choice of reconstruction.

`#include <choice_reconstruction.hpp>`

Public Member Functions

- Choice_reconstruction (Parameters &, TAB &)

  Constructor.
- void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)

  Calculates the second order reconstruction in space.
- virtual ~Choice_reconstruction ()

  Destructor.

4.16.1 Detailed Description

Choice of reconstruction.

Class that calls the reconstruction chosen in the parameters file.
Definition at line 86 of file choice_reconstruction.hpp.

4.16.2 Constructor & Destructor Documentation

Choice_reconstruction()

```
Choice_reconstruction::Choice_reconstruction (
            Parameters & par,
            TAB & z )
```

Constructor.

Defines the reconstruction from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.
in	*z*	array that represents the topography.

Definition at line 60 of file choice_reconstruction.cpp.

~Choice_reconstruction()

```
Choice_reconstruction::~Choice_reconstruction ( ) [virtual]
```

Destructor.

Definition at line 112 of file choice_reconstruction.cpp.

4.16.3 Member Function Documentation

calcul()

```
void Choice_reconstruction::calcul (
            TAB & h,
            TAB & u,
            TAB & v,
            TAB & z,
            TAB & delzc1,
            TAB & delzc2,
            TAB & delz1,
            TAB & delz2,
            TAB & h1r,
            TAB & u1r,
            TAB & v1r,
            TAB & h1l,
            TAB & u1l,
            TAB & v1l,
            TAB & h2r,
            TAB & u2r,
            TAB & v2r,
            TAB & h2l,
            TAB & u2l,
            TAB & v2l )
```

Calculates the second order reconstruction in space.

Calls the calculation of the second order reconstruction in space.

Parameters

in	*h*	water height.
in	*u*	velocity of the flow in the first direction.
in	*v*	velocity of the flow in the second direction.
in	*z*	topography.
out	*delzc1*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the first direction.
out	*delzc2*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the second direction.
out	*delz1*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the first direction.
out	*delz2*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the seond direction.
out	*h1r*	reconstructed water height on the right of the cell in the first direction.
out	*u1r*	first component of the reconstructed velocity on the right of the cell in the first direction.
out	*v1r*	second component of the reconstructed velocity on the right of the cell in the first direction.
out	*h1l*	reconstructed water height on the left of the cell in the first direction.
out	*u1l*	first component of the reconstructed velocity on the left of the cell in the first direction.
out	*v1l*	second component of the reconstructed velocity on the left of the cell in the first direction.
out	*h2r*	reconstructed water height on the right of the cell in the second direction.
out	*u2r*	first component of the reconstructed velocity on the right of the cell in the second direction.

Parameters

out	*v2r*	second component of the reconstructed velocity on the right of the cell in the second direction.
out	*h2l*	reconstructed water height on the left of the cell in the second direction.
out	*u2l*	first component of the reconstructed velocity on the left of the cell in the second direction.
out	*v2l*	second component of the reconstructed velocity on the left of the cell in the second direction.

Definition at line 82 of file choice_reconstruction.cpp.
The documentation for this class was generated from the following files:

- Headers/libreconstructions/choice_reconstruction.hpp
- Sources/libreconstructions/choice_reconstruction.cpp

4.17 Choice_save_specific_points Class Reference

Choice of the output of the specific points.

```
#include <choice_save_specific_points.hpp>
```

Public Member Functions

- Choice_save_specific_points (Parameters &)
  Constructor.
- void save (const TAB &, const TAB &, const TAB &, const SCALAR)
  Save the current time.
- virtual ~Choice_save_specific_points ()
  Destructor.

4.17.1 Detailed Description

Choice of the output of the specific points.
From the value of the corresponding parameter, calls the savings of the corresponding specific points.
Definition at line 84 of file choice_save_specific_points.hpp.

4.17.2 Constructor & Destructor Documentation

Choice_save_specific_points()

```
Choice_save_specific_points::Choice_save_specific_points (
            Parameters & par )
```

Constructor.
Defines the number (0,1,>1) of points to save from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file choice_save_specific_points.cpp.

~Choice_save_specific_points()

```
Choice_save_specific_points::~Choice_save_specific_points ( )  [virtual]
```

Destructor.

Definition at line 96 of file choice_save_specific_points.cpp.

4.17.3 Member Function Documentation

save()

```
void Choice_save_specific_points::save (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const SCALAR time )
```

Save the current time.

Calls the saving of the current time.

Parameters

in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.
in	*time*	value of the current time.

Definition at line 82 of file choice_save_specific_points.cpp.

The documentation for this class was generated from the following files:

- Headers/libsave/choice_save_specific_points.hpp
- Sources/libsave/choice_save_specific_points.cpp

4.18 Choice_scheme Class Reference

Choice of numerical scheme.

```
#include <choice_scheme.hpp>
```

Public Member Functions

- Choice_scheme (Parameters &)

  Constructor.
- void calcul ()

  Performs the scheme.
- virtual ~Choice_scheme ()

  Destructor.

4.18.1 Detailed Description

Choice of numerical scheme.

Class that calls the numerical scheme chosen in the parameters file.

Definition at line 81 of file choice_scheme.hpp.

4.18.2 Constructor & Destructor Documentation

Choice_scheme()

```
Choice_scheme::Choice_scheme (
            Parameters & par )
```

Constructor.

Defines the numerical scheme from the value given in the parameters file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file choice_scheme.cpp.

~Choice_scheme()

```
Choice_scheme::~Choice_scheme ( )  [virtual]
```

Destructor.

Definition at line 88 of file choice_scheme.cpp.

4.18.3 Member Function Documentation

calcul()

```
void Choice_scheme::calcul ( )
```

Performs the scheme.

Calls the computation of the solution.

Definition at line 78 of file choice_scheme.cpp.

The documentation for this class was generated from the following files:

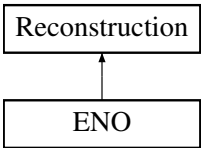
- Headers/libschemes/choice_scheme.hpp
- Sources/libschemes/choice_scheme.cpp

4.19 ENO Class Reference

ENO recontruction

`#include <eno.hpp>`

Inheritance diagram for ENO:



Public Member Functions

- ENO (Parameters &, TAB &)

  Constructor.
- void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)

  Calculates the reconstruction in space.
- ∼ENO ()

  Destructor.

Public Member Functions inherited from Reconstruction

- Reconstruction (Parameters &, TAB &)

  Constructor.
- virtual void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)=0

  Function to be specified in each reconstruction.
- virtual ∼Reconstruction ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Reconstruction

- const int NXCELL
- const int NYCELL
- TAB z1r
- TAB z1l
- TAB z2r
- TAB z2l
- TAB delta_z1
- TAB delta_z2
- Choice_limiter ∗ limiter

4.19.1 Detailed Description

ENO recontruction

Class that computes ENO reconstruction in space.

Definition at line 73 of file eno.hpp.

4.19.2 Constructor & Destructor Documentation

ENO()

```
ENO::ENO (
            Parameters & par,
            TAB & z )
```

Constructor.

Initializations.

Parameters

in	*par*	parameter, contains all the values from the parameters file.
in	*z*	topography.

Definition at line 59 of file eno.cpp.

~ENO()

```
ENO::~ENO ( )
```

Destructor.

Definition at line 370 of file eno.cpp.

4.19.3 Member Function Documentation

calcul()

```
void ENO::calcul (
            TAB & h,
            TAB & u,
            TAB & v,
            TAB & z,
            TAB & delzc1,
            TAB & delzc2,
            TAB & delz1,
            TAB & delz2,
            TAB & h1r,
            TAB & u1r,
            TAB & v1r,
            TAB & h1l,
            TAB & u1l,
            TAB & v1l,
            TAB & h2r,
            TAB & u2r,
            TAB & v2r,
            TAB & h2l,
            TAB & u2l,
            TAB & v2l ) [virtual]
```

Calculates the reconstruction in space.

Calls the calculation of the second order reconstruction in space, with ENO formulation, see Harten et al. [1986], Harten et al. [1987], Shu and Osher [1988], Bouchut [2004], Bouchut [2007] .

Parameters

in	*h*	water height.
in	*u*	velocity of the flow in the first direction.
in	*v*	velocity of the flow in the second direction.
in	*z*	topography.
out	*delzc1*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the first direction.

Parameters

out	*delzc2*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the second direction.
out	*delz1*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the first direction.
out	*delz2*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the seond direction.
out	*h1r*	reconstructed water height on the right of the cell in the first direction.
out	*u1r*	first component of the reconstructed velocity on the right of the cell in the first direction.
out	*v1r*	second component of the reconstructed velocity on the right of the cell in the first direction.
out	*h1l*	reconstructed water height on the left of the cell in the first direction.
out	*u1l*	first component of the reconstructed velocity on the left of the cell in the first direction.
out	*v1l*	second component of the reconstructed velocity on the left of the cell in the first direction.
out	*h2r*	reconstructed water height on the right of the cell in the second direction.
out	*u2r*	first component of the reconstructed velocity on the right of the cell in the second direction.
out	*v2r*	second component of the reconstructed velocity on the right of the cell in the second direction.
out	*h2l*	reconstructed water height on the left of the cell in the second direction.
out	*u2l*	first component of the reconstructed velocity on the left of the cell in the second direction.
out	*v2l*	second component of the reconstructed velocity on the left of the cell in the second direction.

Implements Reconstruction.
Definition at line 88 of file eno.cpp.
The documentation for this class was generated from the following files:

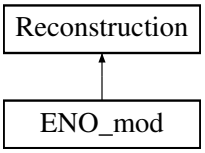
- Headers/libreconstructions/eno.hpp
- Sources/libreconstructions/eno.cpp

4.20 ENO_mod Class Reference

Modified ENO recontruction.

`#include <eno_mod.hpp>`

Inheritance diagram for ENO_mod:



Public Member Functions

- ENO_mod (Parameters &, TAB &)

  Constructor.
- void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)

  Calculates the reconstruction in space.
- ~ENO_mod ()

  Destructor.

Public Member Functions inherited from Reconstruction

- Reconstruction (Parameters &, TAB &)

  Constructor.
- virtual void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)=0

  Function to be specified in each reconstruction.
- virtual ~Reconstruction ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Reconstruction

- const int NXCELL
- const int NYCELL
- TAB z1r
- TAB z1l
- TAB z2r
- TAB z2l
- TAB delta_z1
- TAB delta_z2
- Choice_limiter * limiter

4.20.1 Detailed Description

Modified ENO recontruction.

Class that computes the modified ENO reconstruction in space.

Definition at line 73 of file eno_mod.hpp.

4.20.2 Constructor & Destructor Documentation

ENO_mod()

```
ENO_mod::ENO_mod (
            Parameters & par,
            TAB & z )
```

Constructor.

Initializations.

Parameters

in	*par*	parameter, contains all the values from the parameters file.
in	*z*	topography.

Definition at line 60 of file eno_mod.cpp.

~ENO_mod()

```
ENO_mod::~ENO_mod ( )
```

Destructor.

Definition at line 398 of file eno_mod.cpp.

4.20.3 Member Function Documentation

calcul()

```
void ENO_mod::calcul (
            TAB & h,
            TAB & u,
            TAB & v,
            TAB & z,
            TAB & delzc1,
            TAB & delzc2,
            TAB & delz1,
            TAB & delz2,
            TAB & h1r,
            TAB & u1r,
            TAB & v1r,
            TAB & h1l,
            TAB & u1l,
            TAB & v1l,
            TAB & h2r,
            TAB & u2r,
            TAB & v2r,
            TAB & h2l,
            TAB & u2l,
            TAB & v2l ) [virtual]
```

Calculates the reconstruction in space.

Calls the calculation of the second order reconstruction in space, with a modified ENO formulation, see Bouchut [2004], Bouchut [2007].

Parameters

in	*h*	water height.
in	*u*	velocity of the flow in the first direction.
in	*v*	velocity of the flow in the second direction.
in	*z*	topography.
out	*delzc1*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the first direction.
out	*delzc2*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the second direction.
out	*delz1*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the first direction.
out	*delz2*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the seond direction.
out	*h1r*	reconstructed water height on the right of the cell in the first direction.
out	*u1r*	first component of the reconstructed velocity on the right of the cell in the first direction.
out	*v1r*	second component of the reconstructed velocity on the right of the cell in the first direction.
out	*h1l*	reconstructed water height on the left of the cell in the first direction.
out	*u1l*	first component of the reconstructed velocity on the left of the cell in the first direction.
out	*v1l*	second component of the reconstructed velocity on the left of the cell in the first direction.

Parameters

out	*h2r*	reconstructed water height on the right of the cell in the second direction.
out	*u2r*	first component of the reconstructed velocity on the right of the cell in the second direction.
out	*v2r*	second component of the reconstructed velocity on the right of the cell in the second direction.
out	*h2l*	reconstructed water height on the left of the cell in the second direction.
out	*u2l*	first component of the reconstructed velocity on the left of the cell in the second direction.
out	*v2l*	second component of the reconstructed velocity on the left of the cell in the second direction.

Implements Reconstruction.
Definition at line 85 of file eno_mod.cpp.
The documentation for this class was generated from the following files:

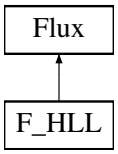
- Headers/libreconstructions/eno_mod.hpp
- Sources/libreconstructions/eno_mod.cpp

4.21 F_HLL Class Reference

HLL flux.

`#include <f_hll.hpp>`

Inheritance diagram for F_HLL:



Public Member Functions

- F_HLL ()

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)

  Calculates the numerical flux.
- virtual ~F_HLL ()

  Destructor.

Public Member Functions inherited from Flux

- Flux ()

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)=0

  Function to be specified in each numerical flux.
- void set_tx (SCALAR)

  Sets the variable Flux::tx.
- SCALAR get_f1 () const

  Gives the first component of the numerical flux.
- SCALAR get_f2 () const

  Gives the second component of the numerical flux.

- SCALAR get_f3 () const

  Gives the third component of the numerical flux.
- SCALAR get_cfl () const

  Gives the CFL value.
- virtual ~Flux ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Flux

- SCALAR f1
- SCALAR f2
- SCALAR f3
- SCALAR cfl
- SCALAR tx

4.21.1 Detailed Description

HLL flux.

Class that computes HLL numerical flux.

Definition at line 72 of file f_hll.hpp.

4.21.2 Constructor & Destructor Documentation

F_HLL()

```
F_HLL::F_HLL ( )
```

Constructor.

Definition at line 60 of file f_hll.cpp.

~F_HLL()

```
F_HLL::~F_HLL ( )  [virtual]
```

Destructor.

Definition at line 135 of file f_hll.cpp.

4.21.3 Member Function Documentation

calcul()

```
void F_HLL::calcul (
            SCALAR h_L,
            SCALAR u_L,
            SCALAR v_L,
            SCALAR h_R,
            SCALAR u_R,
            SCALAR v_R )  [virtual]
```

Calculates the numerical flux.

Recall that this is reduced to a one-dimensional computation along the normal of the mesh edge.
If the water heights on the two sides are small or $c_1 \approx c_2 \approx 0$, there is no water. Else, HLL formulation is used (see Bouchut [2004]):

$$\mathscr{F}(U_L,U_R) = \begin{cases} F(U_L) & \text{if} \quad 0 < c_1 (\leq c_2), \\ \dfrac{c_2 F(U_L) - c_1 F(U_R)}{c_2 - c_1} + \dfrac{c_1 c_2}{c_2 - c_1}(U_R - U_L) & \text{if} \quad c_1 < 0 < c_2, \\ F(U_R) & \text{if} \quad (c_1 \leq) c_2 < 0, \end{cases}$$

with

$$c_1 = \inf_{U=U_L,U_R} \left(\inf_{j\in\{1,2\}} \lambda_j(U) \right) \text{ and } c_2 = \sup_{U=U_L,U_R} \left(\sup_{j\in\{1,2\}} \lambda_j(U) \right),$$

where $\lambda_1(U) = u - \sqrt{gh}$ and $\lambda_2(U) = u + \sqrt{gh}$ are the eigenvalues of the Shallow Water system, $U = {}^t(h, hu, hv)$ and $F(U) = {}^t(hu, hu^2 + gh^2/2, hv^2)$.

Parameters

in	*h*↩ *_L*	water height at the left of the interface where the flux is calculated.
in	*u*↩ *_L*	velocity (in the x direction) at the left of the interface where the flux is calculated.
in	*v*↩ *_L*	velocity (in the y direction) at the left of the interface where the flux is calculated.
in	*h*↩ *_R*	water height at the right of the interface where the flux is calculated.
in	*u*↩ *_R*	velocity (in the x direction) at the right of the interface where the flux is calculated.
in	*v*↩ *_R*	velocity (in the y direction) at the right of the interface where the flux is calculated.

Modifies

Flux::f1 first component of the numerical flux.
Flux::f2 second component of the numerical flux.
Flux::f3 third component of the numerical flux.
Flux::cfl value of the CFL.

Implements Flux.
Definition at line 63 of file f_hll.cpp.
The documentation for this class was generated from the following files:

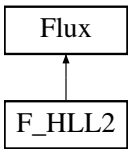
- Headers/libflux/f_hll.hpp
- Sources/libflux/f_hll.cpp

4.22 F_HLL2 Class Reference

HLL2 flux.

```
#include <f_hll2.hpp>
```

Inheritance diagram for F_HLL2:



Public Member Functions

- F_HLL2 ()

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)

  Calculates the numerical flux.
- virtual ~F_HLL2 ()

  Destructor.

Public Member Functions inherited from Flux

- Flux ()

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)=0

  Function to be specified in each numerical flux.
- void set_tx (SCALAR)

  Sets the variable Flux::tx.
- SCALAR get_f1 () const

  Gives the first component of the numerical flux.
- SCALAR get_f2 () const

  Gives the second component of the numerical flux.
- SCALAR get_f3 () const

  Gives the third component of the numerical flux.
- SCALAR get_cfl () const

  Gives the CFL value.
- virtual ~Flux ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Flux

- SCALAR f1
- SCALAR f2
- SCALAR f3
- SCALAR cfl
- SCALAR tx

4.22.1 Detailed Description

HLL2 flux.

Class that computes HLL numerical flux with a reduced formulation.

Definition at line 71 of file f_hll2.hpp.

4.22.2 Constructor & Destructor Documentation

F_HLL2()

```
F_HLL2::F_HLL2 ( )
```

Constructor.

Definition at line 61 of file f_hll2.cpp.

~F_HLL2()

```
F_HLL2::~F_HLL2 ( )  [virtual]
```

Destructor.

Definition at line 120 of file f_hll2.cpp.

4.22.3 Member Function Documentation

calcul()

```
void F_HLL2::calcul (
            SCALAR h_L,
            SCALAR u_L,
            SCALAR v_L,
            SCALAR h_R,
            SCALAR u_R,
            SCALAR v_R )  [virtual]
```

Calculates the numerical flux.

Recall that this is reduced to a one-dimensional computation along the normal of the mesh edge.
If the water heights on the two sides are small or $c_1 \approx c_2 \approx 0$, there is no water. Else, HLL reduced formulation is used (see Batten et al. [1997]):

$$\mathscr{F}(U_L,U_R) = t_1 F(U_R) + t_2 F(U_L) - t_3 (U_R - U_L),$$

with

$$t_1 = \frac{\min(c_2,0) - \min(c_1,0)}{c_2 - c_1}, \quad t_2 = 1 - t_1, \quad t_3 = \frac{c_2|c_1| - c_1|c_2|}{2(c_2 - c_1)},$$

$$c_1 = \inf_{U=U_L,U_R} \left(\inf_{j\in\{1,2\}} \lambda_j(U) \right) \text{ and } c_2 = \sup_{U=U_L,U_R} \left(\sup_{j\in\{1,2\}} \lambda_j(U) \right),$$

where $\lambda_1(U) = u - \sqrt{gh}$ and $\lambda_2(U) = u + \sqrt{gh}$ are the eigenvalues of the Shallow Water system, $U = {}^t(h, hu, hv)$ and $F(U) = {}^t(hu, hu^2 + gh^2/2, hv^2)$.

Parameters

in	h↩_L	water height at the left of the interface where the flux is calculated.
in	u↩_L	velocity (in the x direction) at the left of the interface where the flux is calculated.
in	v↩_L	velocity (in the y direction) at the left of the interface where the flux is calculated.
in	h↩_R	water height at the right of the interface where the flux is calculated.

Parameters

in	u_R	velocity (in the x direction) at the right of the interface where the flux is calculated.
in	v_R	velocity (in the y direction) at the right of the interface where the flux is calculated.

Modifies

Flux::f1 first component of the numerical flux.
Flux::f2 second component of the numerical flux.
Flux::f3 third component of the numerical flux.
Flux::cfl value of the CFL.

Implements Flux.
Definition at line 65 of file f_hll2.cpp.
The documentation for this class was generated from the following files:

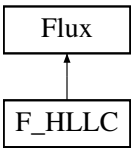
- Headers/libflux/f_hll2.hpp
- Sources/libflux/f_hll2.cpp

4.23 F_HLLC Class Reference

HLLC flux.

`#include <f_hllc.hpp>`

Inheritance diagram for F_HLLC:



Public Member Functions

- F_HLLC ()
  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)
  Calculates the numerical flux.
- virtual ~F_HLLC ()
  Destructor.

Public Member Functions inherited from Flux

- Flux ()
  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)=0
  Function to be specified in each numerical flux.
- void set_tx (SCALAR)
  Sets the variable Flux::tx.
- SCALAR get_f1 () const
  Gives the first component of the numerical flux.

- SCALAR get_f2 () const

  Gives the second component of the numerical flux.
- SCALAR get_f3 () const

  Gives the third component of the numerical flux.
- SCALAR get_cfl () const

  Gives the CFL value.
- virtual ~Flux ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Flux

- SCALAR f1
- SCALAR f2
- SCALAR f3
- SCALAR cfl
- SCALAR tx

4.23.1 Detailed Description

HLLC flux.

Class that computes HLLC numerical flux.

Definition at line 73 of file f_hllc.hpp.

4.23.2 Constructor & Destructor Documentation

F_HLLC()

```
F_HLLC::F_HLLC ( )
```

Constructor.

Definition at line 61 of file f_hllc.cpp.

~F_HLLC()

```
F_HLLC::~F_HLLC ( )  [virtual]
```

Destructor.

Definition at line 159 of file f_hllc.cpp.

4.23.3 Member Function Documentation

calcul()

```
void F_HLLC::calcul (
            SCALAR h_L,
            SCALAR u_L,
            SCALAR v_L,
            SCALAR h_R,
```

```
SCALAR u_R,
SCALAR v_R ) [virtual]
```

Calculates the numerical flux.

The HLLC approximate Riemann solver is a modification of the basic HLL scheme to account for the contact and shear waves (see Toro [2001]).
If the water heights on the two sides are small or $c_1 \approx c_2 \approx 0$, there is no water. Else, HLL formulation is used (see Bouchut [2004]):

$$\mathscr{F}(U_L,U_R) = \begin{cases} F(U_L) & \text{if} \quad 0 < c_1 (\leq c_2), \\ \dfrac{c_2 F(U_L) - c_1 F(U_R)}{c_2 - c_1} + \dfrac{c_1 c_2}{c_2 - c_1}(U_R - U_L) & \text{if} \quad c_1 < 0 < c_2, \\ F(U_R) & \text{if} \quad (c_1 \leq) c_2 < 0, \end{cases}$$

with

$$c_1 = \inf_{U=U_L,U_R} \left(\inf_{j\in\{1,2\}} |\lambda_j(U)| \right) \text{ and } c_2 = \sup_{U=U_L,U_R} \left(\sup_{j\in\{1,2\}} |\lambda_j(U)| \right),$$

where $\lambda_1(U) = u - \sqrt{gh}$ and $\lambda_2(U) = u + \sqrt{gh}$ are the eigenvalues of the Shallow Water system, $U = {}^t(h, hu, hv)$ and $F(U) = {}^t(hu, hu^2 + gh^2/2, hv^2)$.
If we consider the approximate flux HLL

$$F_{i+\frac{1}{2}} = \begin{pmatrix} F^1_{i+\frac{1}{2}} \\ F^2_{i+\frac{1}{2}} \\ F^3_{i+\frac{1}{2}} \end{pmatrix}$$

then to obtain the HLLC solver just add the following expression for the third component

$$F^3_{i+\frac{1}{2}} = \begin{cases} F^1_{i+\frac{1}{2}} * V_L & \text{if} \quad 0 \leq u_*, \\ F^1_{i+\frac{1}{2}} * V_R & \text{if} \quad u_* < 0, \end{cases}$$

Where

$$u_* = \frac{c_1 h_R (u_R - c_2) - c_2 h_L (u_L - c_1)}{h_R (u_R - c_2) - h_L (u_L - c_1)}$$

Parameters

in	*h_L	water height at the left of the interface where the flux is calculated.
in	*u_L	velocity (in the x direction) at the left of the interface where the flux is calculated.
in	*v_L	velocity (in the y direction) at the left of the interface where the flux is calculated.
in	*h_R	water height at the right of the interface where the flux is calculated.
in	*u_R	velocity (in the x direction) at the right of the interface where the flux is calculated.
in	*v_R	velocity (in the y direction) at the right of the interface where the flux is calculated.

Modifies

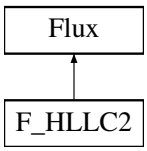
Flux::f1 first component of the numerical flux.
Flux::f2 second component of the numerical flux.
Flux::f3 third component of the numerical flux.
Flux::cfl value of the CFL.

Implements Flux.
Definition at line 64 of file f_hllc.cpp.
The documentation for this class was generated from the following files:

- Headers/libflux/f_hllc.hpp
- Sources/libflux/f_hllc.cpp

4.24 F_HLLC2 Class Reference

HLLC flux.
`#include <f_hllc2.hpp>`
Inheritance diagram for F_HLLC2:



Public Member Functions

- F_HLLC2 ()
  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)
  Calculates the numerical flux.
- virtual ~F_HLLC2 ()
  Destructor.

Public Member Functions inherited from Flux

- Flux ()
  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)=0
  Function to be specified in each numerical flux.
- void set_tx (SCALAR)
  Sets the variable Flux::tx.
- SCALAR get_f1 () const
  Gives the first component of the numerical flux.
- SCALAR get_f2 () const
  Gives the second component of the numerical flux.
- SCALAR get_f3 () const
  Gives the third component of the numerical flux.
- SCALAR get_cfl () const
  Gives the CFL value.
- virtual ~Flux ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Flux

- SCALAR f1
- SCALAR f2
- SCALAR f3
- SCALAR cfl
- SCALAR tx

4.24.1 Detailed Description

HLLC flux.

Class that computes HLLC numerical flux with a reduced formulation.

Definition at line 72 of file f_hllc2.hpp.

4.24.2 Constructor & Destructor Documentation

F_HLLC2()

```
F_HLLC2::F_HLLC2 ( )
```

Constructor.

Definition at line 62 of file f_hllc2.cpp.

~F_HLLC2()

```
F_HLLC2::~F_HLLC2 ( )  [virtual]
```

Destructor.

Definition at line 144 of file f_hllc2.cpp.

4.24.3 Member Function Documentation

calcul()

```
void F_HLLC2::calcul (
            SCALAR h_L,
            SCALAR u_L,
            SCALAR v_L,
            SCALAR h_R,
            SCALAR u_R,
            SCALAR v_R )  [virtual]
```

Calculates the numerical flux.

The HLLC approximate Riemann solver is a modification of the basic HLL scheme to account for the contact and shear waves (see Toro [2001]).

If the water heights on the two sides are small or $c_1 \approx c_2 \approx 0$, there is no water. Else, HLL reduced formulation is used (see Batten et al. [1997]):

$$\mathscr{F}(U_L,U_R) = t_1 F(U_R) + t_2 F(U_L) - t_3 (U_R - U_L),$$

with

$$t_1 = \frac{\min(c_2,0) - \min(c_1,0)}{c_2 - c_1}, \quad t_2 = 1 - t_1, \quad t_3 = \frac{c_2|c_1| - c_1|c_2|}{2(c_2 - c_1)},$$

$$c_1 = \inf_{U=U_L,U_R} \left(\inf_{j\in\{1,2\}} |\lambda_j(U)| \right) \text{ and } c_2 = \sup_{U=U_L,U_R} \left(\sup_{j\in\{1,2\}} |\lambda_j(U)| \right),$$

where $\lambda_1(U) = u - \sqrt{gh}$ and $\lambda_2(U) = u + \sqrt{gh}$ are the eigenvalues of the Shallow Water system, $U = {}^t(h, hu, hv)$ and $F(U) = {}^t(hu, hu^2 + gh^2/2, hv^2)$.
If we consider the approximate flux HLL

$$F_{i+\frac{1}{2}} = \begin{pmatrix} F^1_{i+\frac{1}{2}} \\ F^2_{i+\frac{1}{2}} \\ F^3_{i+\frac{1}{2}} \end{pmatrix}$$

then to obtain the HLLC solver just add the following expression for the third component

$$F^3_{i+\frac{1}{2}} = \begin{cases} F^1_{i+\frac{1}{2}} * V_L & \text{if} \quad 0 \leq u_*, \\ F^1_{i+\frac{1}{2}} * V_R & \text{if} \quad u_* < 0, \end{cases}$$

Where

$$u_* = \frac{c_1 h_R (u_R - c_2) - c_2 h_L (u_L - c_1)}{h_R (u_R - c_2) - h_L (u_L - c_1)}$$

Parameters

in	h_L	water height at the left of the interface where the flux is calculated.
in	u_L	velocity (in the x direction) at the left of the interface where the flux is calculated.
in	v_L	velocity (in the y direction) at the left of the interface where the flux is calculated.
in	h_R	water height at the right of the interface where the flux is calculated.
in	u_R	velocity (in the x direction) at the right of the interface where the flux is calculated.
in	v_R	velocity (in the y direction) at the right of the interface where the flux is calculated.

Modifies

Flux::f1 first component of the numerical flux.
Flux::f2 second component of the numerical flux.
Flux::f3 third component of the numerical flux.
Flux::cfl value of the CFL.

Implements Flux.
Definition at line 66 of file f_hllc2.cpp.
The documentation for this class was generated from the following files:

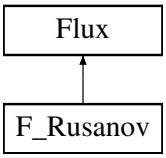
- Headers/libflux/f_hllc2.hpp
- Sources/libflux/f_hllc2.cpp

4.25 F_Rusanov Class Reference

Rusanov flux.

```
#include <f_rusanov.hpp>
```

Inheritance diagram for F_Rusanov:



Public Member Functions

- F_Rusanov ()

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)

  Calculates the numerical flux.
- virtual ~F_Rusanov ()

  Destructor.

Public Member Functions inherited from Flux

- Flux ()

  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)=0

  Function to be specified in each numerical flux.
- void set_tx (SCALAR)

  Sets the variable Flux::tx.
- SCALAR get_f1 () const

  Gives the first component of the numerical flux.
- SCALAR get_f2 () const

  Gives the second component of the numerical flux.
- SCALAR get_f3 () const

  Gives the third component of the numerical flux.
- SCALAR get_cfl () const

  Gives the CFL value.
- virtual ~Flux ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Flux

- SCALAR f1
- SCALAR f2
- SCALAR f3
- SCALAR cfl
- SCALAR tx

4.25.1 Detailed Description

Rusanov flux.

Class that computes Rusanov numerical flux.

Definition at line 71 of file f_rusanov.hpp.

4.25.2 Constructor & Destructor Documentation

F_Rusanov()

```
F_Rusanov::F_Rusanov ( )
```

Constructor.

Definition at line 59 of file f_rusanov.cpp.

~F_Rusanov()

```
F_Rusanov::~F_Rusanov ( )  [virtual]
```

Destructor.

Definition at line 106 of file f_rusanov.cpp.

4.25.3 Member Function Documentation

calcul()

```
void F_Rusanov::calcul (
            SCALAR h_L,
            SCALAR u_L,
            SCALAR v_L,
            SCALAR h_R,
            SCALAR u_R,
            SCALAR v_R )  [virtual]
```

Calculates the numerical flux.

Recall that this is reduced to a one-dimensional computation along the normal of the mesh edge.
If the water heights on the two sides are small, there is no water. Else, Rusanov formulation is used (see Bouchut [2004]):

$$\mathscr{F}(U_L,U_R) = \frac{F(U_L)+F(U_R)}{2} - c\frac{U_R-U_L}{2},$$

with $c = \max\left(|u_L|+\sqrt{gh_L}, |u_R|+\sqrt{gh_R}\right)$, $U = {}^t(h,hu,hv)$ and $F(U) = {}^t(hu, hu^2+gh^2/2, hv^2)$.

Parameters

in	h_L	water height at the left of the interface where the flux is calculated.
in	u_L	velocity (in the x direction) at the left of the interface where the flux is calculated.
in	v_L	velocity (in the y direction) at the left of the interface where the flux is calculated.
in	h_R	water height at the right of the interface where the flux is calculated.
in	u_R	velocity (in the x direction) at the right of the interface where the flux is calculated.
in	v_R	velocity (in the y direction) at the right of the interface where the flux is calculated.

Modifies

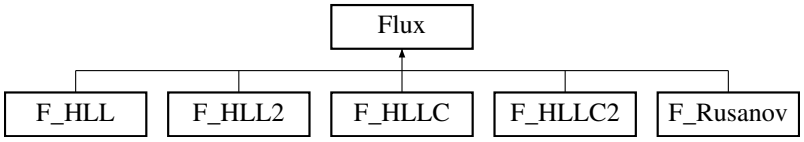
Flux::f1 first component of the numerical flux.
Flux::f2 second component of the numerical flux.
Flux::f3 third component of the numerical flux.
Flux::cfl value of the CFL.

Implements Flux.
Definition at line 63 of file f_rusanov.cpp.
The documentation for this class was generated from the following files:

- Headers/libflux/f_rusanov.hpp
- Sources/libflux/f_rusanov.cpp

4.26 Flux Class Reference

Numerical flux.
`#include <flux.hpp>`
Inheritance diagram for Flux:



Public Member Functions

- Flux ()
  Constructor.
- virtual void calcul (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR)=0
  Function to be specified in each numerical flux.
- void set_tx (SCALAR)
  Sets the variable Flux::tx.
- SCALAR get_f1 () const
  Gives the first component of the numerical flux.
- SCALAR get_f2 () const
  Gives the second component of the numerical flux.
- SCALAR get_f3 () const
  Gives the third component of the numerical flux.
- SCALAR get_cfl () const
  Gives the CFL value.
- virtual ~Flux ()
  Destructor.

Protected Attributes

- SCALAR f1
- SCALAR f2
- SCALAR f3
- SCALAR cfl
- SCALAR tx

4.26.1 Detailed Description

Numerical flux.

Class that contains all the common declarations for the numerical fluxes.

Definition at line 68 of file flux.hpp.

4.26.2 Constructor & Destructor Documentation

Flux()

```
Flux::Flux ( )
```

Constructor.

Definition at line 59 of file flux.cpp.

~Flux()

```
Flux::~Flux ( )  [virtual]
```

Destructor.

Definition at line 116 of file flux.cpp.

4.26.3 Member Function Documentation

calcul()

```
virtual void Flux::calcul (
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR ,
            SCALAR  )  [pure virtual]
```

Function to be specified in each numerical flux.

Implemented in F_HLL, F_HLL2, F_HLLC, F_HLLC2, and F_Rusanov.

get_cfl()

```
SCALAR Flux::get_cfl ( ) const
```

Gives the CFL value.

Returns

Flux::cfl value of the CFL.

Definition at line 106 of file flux.cpp.

get_f1()

```
SCALAR Flux::get_f1 ( ) const
```

Gives the first component of the numerical flux.

Returns

Flux::f1 first component of the numerical flux.

Definition at line 76 of file flux.cpp.

get_f2()

```
SCALAR Flux::get_f2 ( ) const
```

Gives the second component of the numerical flux.

Returns

Flux::f2 second component of the numerical flux.

Definition at line 86 of file flux.cpp.

get_f3()

```
SCALAR Flux::get_f3 ( ) const
```

Gives the third component of the numerical flux.

Returns

Flux::f3 third component of the numerical flux.

Definition at line 96 of file flux.cpp.

set_tx()

```
void Flux::set_tx (
            SCALAR tx )
```

Sets the variable Flux::tx.
Sets the value given in parameter to the variable ***tx***.

Parameters

in	*tx*	value of dt/dx.

Definition at line 66 of file flux.cpp.

4.26.4 Member Data Documentation

cfl

```
SCALAR Flux::cfl  [protected]
```

CFL value.
Definition at line 105 of file flux.hpp.

f1

`SCALAR Flux::f1 [protected]`

First component of the numerical flux.
Definition at line 99 of file flux.hpp.

f2

`SCALAR Flux::f2 [protected]`

Second component of the numerical flux.
Definition at line 101 of file flux.hpp.

f3

`SCALAR Flux::f3 [protected]`

Third component of the numerical flux.
Definition at line 103 of file flux.hpp.

tx

`SCALAR Flux::tx [protected]`

Value of dt/dx.
Definition at line 107 of file flux.hpp.
The documentation for this class was generated from the following files:

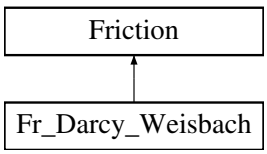
- Headers/libflux/flux.hpp
- Sources/libflux/flux.cpp

4.27 Fr_Darcy_Weisbach Class Reference

Darcy-Weisbach law.

`#include <fr_darcy_weisbach.hpp>`

Inheritance diagram for Fr_Darcy_Weisbach:



Public Member Functions

- Fr_Darcy_Weisbach (Parameters &)

  Constructor.
- void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)

  Calculates the Darcy-Weisbach friction term.
- void calculSf (const TAB &, const TAB &, const TAB &)

  Calculates the explicit Darcy-Weisbach friction term.

- virtual ~Fr_Darcy_Weisbach ()

  Destructor

Public Member Functions inherited from Friction

- Friction (Parameters &)

  Constructor.
- virtual void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)=0

  Function to be specified in each friction law.
- virtual TAB get_q1mod () const

  Gives the discharge in the first direction modified by the friction term.
- virtual TAB get_q2mod () const

  Gives the discharge in the second direction modified by the friction term.
- virtual void calculSf (const TAB &, const TAB &, const TAB &)=0

  Calculates the explicit friction term. It will be used for computations with erosion.
- virtual TAB get_Sf1 () const

  Gives the explicit friction term in the first direction.
- virtual TAB get_Sf2 () const

  Gives the explicit friction term in the second direction.
- virtual ~Friction ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Friction

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB q1mod
- TAB q2mod
- TAB Sf1
- TAB Sf2
- TAB Fric_tab

4.27.1 Detailed Description

Darcy-Weisbach law.

General formulation: $S_f = \frac{fU|U|}{8gh}$. This term is integrated in the code thanks to a semi-implicit method.

Definition at line 71 of file fr_darcy_weisbach.hpp.

4.27.2 Constructor & Destructor Documentation

Fr_Darcy_Weisbach()

```
Fr_Darcy_Weisbach::Fr_Darcy_Weisbach (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file fr_darcy_weisbach.cpp.

~Fr_Darcy_Weisbach()

```
Fr_Darcy_Weisbach::~Fr_Darcy_Weisbach ( )  [virtual]
```

Destructor

Definition at line 125 of file fr_darcy_weisbach.cpp.

4.27.3 Member Function Documentation

calcul()

```
void Fr_Darcy_Weisbach::calcul (
            const TAB & uold,
            const TAB & vold,
            const TAB & hnew,
            const TAB & q1new,
            const TAB & q2new,
            SCALAR dt )  [virtual]
```

Calculates the Darcy-Weisbach friction term.

General formulation (see Smith et al. [2007]): $S_f = \frac{fU|U|}{8gh}$. This term is integrated in the code thanks to a semi-implicit method:

$$q_{1/2_i}^{n+1} = \frac{q_{1/2_i}^*}{1 + dt\frac{f|U_i^n|}{8h_i^{n+1}}}$$

where f is the friction coefficient.

Parameters

in	*uold*	velocity in the first direction at the previous time (n if you are calculating the $n+1$th time step), first component of U_i^n in the above formula.
in	*vold*	velocity in the second direction at the previous time (n if you are calculating the $n+1$th time step), second component of U_i^n in the above formula.
in	*hnew*	water height after the Shallow-Water computation (without friction), denoted by h_i^{n+1} in the above formula.
in	*q1new*	discharge in the first direction after the Shallow-Water computation (without friction), denoted by $q_{1_i}^*$ in the above formula.
in	*q2new*	discharge in the second direction after the Shallow-Water computation (without friction), denoted by $q_{2_i}^*$ in the above formula.
in	*dt*	time step.

Modifies

Friction::q1mod discharge in the first direction modified by the friction term,
Friction::q2mod discharge in the second direction modified by the friction term.

Note

The friction only affects the discharge ($h^{n+1} = h^*$).

Implements Friction.
Definition at line 68 of file fr_darcy_weisbach.cpp.

calculSf()

```
void Fr_Darcy_Weisbach::calculSf (
            const TAB & h,
            const TAB & u,
            const TAB & v )  [virtual]
```

Calculates the explicit Darcy-Weisbach friction term.
Explicit friction term: $S_f = \frac{fU|U|}{8gh}$ where f is the friction coefficient.

Parameters

in	*h*	water height.
in	*u*	velocity in the first direction, first component of U in the above formula.
in	*v*	velocity in the second direction, second component of U in the above formula.

Modifies

Friction::Sf1 explicit friction term in the first direction,
Friction::Sf2 explicit friction term in the second direction.

Note

This explicit friction term will be used for erosion.

Implements Friction.
Definition at line 96 of file fr_darcy_weisbach.cpp.
The documentation for this class was generated from the following files:

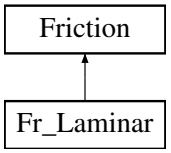
- Headers/libfrictions/fr_darcy_weisbach.hpp
- Sources/libfrictions/fr_darcy_weisbach.cpp

4.28 Fr_Laminar Class Reference

Laminar law.

`#include <fr_laminar.hpp>`

Inheritance diagram for Fr_Laminar:



Public Member Functions

- Fr_Laminar (Parameters &)

  Constructor.
- void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)

  Calculates the laminar friction term.
- void calculSf (const TAB &, const TAB &, const TAB &)

  Calculates the explicit Manning friction term.
- virtual ~Fr_Laminar ()

  Destructor.

Public Member Functions inherited from Friction

- Friction (Parameters &)

  Constructor.
- virtual void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)=0

  Function to be specified in each friction law.
- virtual TAB get_q1mod () const

  Gives the discharge in the first direction modified by the friction term.
- virtual TAB get_q2mod () const

  Gives the discharge in the second direction modified by the friction term.
- virtual void calculSf (const TAB &, const TAB &, const TAB &)=0

  Calculates the explicit friction term. It will be used for computations with erosion.
- virtual TAB get_Sf1 () const

  Gives the explicit friction term in the first direction.
- virtual TAB get_Sf2 () const

  Gives the explicit friction term in the second direction.
- virtual ~Friction ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Friction

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB q1mod
- TAB q2mod
- TAB Sf1
- TAB Sf2
- TAB Fric_tab

4.28.1 Detailed Description

Laminar law.

General formulation: $S_f = \nu \frac{1}{gh} \frac{U}{h}$. This term is integrated in the code thanks to an implicit method.

Definition at line 71 of file fr_laminar.hpp.

4.28.2 Constructor & Destructor Documentation

Fr_Laminar()

```
Fr_Laminar::Fr_Laminar (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 58 of file fr_laminar.cpp.

~Fr_Laminar()

```
Fr_Laminar::~Fr_Laminar ( )  [virtual]
```

Destructor.

Definition at line 128 of file fr_laminar.cpp.

4.28.3 Member Function Documentation

calcul()

```
void Fr_Laminar::calcul (
            const TAB & uold,
            const TAB & vold,
            const TAB & hnew,
            const TAB & q1new,
            const TAB & q2new,
            SCALAR dt )  [virtual]
```

Calculates the laminar friction term.

General formulation: $S_f = \nu \frac{1}{gh} \frac{U}{h}$. This term is integrated in the code thanks to an implicit method:

$$q_{1/2_i}^{n+1} = \frac{q_{1/2_i}^{*}}{1 + \nu dt \frac{1}{(h_i^{n+1})^2}}$$

where ν is the friction coefficient.

Parameters

in	*uold*	velocity in the first direction at the previous time (n if you are calculating the $n+1$th time step), first component of U_i^n in the above formula.
in	*vold*	velocity in the second direction at the previous time (n if you are calculating the $n+1$th time step), second component of U_i^n in the above formula.
in	*hnew*	water height after the Shallow-Water computation (without friction), denoted by h_i^{n+1} in the above formula.
in	*q1new*	discharge in the first direction after the Shallow-Water computation (without friction), denoted by $q_{1_i}^{*}$ in the above formula.

Parameters

in	*q2new*	discharge in the second direction after the Shallow-Water computation (without friction), denoted by $q_{2_i}^*$ in the above formula.
in	*dt*	time step.

Modifies

Friction::q1mod discharge in the first direction modified by the friction term,
Friction::q2mod discharge in the second direction modified by the friction term.

Note

The friction only affects the discharge ($h^{n+1} = h^*$).

Implements Friction.
Definition at line 67 of file fr_laminar.cpp.

calculSf()

```
void Fr_Laminar::calculSf (
            const TAB & h,
            const TAB & u,
            const TAB & v )  [virtual]
```

Calculates the explicit Manning friction term.
Explicit friction term: $S_f = \nu \frac{1}{gh} \frac{U}{h}$ where nu is the friction coefficient.

Parameters

in	*h*	water height.
in	*u*	velocity in the first direction, first component of U in the above formula.
in	*v*	velocity in the second direction, second component of U in the above formula.

Modifies

Friction::Sf1 explicit friction term in the first direction,
Friction::Sf2 explicit friction term in the second direction.

Note

This explicit friction term will be used for erosion.

Implements Friction.
Definition at line 98 of file fr_laminar.cpp.
The documentation for this class was generated from the following files:

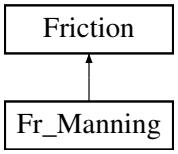
- Headers/libfrictions/fr_laminar.hpp
- Sources/libfrictions/fr_laminar.cpp

4.29 Fr_Manning Class Reference

Manning law.

`#include <fr_manning.hpp>`

Inheritance diagram for Fr_Manning:



Public Member Functions

- Fr_Manning (Parameters &)
  Constructor.
- void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)
  Calculates the Manning friction term.
- void calculSf (const TAB &, const TAB &, const TAB &)
  Calculates the explicit Manning friction term.
- virtual ~Fr_Manning ()
  Destructor.

Public Member Functions inherited from Friction

- Friction (Parameters &)
  Constructor.
- virtual void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)=0
  Function to be specified in each friction law.
- virtual TAB get_q1mod () const
  Gives the discharge in the first direction modified by the friction term.
- virtual TAB get_q2mod () const
  Gives the discharge in the second direction modified by the friction term.
- virtual void calculSf (const TAB &, const TAB &, const TAB &)=0
  Calculates the explicit friction term. It will be used for computations with erosion.
- virtual TAB get_Sf1 () const
  Gives the explicit friction term in the first direction.
- virtual TAB get_Sf2 () const
  Gives the explicit friction term in the second direction.
- virtual ~Friction ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Friction

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB q1mod
- TAB q2mod
- TAB Sf1
- TAB Sf2
- TAB Fric_tab

4.29.1 Detailed Description

Manning law.

General formulation: $S_f = c^2 \frac{U|U|}{h^{4/3}}$. This term is integrated in the code thanks to a semi-implicit method.

Definition at line 72 of file fr_manning.hpp.

4.29.2 Constructor & Destructor Documentation

Fr_Manning()

```
Fr_Manning::Fr_Manning (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file fr_manning.cpp.

~Fr_Manning()

```
Fr_Manning::~Fr_Manning ( )  [virtual]
```

Destructor.

Definition at line 126 of file fr_manning.cpp.

4.29.3 Member Function Documentation

calcul()

```
void Fr_Manning::calcul (
            const TAB & uold,
            const TAB & vold,
            const TAB & hnew,
            const TAB & q1new,
            const TAB & q2new,
            SCALAR dt )  [virtual]
```

Calculates the Manning friction term.

General formulation (see Smith et al. [2007]): $S_f = c^2 \frac{U|U|}{h^{4/3}}$. This term is integrated in the code thanks to a semi-implicit method:

$$q_{1/2_i}^{n+1} = \frac{q_{1/2_i}^*}{1 + dt \frac{c^2 g |U_i^n|}{(h_i^{n+1})^{4/3}}}$$

where c is the friction coefficient.

Parameters

in	*uold*	velocity in the first direction at the previous time (n if you are calculating the $n+1$th time step), first component of U_i^n in the above formula.

Parameters

in	*vold*	velocity in the second direction at the previous time (n if you are calculating the $n+1$th time step), second component of U_i^n in the above formula.
in	*hnew*	water height after the Shallow-Water computation (without friction), denoted by h_i^{n+1} in the above formula.
in	*q1new*	discharge in the first direction after the Shallow-Water computation (without friction), denoted by $q1_i^*$ in the above formula.
in	*q2new*	discharge in the second direction after the Shallow-Water computation (without friction), denoted by $q2_i^*$ in the above formula.
in	*dt*	time step.

Modifies

Friction::q1mod discharge in the first direction modified by the friction term,
Friction::q2mod discharge in the second direction modified by the friction term.

Note

The friction only affects the discharge ($h^{n+1} = h^*$).

Implements Friction.
Definition at line 68 of file fr_manning.cpp.

calculSf()

```
void Fr_Manning::calculSf (
            const TAB & h,
            const TAB & u,
            const TAB & v )  [virtual]
```

Calculates the explicit Manning friction term.
Explicit friction term: $S_f = c^2 \frac{U|U|}{h^{4/3}}$ where c is the friction coefficient.

Parameters

in	*h*	water height.
in	*u*	velocity in the first direction, first component of U in the above formula.
in	*v*	velocity in the second direction, second component of U in the above formula.

Modifies

Friction::Sf1 explicit friction term in the first direction,
Friction::Sf2 explicit friction term in the second direction.

Note

This explicit friction term will be used for erosion.

Implements Friction.
Definition at line 97 of file fr_manning.cpp.
The documentation for this class was generated from the following files:

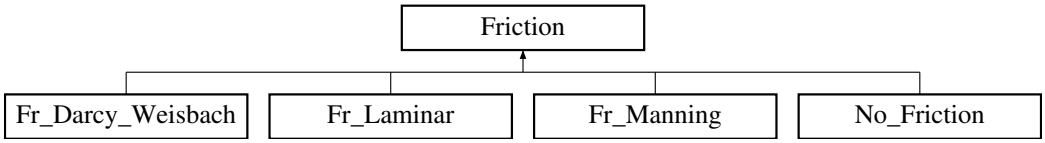
- Headers/libfrictions/fr_manning.hpp
- Sources/libfrictions/fr_manning.cpp

4.30 Friction Class Reference

Friction law

`#include <friction.hpp>`

Inheritance diagram for Friction:



Public Member Functions

- Friction (Parameters &)

  Constructor.
- virtual void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)=0

  Function to be specified in each friction law.
- virtual TAB get_q1mod () const

  Gives the discharge in the first direction modified by the friction term.
- virtual TAB get_q2mod () const

  Gives the discharge in the second direction modified by the friction term.
- virtual void calculSf (const TAB &, const TAB &, const TAB &)=0

  Calculates the explicit friction term. It will be used for computations with erosion.
- virtual TAB get_Sf1 () const

  Gives the explicit friction term in the first direction.
- virtual TAB get_Sf2 () const

  Gives the explicit friction term in the second direction.
- virtual ~Friction ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB q1mod
- TAB q2mod
- TAB Sf1
- TAB Sf2
- TAB Fric_tab

4.30.1 Detailed Description

Friction law

Class that contains all the common declarations for the friction law. The friction is computed with a semi-implicit method.

Definition at line 72 of file friction.hpp.

4.30.2 Constructor & Destructor Documentation

Friction()

```
Friction::Friction (
            Parameters & par )
```

Constructor.

Defines the number of cells, the space steps and initializes Friction::Fric_tab, Friction::q1mod, Friction::q2mod, Friction::Sf1, Friction::Sf2.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

***: ERROR: the value at the point ***.

Initialization of Friction

Definition at line 59 of file friction.cpp.

~Friction()

```
Friction::~Friction ( )  [virtual]
```

Destructor.

Deallocation of Friction::Fric_tab, Friction::q1mod, Friction::q2mod, Friction::Sf1, Friction::Sf2

Definition at line 161 of file friction.cpp.

4.30.3 Member Function Documentation

calcul()

```
virtual void Friction::calcul (
            const TAB & ,
            const TAB & ,
            const TAB & ,
            const TAB & ,
            const TAB & ,
            SCALAR  )  [pure virtual]
```

Function to be specified in each friction law.

Implemented in Fr_Darcy_Weisbach, Fr_Laminar, Fr_Manning, and No_Friction.

calculSf()

```
virtual void Friction::calculSf (
            const TAB & ,
            const TAB & ,
            const TAB &  )  [pure virtual]
```

Calculates the explicit friction term. It will be used for computations with erosion.

Implemented in Fr_Darcy_Weisbach, Fr_Laminar, Fr_Manning, and No_Friction.

get_q1mod()

`TAB Friction::get_q1mod ( ) const [virtual]`

Gives the discharge in the first direction modified by the friction term.

Returns

Friction::q1mod discharge in the first direction modified by the friction term.

Definition at line 121 of file friction.cpp.

get_q2mod()

`TAB Friction::get_q2mod ( ) const [virtual]`

Gives the discharge in the second direction modified by the friction term.

Returns

Friction::q2mod discharge in the second direction modified by the friction term.

Definition at line 131 of file friction.cpp.

get_Sf1()

`TAB Friction::get_Sf1 ( ) const [virtual]`

Gives the explicit friction term in the first direction.

Returns

Friction::Sf1 explicit friction term in the first direction.

Definition at line 141 of file friction.cpp.

get_Sf2()

`TAB Friction::get_Sf2 ( ) const [virtual]`

Gives the explicit friction term in the second direction.

Returns

Friction::Sf2 explicit friction term in the second direction.

Definition at line 151 of file friction.cpp.

4.30.4 Member Data Documentation

DX

`const SCALAR Friction::DX [protected]`

Space step in the first (x) direction.

Definition at line 106 of file friction.hpp.

DY

`const SCALAR Friction::DY [protected]`

Space step in the second (y) direction.

Definition at line 108 of file friction.hpp.

Fric_tab

`TAB Friction::Fric_tab [protected]`

Array that contains the friction coefficient by cell.

Definition at line 119 of file friction.hpp.

NXCELL

`const int Friction::NXCELL [protected]`

Number of cells in space in the first (x) direction.

Definition at line 102 of file friction.hpp.

NYCELL

`const int Friction::NYCELL [protected]`

Number of cells in space in the second (y) direction.

Definition at line 104 of file friction.hpp.

q1mod

`TAB Friction::q1mod [protected]`

Discharge in the first direction modified by the friction term.

Definition at line 111 of file friction.hpp.

q2mod

`TAB Friction::q2mod [protected]`

Discharge in the second direction modified by the friction term.

Definition at line 113 of file friction.hpp.

Sf1

`TAB Friction::Sf1 [protected]`

Explicit friction term in the first direction.

Definition at line 115 of file friction.hpp.

Sf2

`TAB Friction::Sf2 [protected]`

Explicit friction term in the second direction.

Definition at line 117 of file friction.hpp.

The documentation for this class was generated from the following files:

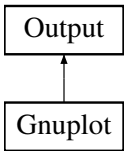
- Headers/libfrictions/friction.hpp
- Sources/libfrictions/friction.cpp

4.31 Gnuplot Class Reference

Gnuplot output

`#include <gnuplot.hpp>`

Inheritance diagram for Gnuplot:



Public Member Functions

- Gnuplot (Parameters &)

  Constructor.
- void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)

  Saves one time step.
- virtual ~Gnuplot ()

  Destructor.

Public Member Functions inherited from Output

- Output (Parameters &)

  Constructor.
- virtual void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)=0

  Function to be specified in each output format.
- void check_vol (const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &) const

  Saves the infiltrated and rain volumes.
- void result (const SCALAR &, const clock_t &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const int &, const SCALAR &) const

  Saves global values.
- void initial (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the initial time.
- void final (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the final time.
- SCALAR boundaries_flux (const SCALAR &, const TAB &, const TAB &, const SCALAR &, const SCALAR &, const int &, const int &)

  Saves the cumulated fluxes on the boundaries.
- void boundaries_flux_LR (const SCALAR &, const TAB &) const

  Saves the fluxes on the left and right boundaries.
- void boundaries_flux_BT (const SCALAR &, const TAB &) const

  Saves the fluxes on the bottom and top boundaries.
- SCALAR round5 (const SCALAR &) const

  Rounded value up to 10^-5.
- virtual ~Output ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Output

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- string outputDirectory
- string namefile_check_volume
- string namefile_res
- string namefile_init
- string namefile_final
- string namefile_Bound_flux
- string namefile_Bound_flux_BT
- string namefile_Bound_flux_LR

4.31.1 Detailed Description

Gnuplot output

Class that writes the result in the output file with a structure optimized for Gnuplot.

Definition at line 73 of file gnuplot.hpp.

4.31.2 Constructor & Destructor Documentation

Gnuplot()

```
Gnuplot::Gnuplot (
            Parameters & par )
```

Constructor.

Writes the header of the file 'huz_evolution.dat'.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

Impossible to open the *** file. Verify if the directory *** exists.

Note

If huz_evolution.dat cannot be opened, the code will exit with failure termination code.

Definition at line 61 of file gnuplot.cpp.

~Gnuplot()

```
Gnuplot::~Gnuplot ( )  [virtual]
```

Destructor.

Definition at line 126 of file gnuplot.cpp.

4.31.3 Member Function Documentation

write()

```
void Gnuplot::write (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const TAB & z,
            const SCALAR & time ) [virtual]
```

Saves one time step.

Writes the values of Scheme::h, Scheme::u (=q1/h), Scheme::v (=q2/h), Scheme::h+ Scheme::z (free surface), Scheme::z, $|U| = \sqrt{u^2+v^2}$, the Froude number $\frac{|U|}{\sqrt{gh}}$, Scheme::q1, Scheme::q2, and $h|U|$ at the current time in huz_evolution.dat.
If the water height is too small, we replace it by 0, the velocity is null and the Froude number does not exist.

Parameters

in	*h*	the water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.
in	*z*	the topography.
in	*time*	the current time.

Implements Output.
Definition at line 90 of file gnuplot.cpp.
The documentation for this class was generated from the following files:

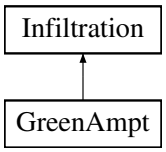
- Headers/libsave/gnuplot.hpp
- Sources/libsave/gnuplot.cpp

4.32 GreenAmpt Class Reference

Green-Ampt law.

`#include <greenampt.hpp>`

Inheritance diagram for GreenAmpt:



Public Member Functions

- GreenAmpt (Parameters &)

  Constructor.
- SCALAR capacity (const SCALAR, const SCALAR, const SCALAR, const SCALAR Kc, const SCALAR Ks, const SCALAR dtheta, const SCALAR Psi, const SCALAR zcrust)

  Calculates the infiltration capacity.
- void calcul (const TAB &, const TAB &, const SCALAR)

Calculates the infiltrated volume.

- virtual ~GreenAmpt ()

  Destructor.

Public Member Functions inherited from Infiltration

- Infiltration (Parameters &)

  Constructor.
- virtual void calcul (const TAB &, const TAB &, const SCALAR)=0

  Function to be specified in each case.
- TAB get_hmod () const

  Gives the modified valued of the water height.
- TAB get_Vin () const

  Gives the infiltrated volume.
- virtual ~Infiltration ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Infiltration

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB hmod
- TAB Vin

4.32.1 Detailed Description

Green-Ampt law.

Class that computes the infiltrated volume and modified water height with Green-Ampt 1d law.

Definition at line 72 of file greenampt.hpp.

4.32.2 Constructor & Destructor Documentation

GreenAmpt()

```
GreenAmpt::GreenAmpt (
            Parameters & par )
```

Constructor.

Initializes the values for Green-Ampt infiltration.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

***: ERROR: the value at the point ***.

Definition at line 59 of file greenampt.cpp.

~GreenAmpt()

```
GreenAmpt::~GreenAmpt ( )  [virtual]
```

Destructor.
Definition at line 306 of file greenampt.cpp.

4.32.3 Member Function Documentation

calcul()

```
void GreenAmpt::calcul (
            const TAB & h,
            const TAB & Vin_tot,
            const SCALAR dt )  [virtual]
```

Calculates the infiltrated volume.

Parameters

in	*h*	water height.
in	*Vin_tot*	total infiltrated volume.
in	*dt*	time step.

Modifies

Infiltration::hmod modified water height.
Infiltration::Vin total infiltrated volume containing the current time step.

Implements Infiltration.
Definition at line 260 of file greenampt.cpp.

capacity()

```
SCALAR GreenAmpt::capacity (
            const SCALAR h,
            const SCALAR Vin_tot,
            const SCALAR dt,
            const SCALAR Kc,
            const SCALAR Ks,
            const SCALAR dtheta,
            const SCALAR Psi,
            const SCALAR zcrust )
```

Calculates the infiltration capacity.

the infiltration capacity is given by:

$$I_C = \begin{cases} K_s(1+\frac{Psi+h}{Z_f}) & \text{if} \quad zcrust = 0 \\ K_c(1+\frac{Psi+h}{Z_f}) & \text{if} \quad Z_f \leq zcrust \\ K_e(1+\frac{Psi+h}{Z_f}) & \end{cases},$$

with the effective hydraulic conductivity

$$K_e = \frac{1}{\frac{1}{Ks} * (1 - \frac{zcrust * dtheta}{Vin\_tot}) + zcrust * \frac{dtheta}{Vin\_tot} * \frac{1}{Kc}}$$

Parameters

in	*h*	water height.
in	*Vin_tot*	total infiltrated volume.
in	*dt*	time step.
in	*Kc*	hydraulic conductivity of the (upper) crust.
in	*Ks*	hydraulic conductivity of the (lower) soil.
in	*dtheta*	initial water deficit.
in	*Psi*	load pressure.
in	*zcrust*	thickness of the (upper) crust.

Returns

Ic: infiltration capacity.

Definition at line 216 of file greenampt.cpp.
The documentation for this class was generated from the following files:

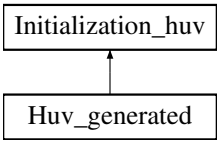
- Headers/librain_infiltration/greenampt.hpp
- Sources/librain_infiltration/greenampt.cpp

4.33 Huv_generated Class Reference

No water configuration.

`#include <huv_generated.hpp>`

Inheritance diagram for Huv_generated:



Public Member Functions

- Huv_generated (Parameters &)

  Constructor.
- void initialization (TAB &, TAB &, TAB &)

Performs the initialization.

- virtual ~Huv_generated ()

  Destructor.

Public Member Functions inherited from Initialization_huv

- Initialization_huv (Parameters &)

  Constructor.
- virtual void initialization (TAB &, TAB &, TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_huv ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_huv

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.33.1 Detailed Description

No water configuration.

Class that initializes the water height and the velocity for a dry domain.

Definition at line 73 of file huv_generated.hpp.

4.33.2 Constructor & Destructor Documentation

Huv_generated()

```
Huv_generated::Huv_generated (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 60 of file huv_generated.cpp.

~Huv_generated()

```
Huv_generated::~Huv_generated ( )  [virtual]
```

Destructor.

Definition at line 86 of file huv_generated.cpp.

4.33.3 Member Function Documentation

initialization()

```
void Huv_generated::initialization (
            TAB & h,
            TAB & u,
            TAB & v ) [virtual]
```

Performs the initialization.

Initializes the water height and the velocity at 0.

Parameters

in,out	*h*	water height.
in,out	*u*	first component of the velocity.
in,out	*v*	second component of the velocity.

Implements Initialization_huv.

Definition at line 67 of file huv_generated.cpp.

The documentation for this class was generated from the following files:

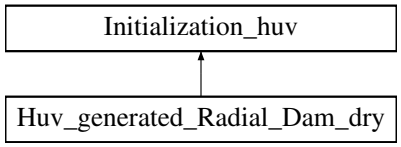
- Headers/libinitializations/huv_generated.hpp
- Sources/libinitializations/huv_generated.cpp

4.34 Huv_generated_Radial_Dam_dry Class Reference

Dry radial dam break configuration.

`#include <huv_generated_radial_dam_dry.hpp>`

Inheritance diagram for Huv_generated_Radial_Dam_dry:



Public Member Functions

- Huv_generated_Radial_Dam_dry (Parameters &)

  Constructor.
- void initialization (TAB &, TAB &, TAB &)

  Performs the initialization.
- virtual ~Huv_generated_Radial_Dam_dry ()

  Destructor.

Public Member Functions inherited from Initialization_huv

- Initialization_huv (Parameters &)

  Constructor.
- virtual void initialization (TAB &, TAB &, TAB &)=0

  Function to be specified in each initialization.

- virtual ~Initialization_huv ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_huv

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.34.1 Detailed Description

Dry radial dam break configuration.

Class that initializes the water height and the velocity for a radial dam break on a dry domain.

Definition at line 74 of file huv_generated_radial_dam_dry.hpp.

4.34.2 Constructor & Destructor Documentation

Huv_generated_Radial_Dam_dry()

```
Huv_generated_Radial_Dam_dry::Huv_generated_Radial_Dam_dry (
            Parameters & par )
```

Constructor.

Defines the position of the dam (half of the domain), the water height before the dam (5 millimeters), the water height after the dam (0 meter) and the velocity (0 m/s), see Goutal and Maurel [1997], Audusse et al. [2000].

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 61 of file huv_generated_radial_dam_dry.cpp.

~Huv_generated_Radial_Dam_dry()

```
Huv_generated_Radial_Dam_dry::~Huv_generated_Radial_Dam_dry ( )  [virtual]
```

Destructor.

Definition at line 101 of file huv_generated_radial_dam_dry.cpp.

4.34.3 Member Function Documentation

initialization()

```
void Huv_generated_Radial_Dam_dry::initialization (
            TAB & h,
            TAB & u,
            TAB & v )  [virtual]
```

Performs the initialization.

Initializes the water height and the velocity, before and after the dam.

Parameters

in,out	*h*	water height.
in,out	*u*	first component of the velocity.
in,out	*v*	second component of the velocity.

Implements Initialization_huv.
Definition at line 80 of file huv_generated_radial_dam_dry.cpp.
The documentation for this class was generated from the following files:

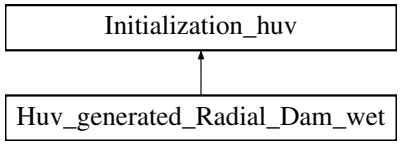
- Headers/libinitializations/huv_generated_radial_dam_dry.hpp
- Sources/libinitializations/huv_generated_radial_dam_dry.cpp

4.35 Huv_generated_Radial_Dam_wet Class Reference

Wet radial dam break configuration.

`#include <huv_generated_radial_dam_wet.hpp>`

Inheritance diagram for Huv_generated_Radial_Dam_wet:



Public Member Functions

- Huv_generated_Radial_Dam_wet (Parameters &)

  Constructor.
- void initialization (TAB &, TAB &, TAB &)

  Performs the initialization.
- virtual ~Huv_generated_Radial_Dam_wet ()

  Destructor.

Public Member Functions inherited from Initialization_huv

- Initialization_huv (Parameters &)

  Constructor.
- virtual void initialization (TAB &, TAB &, TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_huv ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_huv

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.35.1 Detailed Description

Wet radial dam break configuration.

Class for the initialization of the water height and velocity for a radial dam break on a wet domain.

Definition at line 75 of file huv_generated_radial_dam_wet.hpp.

4.35.2 Constructor & Destructor Documentation

Huv_generated_Radial_Dam_wet()

```
Huv_generated_Radial_Dam_wet::Huv_generated_Radial_Dam_wet (
            Parameters & par )
```

Constructor.

Defines the position of the dam (half of the domain), the water height before the dam (5 millimeters), the water height after the dam (4 millimeter) and the velocity (0 m/s), see Goutal and Maurel [1997], Audusse et al. [2000].

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 61 of file huv_generated_radial_dam_wet.cpp.

~Huv_generated_Radial_Dam_wet()

```
Huv_generated_Radial_Dam_wet::~Huv_generated_Radial_Dam_wet ( )  [virtual]
```

Destructor.

Definition at line 102 of file huv_generated_radial_dam_wet.cpp.

4.35.3 Member Function Documentation

initialization()

```
void Huv_generated_Radial_Dam_wet::initialization (
            TAB & h,
            TAB & u,
            TAB & v )  [virtual]
```

Performs the initialization.

Initializes the water height and the velocity, before and after the dam.

Parameters

in,out	*h*	water height.
in,out	*u*	first component of the velocity.
in,out	*v*	second component of the velocity.

Implements Initialization_huv.

Definition at line 81 of file huv_generated_radial_dam_wet.cpp.

The documentation for this class was generated from the following files:

- Headers/libinitializations/huv_generated_radial_dam_wet.hpp

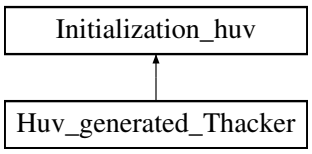
- Sources/libinitializations/huv_generated_radial_dam_wet.cpp

4.36 Huv_generated_Thacker Class Reference

Thacker configuration.

`#include <huv_generated_thacker.hpp>`

Inheritance diagram for Huv_generated_Thacker:



Public Member Functions

- Huv_generated_Thacker (Parameters &)

  Constructor.
- void initialization (TAB &, TAB &, TAB &)

  Performs the initialization.
- virtual ~Huv_generated_Thacker ()

  Destructor.

Public Member Functions inherited from Initialization_huv

- Initialization_huv (Parameters &)

  Constructor.
- virtual void initialization (TAB &, TAB &, TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_huv ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_huv

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.36.1 Detailed Description

Thacker configuration.

Class that initializes the water height and the velocity for Thacker's benchmark.

Definition at line 74 of file huv_generated_thacker.hpp.

4.36.2 Constructor & Destructor Documentation

Huv_generated_Thacker()

```
Huv_generated_Thacker::Huv_generated_Thacker (
            Parameters & par )
```

Constructor.

Defines the characteristics of the paraboloid.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 60 of file huv_generated_thacker.cpp.

~Huv_generated_Thacker()

```
Huv_generated_Thacker::~Huv_generated_Thacker ( )  [virtual]
```

Destructor.

Definition at line 106 of file huv_generated_thacker.cpp.

4.36.3 Member Function Documentation

initialization()

```
void Huv_generated_Thacker::initialization (
            TAB & h,
            TAB & u,
            TAB & v )  [virtual]
```

Performs the initialization.

Initializes the water height to a plane surface and the velocity to zero, see Thacker [1981].

Parameters

in,out	*h*	water height.
in,out	*u*	first component of the velocity.
in,out	*v*	second component of the velocity.

Implements Initialization_huv.

Definition at line 80 of file huv_generated_thacker.cpp.

The documentation for this class was generated from the following files:

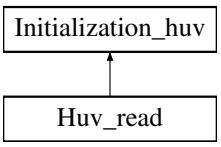
- Headers/libinitializations/huv_generated_thacker.hpp
- Sources/libinitializations/huv_generated_thacker.cpp

4.37 Huv_read Class Reference

File configuration.

```
#include <huv_read.hpp>
```

Inheritance diagram for Huv_read:



Public Member Functions

- Huv_read (Parameters &)

  Constructor.
- void initialization (TAB &, TAB &, TAB &)

  Performs the initialization.
- virtual ~Huv_read ()

  Destructor.

Public Member Functions inherited from Initialization_huv

- Initialization_huv (Parameters &)

  Constructor.
- virtual void initialization (TAB &, TAB &, TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_huv ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_huv

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.37.1 Detailed Description

File configuration.

Class that initializes the water height and of the velocity to the values read in a file.

Definition at line 72 of file huv_read.hpp.

4.37.2 Constructor & Destructor Documentation

Huv_read()

```
Huv_read::Huv_read (
            Parameters & par )
```

Constructor.

Defines the name of the file for the initialization.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file huv_read.cpp.

~Huv_read()

```
Huv_read::~Huv_read ( )  [virtual]
```

Destructor.
Definition at line 200 of file huv_read.cpp.

4.37.3 Member Function Documentation

initialization()

```
void Huv_read::initialization (
            TAB & h,
            TAB & u,
            TAB & v )  [virtual]
```

Performs the initialization.
Initializes the water height and the velocity to the values read in the corresponding file.

Parameters

in,out	*h*	water height.
in,out	*u*	first component of the velocity.
in,out	*v*	second component of the velocity.

Warning

(huv_namefile): ERROR: cannot open the huv file.

(huv_namefile): ERROR: the number of data in this file is too big

(huv_namefile): ERROR: line ***.

(huv_namefile): WARNING: line ***.

(huv_namefile): ERROR: the number of data in this file is too small

(huv_namefile): ERROR: the value for the point x *** y *** is missing

Note

If the file cannot be opened or if the data are not correct, the code will exit with failure termination code.

Implements Initialization_huv.
Definition at line 72 of file huv_read.cpp.
The documentation for this class was generated from the following files:

- Headers/libinitializations/huv_read.hpp
- Sources/libinitializations/huv_read.cpp

4.38 Hydrostatic Class Reference

Hydrostatic reconstruction

```
#include <hydrostatic.hpp>
```

Public Member Functions

- Hydrostatic ()

  Constructor.
- void calcul (SCALAR, SCALAR, SCALAR)

  Calculates the hydrostatic reconstruction.
- SCALAR get_hhydro_l ()

  Gives the reconstructed water height on the left.
- SCALAR get_hhydro_r ()

  Gives the reconstructed water height on the right.
- virtual ∼Hydrostatic ()

  Destructor.

Protected Attributes

- SCALAR hl_rec
- SCALAR hr_rec

4.38.1 Detailed Description

Hydrostatic reconstruction

Class that computes the hydrostatic reconstruction.

Definition at line 67 of file hydrostatic.hpp.

4.38.2 Constructor & Destructor Documentation

Hydrostatic()

```
Hydrostatic::Hydrostatic ( )
```

Constructor.

Definition at line 60 of file hydrostatic.cpp.

∼Hydrostatic()

```
Hydrostatic::∼Hydrostatic ( )  [virtual]
```

Destructor.

Definition at line 102 of file hydrostatic.cpp.

4.38.3 Member Function Documentation

calcul()

```
void Hydrostatic::calcul (
            SCALAR hl,
            SCALAR hr,
            SCALAR dz )
```

Calculates the hydrostatic reconstruction.

See Audusse et al. [2004] for more details.

Parameters

in	*hl*	water height on the cell located at the left of the boundary.
in	*hr*	water height on the cell located at the right of the boundary.
in	*dz*	Difference between the values of the topography of the two adjacent cells.

Modifies

Hydrostatic::hl_rec, set to $(hl - \max(0, dz))_+$.
Hydrostatic::hr_rec, set to $(hr - \max(0, -dz))_+$.

Definition at line 63 of file hydrostatic.cpp.

get_hhydro_l()

`SCALAR Hydrostatic::get_hhydro_l ( )`

Gives the reconstructed water height on the left.

Returns

Hydrostatic::hl_rec Hydrostatic reconstruction on the left.

Definition at line 81 of file hydrostatic.cpp.

get_hhydro_r()

`SCALAR Hydrostatic::get_hhydro_r ( )`

Gives the reconstructed water height on the right.

Returns

Hydrostatic::hr_rec Hydrostatic reconstruction on the right.

Definition at line 92 of file hydrostatic.cpp.

4.38.4 Member Data Documentation

hl_rec

`SCALAR Hydrostatic::hl_rec [protected]`

Hydrostatic reconstruction on the left
Definition at line 87 of file hydrostatic.hpp.

hr_rec

`SCALAR Hydrostatic::hr_rec [protected]`

Hydrostatic reconstruction on the right
Definition at line 89 of file hydrostatic.hpp.
The documentation for this class was generated from the following files:

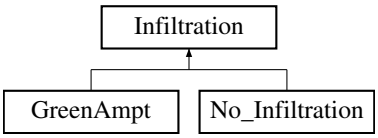
- Headers/libreconstructions/hydrostatic.hpp
- Sources/libreconstructions/hydrostatic.cpp

4.39 Infiltration Class Reference

Definition of infiltration law.

`#include <infiltration.hpp>`

Inheritance diagram for Infiltration:



Public Member Functions

- Infiltration (Parameters &)

  Constructor.
- virtual void calcul (const TAB &, const TAB &, const SCALAR)=0

  Function to be specified in each case.
- TAB get_hmod () const

  Gives the modified valued of the water height.
- TAB get_Vin () const

  Gives the infiltrated volume.
- virtual ~Infiltration ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB hmod
- TAB Vin

4.39.1 Detailed Description

Definition of infiltration law.

Class that contains all the common declarations for the infiltration law.

Definition at line 71 of file infiltration.hpp.

4.39.2 Constructor & Destructor Documentation

Infiltration()

```
Infiltration::Infiltration (
            Parameters & par )
```

Constructor.

Defines the number of cells, the space steps and initializes Infiltration::hmod and Infiltration::Vin.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file infiltration.cpp.

~Infiltration()

```
Infiltration::~Infiltration ( )  [virtual]
```

Destructor.

Definition at line 103 of file infiltration.cpp.

4.39.3 Member Function Documentation

calcul()

```
virtual void Infiltration::calcul (
            const TAB & ,
            const TAB & ,
            const SCALAR )  [pure virtual]
```

Function to be specified in each case.

Implemented in GreenAmpt, and No_Infiltration.

get_hmod()

```
TAB Infiltration::get_hmod ( ) const
```

Gives the modified valued of the water height.

Returns

The value of Infiltration::hmod.

Definition at line 83 of file infiltration.cpp.

get_Vin()

```
TAB Infiltration::get_Vin ( ) const
```

Gives the infiltrated volume.

Returns

The value of Infiltration::Vin.

Definition at line 93 of file infiltration.cpp.

4.39.4 Member Data Documentation

DX

`const SCALAR Infiltration::DX [protected]`

Space step in the first (x) direction.

Definition at line 96 of file infiltration.hpp.

DY

`const SCALAR Infiltration::DY [protected]`

Space step in the second (y) direction.

Definition at line 98 of file infiltration.hpp.

hmod

`TAB Infiltration::hmod [protected]`

Modified valued of the water height

Definition at line 100 of file infiltration.hpp.

NXCELL

`const int Infiltration::NXCELL [protected]`

Number of cells in space in the first (x) direction.

Definition at line 92 of file infiltration.hpp.

NYCELL

`const int Infiltration::NYCELL [protected]`

Number of cells in space in the second (y) direction.

Definition at line 94 of file infiltration.hpp.

Vin

`TAB Infiltration::Vin [protected]`

Infiltrated volume

Definition at line 102 of file infiltration.hpp.

The documentation for this class was generated from the following files:

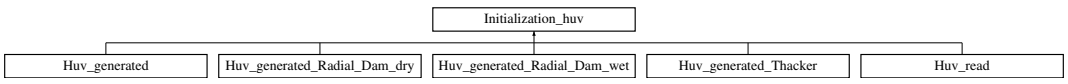
- Headers/librain_infiltration/infiltration.hpp
- Sources/librain_infiltration/infiltration.cpp

4.40 Initialization_huv Class Reference

Initialization of h, u and v.

`#include <initialization_huv.hpp>`

Inheritance diagram for Initialization_huv:



Public Member Functions

- Initialization_huv (Parameters &)

  Constructor.
- virtual void initialization (TAB &, TAB &, TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_huv ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.40.1 Detailed Description

Initialization of h, u and v.

Class that contains all the common declarations for the initialization of the water height and of the velocity.

Definition at line 71 of file initialization_huv.hpp.

4.40.2 Constructor & Destructor Documentation

Initialization_huv()

```
Initialization_huv::Initialization_huv (
            Parameters & par )
```

Constructor.

Defines the numbers of cells and the space steps.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file initialization_huv.cpp.

~Initialization_huv()

```
Initialization_huv::~Initialization_huv ( )  [virtual]
```

Destructor.

Definition at line 70 of file initialization_huv.cpp.

4.40.3 Member Function Documentation

initialization()

```
virtual void Initialization_huv::initialization (
            TAB & ,
```

```
TAB & ,
TAB &  ) [pure virtual]
```

Function to be specified in each initialization.

Implemented in Huv_generated, Huv_generated_Radial_Dam_dry, Huv_generated_Radial_Dam_wet, Huv_generated_Thacker, and Huv_read.

4.40.4 Member Data Documentation

DX

```
const SCALAR Initialization_huv::DX [protected]
```

Space step in the x direction.

Definition at line 90 of file initialization_huv.hpp.

DY

```
const SCALAR Initialization_huv::DY [protected]
```

Space step in the y direction.

Definition at line 92 of file initialization_huv.hpp.

NXCELL

```
const int Initialization_huv::NXCELL [protected]
```

Number of cells in space in the x direction.

Definition at line 86 of file initialization_huv.hpp.

NYCELL

```
const int Initialization_huv::NYCELL [protected]
```

Number of cells in space in the y direction.

Definition at line 88 of file initialization_huv.hpp.

The documentation for this class was generated from the following files:

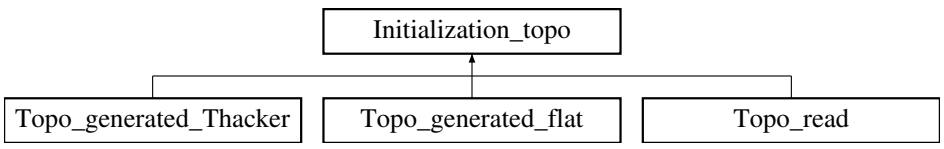
- Headers/libinitializations/initialization_huv.hpp
- Sources/libinitializations/initialization_huv.cpp

4.41 Initialization_topo Class Reference

Initialization of z.

```
#include <initialization_topo.hpp>
```

Inheritance diagram for Initialization_topo:



Public Member Functions

- Initialization_topo (Parameters &)

  Constructor.
- virtual void initialization (TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_topo ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.41.1 Detailed Description

Initialization of z.

Class that contains all the common declarations for the initialization of the topography.

Definition at line 71 of file initialization_topo.hpp.

4.41.2 Constructor & Destructor Documentation

Initialization_topo()

```
Initialization_topo::Initialization_topo (
            Parameters & par )
```

Constructor.

Defines the numbers of cells and the space steps.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file initialization_topo.cpp.

~Initialization_topo()

```
Initialization_topo::~Initialization_topo ( )  [virtual]
```

Destructor.

Definition at line 70 of file initialization_topo.cpp.

4.41.3 Member Function Documentation

initialization()

```
virtual void Initialization_topo::initialization (
            TAB &  )  [pure virtual]
```

Function to be specified in each initialization.
Implemented in Topo_generated_flat, Topo_generated_Thacker, and Topo_read.

4.41.4 Member Data Documentation

DX

```
const SCALAR Initialization_topo::DX [protected]
```

Space step in the x direction.
Definition at line 91 of file initialization_topo.hpp.

DY

```
const SCALAR Initialization_topo::DY [protected]
```

Space step in the y direction.
Definition at line 93 of file initialization_topo.hpp.

NXCELL

```
const int Initialization_topo::NXCELL [protected]
```

Number of cells in space in the x direction.
Definition at line 87 of file initialization_topo.hpp.

NYCELL

```
const int Initialization_topo::NYCELL [protected]
```

Number of cells in space in the y direction.
Definition at line 89 of file initialization_topo.hpp.
The documentation for this class was generated from the following files:

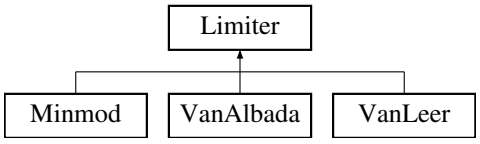
- Headers/libinitializations/initialization_topo.hpp
- Sources/libinitializations/initialization_topo.cpp

4.42 Limiter Class Reference

Slope limiter.

```
#include <limiter.hpp>
```

Inheritance diagram for Limiter:



Public Member Functions

- Limiter ()

  Constructor.
- virtual void calcul (SCALAR, SCALAR)=0

  Function to be specified in each slope limiter.
- SCALAR get_rec () const

  Gives the reconstructed value.
- virtual ∼Limiter ()

  Destructor.

Protected Attributes

- SCALAR rec

4.42.1 Detailed Description

Slope limiter.

Class that contains all the common declarations for the slope limters.

Definition at line 71 of file limiter.hpp.

4.42.2 Constructor & Destructor Documentation

Limiter()

```
Limiter::Limiter ( )
```

Constructor.

Definition at line 59 of file limiter.cpp.

∼Limiter()

```
Limiter::~Limiter ( )  [virtual]
```

Destructor.

Definition at line 73 of file limiter.cpp.

4.42.3 Member Function Documentation

calcul()

```
virtual void Limiter::calcul (
            SCALAR ,
            SCALAR  )  [pure virtual]
```

Function to be specified in each slope limiter.

Implemented in Minmod, VanAlbada, and VanLeer.

get_rec()

`SCALAR Limiter::get_rec ( ) const`

Gives the reconstructed value.

Returns

Limiter::rec reconstructed value.

Definition at line 63 of file limiter.cpp.

4.42.4 Member Data Documentation

rec

`SCALAR Limiter::rec [protected]`

Reconstructed value

Definition at line 90 of file limiter.hpp.

The documentation for this class was generated from the following files:

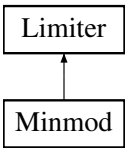
- Headers/liblimitations/limiter.hpp
- Sources/liblimitations/limiter.cpp

4.43 Minmod Class Reference

Minmod slope limiter

`#include <minmod.hpp>`

Inheritance diagram for Minmod:



Public Member Functions

- Minmod ()
  Constructor.
- void calcul (SCALAR, SCALAR)
  Calculates the value of the slope limiter.
- virtual ~Minmod ()
  Destructor.

Public Member Functions inherited from Limiter

- Limiter ()
  Constructor.
- virtual void calcul (SCALAR, SCALAR)=0
  Function to be specified in each slope limiter.
- SCALAR get_rec () const
  Gives the reconstructed value.
- virtual ~Limiter ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Limiter

- SCALAR rec

4.43.1 Detailed Description

Minmod slope limiter

Class that calculates the minmod slope limiter.

Definition at line 71 of file minmod.hpp.

4.43.2 Constructor & Destructor Documentation

Minmod()

```
Minmod::Minmod ( )
```

Constructor.

Definition at line 59 of file minmod.cpp.

~Minmod()

```
Minmod::~Minmod ( )  [virtual]
```

Destructor.

Definition at line 88 of file minmod.cpp.

4.43.3 Member Function Documentation

calcul()

```
void Minmod::calcul (
            SCALAR a,
            SCALAR b )  [virtual]
```

Calculates the value of the slope limiter.

Minmod function:

$$\text{minmod}(x,y) = \begin{cases} \min(x,y) & \text{if } x,y \geq 0, \\ \max(x,y) & \text{if } x,y \leq 0, \\ 0 & \text{else.} \end{cases}$$

Parameters

in	*a*	slope on the left of the cell.
in	*b*	slope on the right of the cell.

Modifies

Limiter::rec recontructed value.

Implements Limiter.

Definition at line 62 of file minmod.cpp.

The documentation for this class was generated from the following files:

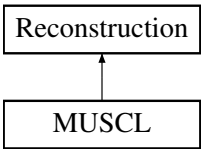
- Headers/liblimitations/minmod.hpp
- Sources/liblimitations/minmod.cpp

4.44 MUSCL Class Reference

MUSCL recontruction

`#include <muscl.hpp>`

Inheritance diagram for MUSCL:



Public Member Functions

- MUSCL (Parameters &, TAB &)

  Constructor.
- void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)

  Calculates the reconstruction in space.
- ∼MUSCL ()

  Destructor.

Public Member Functions inherited from Reconstruction

- Reconstruction (Parameters &, TAB &)

  Constructor.
- virtual void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)=0

  Function to be specified in each reconstruction.
- virtual ∼Reconstruction ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Reconstruction

- const int NXCELL
- const int NYCELL
- TAB z1r
- TAB z1l
- TAB z2r
- TAB z2l
- TAB delta_z1
- TAB delta_z2
- Choice_limiter ∗ limiter

4.44.1 Detailed Description

MUSCL recontruction

Class that computes MUSCL reconstruction in space.

Definition at line 73 of file muscl.hpp.

4.44.2 Constructor & Destructor Documentation

MUSCL()

```
MUSCL::MUSCL (
            Parameters & par,
            TAB & z )
```

Constructor.

Initializations.

Parameters

in	*par*	parameter, contains all the values from the parameters file.
in	*z*	topography.

Definition at line 61 of file muscl.cpp.

~MUSCL()

```
MUSCL::~MUSCL ( )
```

Destructor.

Definition at line 224 of file muscl.cpp.

4.44.3 Member Function Documentation

calcul()

```
void MUSCL::calcul (
            TAB & h,
            TAB & u,
            TAB & v,
            TAB & z,
            TAB & delzc1,
            TAB & delzc2,
            TAB & delz1,
            TAB & delz2,
            TAB & h1r,
            TAB & u1r,
            TAB & v1r,
            TAB & h1l,
            TAB & u1l,
            TAB & v1l,
            TAB & h2r,
            TAB & u2r,
```

```
TAB & v2r,
TAB & h2l,
TAB & u2l,
TAB & v2l ) [virtual]
```

Calculates the reconstruction in space.

Calls the calculation of the second order reconstruction in space with MUSCL formulation, see van Leer [1979] Bouchut [2007].

Parameters

in	*h*	water height.
in	*u*	velocity of the flow in the first direction.
in	*v*	velocity of the flow in the second direction.
in	*z*	topography.
out	*delzc1*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the first direction.
out	*delzc2*	difference between the reconstructed topographies on the left and on the right boundary of a cell in the second direction.
out	*delz1*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the first direction.
out	*delz2*	difference between two reconstructed topographies on the same boundary (from two adjacent cells) in the seond direction.
out	*h1r*	reconstructed water height on the right of the cell in the first direction.
out	*u1r*	first component of the reconstructed velocity on the right of the cell in the first direction.
out	*v1r*	second component of the reconstructed velocity on the right of the cell in the first direction.
out	*h1l*	reconstructed water height on the left of the cell in the first direction.
out	*u1l*	first component of the reconstructed velocity on the left of the cell in the first direction.
out	*v1l*	second component of the reconstructed velocity on the left of the cell in the first direction.
out	*h2r*	reconstructed water height on the right of the cell in the second direction.
out	*u2r*	first component of the reconstructed velocity on the right of the cell in the second direction.
out	*v2r*	second component of the reconstructed velocity on the right of the cell in the second direction.
out	*h2l*	reconstructed water height on the left of the cell in the second direction.
out	*u2l*	first component of the reconstructed velocity on the left of the cell in the second direction.
out	*v2l*	second component of the reconstructed velocity on the left of the cell in the second direction.

Implements Reconstruction.

Definition at line 73 of file muscl.cpp.

The documentation for this class was generated from the following files:

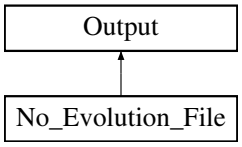
- Headers/libreconstructions/muscl.hpp
- Sources/libreconstructions/muscl.cpp

4.45 No_Evolution_File Class Reference

No output.

```
#include <no_evolution_file.hpp>
```

Inheritance diagram for No_Evolution_File:



Public Member Functions

- No_Evolution_File (Parameters &)

  Constructor.
- void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)

  Saves one time step: nothing to do.
- virtual ~No_Evolution_File ()

  Destructor.

Public Member Functions inherited from Output

- Output (Parameters &)

  Constructor.
- virtual void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)=0

  Function to be specified in each output format.
- void check_vol (const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &) const

  Saves the infiltrated and rain volumes.
- void result (const SCALAR &, const clock_t &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const int &, const SCALAR &) const

  Saves global values.
- void initial (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the initial time.
- void final (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the final time.
- SCALAR boundaries_flux (const SCALAR &, const TAB &, const TAB &, const SCALAR &, const SCALAR &, const int &, const int &)

  Saves the cumulated fluxes on the boundaries.
- void boundaries_flux_LR (const SCALAR &, const TAB &) const

  Saves the fluxes on the left and right boundaries.
- void boundaries_flux_BT (const SCALAR &, const TAB &) const

  Saves the fluxes on the bottom and top boundaries.
- SCALAR round5 (const SCALAR &) const

  Rounded value up to 10^-5.
- virtual ~Output ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Output

- const int NXCELL
- const int NYCELL
- const SCALAR DX

- const SCALAR DY
- string outputDirectory
- string namefile_check_volume
- string namefile_res
- string namefile_init
- string namefile_final
- string namefile_Bound_flux
- string namefile_Bound_flux_BT
- string namefile_Bound_flux_LR

4.45.1 Detailed Description

No output.

No output files with time evolution are created.

Definition at line 69 of file no_evolution_file.hpp.

4.45.2 Constructor & Destructor Documentation

No_Evolution_File()

```
No_Evolution_File::No_Evolution_File (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 58 of file no_evolution_file.cpp.

~No_Evolution_File()

```
No_Evolution_File::~No_Evolution_File ( )  [virtual]
```

Destructor.

Definition at line 88 of file no_evolution_file.cpp.

4.45.3 Member Function Documentation

write()

```
void No_Evolution_File::write (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const TAB & z,
            const SCALAR & time )  [virtual]
```

Saves one time step: nothing to do.

Does nothing.

Parameters

in	*h*	the water height (unused).
in	*u*	first component of the velocity (unused).
in	*v*	second component of the velocity (unused).
in	*z*	the topography (unused).
in	*time*	the current time (unused).

Implements Output.
Definition at line 67 of file no_evolution_file.cpp.
The documentation for this class was generated from the following files:

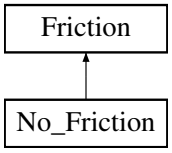
- Headers/libsave/no_evolution_file.hpp
- Sources/libsave/no_evolution_file.cpp

4.46 No_Friction Class Reference

No friction.

`#include <no_friction.hpp>`

Inheritance diagram for No_Friction:



Public Member Functions

- No_Friction (Parameters &)

  Constructor.
- void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)

  Does no calculation.
- void calculSf (const TAB &, const TAB &, const TAB &)

  Return the friction term equal to zero.
- virtual ~No_Friction ()

  Destructor.

Public Member Functions inherited from Friction

- Friction (Parameters &)

  Constructor.
- virtual void calcul (const TAB &, const TAB &, const TAB &, const TAB &, const TAB &, SCALAR)=0

  Function to be specified in each friction law.
- virtual TAB get_q1mod () const

  Gives the discharge in the first direction modified by the friction term.
- virtual TAB get_q2mod () const

  Gives the discharge in the second direction modified by the friction term.
- virtual void calculSf (const TAB &, const TAB &, const TAB &)=0

  Calculates the explicit friction term. It will be used for computations with erosion.

- virtual TAB get_Sf1 () const

  Gives the explicit friction term in the first direction.
- virtual TAB get_Sf2 () const

  Gives the explicit friction term in the second direction.
- virtual ~Friction ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Friction

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB q1mod
- TAB q2mod
- TAB Sf1
- TAB Sf2
- TAB Fric_tab

4.46.1 Detailed Description

No friction.

Does no computation.

Definition at line 71 of file no_friction.hpp.

4.46.2 Constructor & Destructor Documentation

No_Friction()

```
No_Friction::No_Friction (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file no_friction.cpp.

~No_Friction()

```
No_Friction::~No_Friction ( )  [virtual]
```

Destructor.

Definition at line 123 of file no_friction.cpp.

4.46.3 Member Function Documentation

calcul()

```
void No_Friction::calcul (
            const TAB & uold,
            const TAB & vold,
            const TAB & hnew,
            const TAB & q1new,
            const TAB & q2new,
            SCALAR dt )  [virtual]
```

Does no calculation.
No computation (no friction).

Parameters

in	*uold*	velocity in the first direction at the previous time (n if you are calculating the $n+1$th time step) (unused).
in	*vold*	velocity in the second direction at the previous time (n if you are calculating the $n+1$th time step) (unused).
in	*hnew*	water height after the Shallow-Water computation (without friction) (unused).
in	*q1new*	discharge in the first direction after the Shallow-Water computation (without friction) (unused).
in	*q2new*	discharge in the second direction after the Shallow-Water computation (without friction) (unused).
in	*dt*	time step (unused).

Modifies

Friction::q1mod discharge in the first direction modified by the friction term,
Friction::q2mod discharge in the second direction modified by the friction term.

Implements Friction.
Definition at line 68 of file no_friction.cpp.

calculSf()

```
void No_Friction::calculSf (
            const TAB & h,
            const TAB & u,
            const TAB & v )  [virtual]
```

Return the friction term equal to zero.
Explicit friction term: $S_f = 0$.

Parameters

in	*h*	water height (unused).
in	*u*	velocity in the first direction (unused).
in	*v*	velocity in the second direction (unused).

Modifies

Friction::Sf1 explicit friction term in the first direction,
Friction::Sf2 explicit friction term in the second direction.

Note

This explicit friction term will be used for erosion.

Implements Friction.
Definition at line 97 of file no_friction.cpp.
The documentation for this class was generated from the following files:

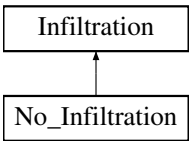
- Headers/libfrictions/no_friction.hpp
- Sources/libfrictions/no_friction.cpp

4.47 No_Infiltration Class Reference

No infiltration.

`#include <no_infiltration.hpp>`

Inheritance diagram for No_Infiltration:



Public Member Functions

- No_Infiltration (Parameters &)
  Constructor.
- void calcul (const TAB &, const TAB &, const SCALAR)
  Does no infiltration.
- virtual ~No_Infiltration ()
  Destructor.

Public Member Functions inherited from Infiltration

- Infiltration (Parameters &)
  Constructor.
- virtual void calcul (const TAB &, const TAB &, const SCALAR)=0
  Function to be specified in each case.
- TAB get_hmod () const
  Gives the modified valued of the water height.
- TAB get_Vin () const
  Gives the infiltrated volume.
- virtual ~Infiltration ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Infiltration

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- TAB hmod
- TAB Vin

4.47.1 Detailed Description

No infiltration.

The water height and infiltrated volume remain unchanged.

Definition at line 70 of file no_infiltration.hpp.

4.47.2 Constructor & Destructor Documentation

No_Infiltration()

```
No_Infiltration::No_Infiltration (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 59 of file no_infiltration.cpp.

~No_Infiltration()

```
No_Infiltration::~No_Infiltration ( )  [virtual]
```

Destructor.

Definition at line 91 of file no_infiltration.cpp.

4.47.3 Member Function Documentation

calcul()

```
void No_Infiltration::calcul (
            const TAB & h,
            const TAB & Vin_tot,
            const SCALAR dt )  [virtual]
```

Does no infiltration.

No computation (water height and infiltrated volume remain unchanged).

Parameters

in	*h*	water height.
in	*Vin_tot*	total infiltrated volume.
in	*dt*	time step (unused).

Modifies

Infiltration::hmod modified water height.
Infiltration::Vin total infiltrated volume containing the current time step.

Implements Infiltration.

Definition at line 68 of file no_infiltration.cpp.

The documentation for this class was generated from the following files:

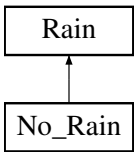
- Headers/librain_infiltration/no_infiltration.hpp
- Sources/librain_infiltration/no_infiltration.cpp

4.48 No_Rain Class Reference

No rain.

`#include <no_rain.hpp>`

Inheritance diagram for No_Rain:



Public Member Functions

- No_Rain (Parameters &)

  Constructor.
- void rain_func (SCALAR, TAB &)

  Sets the rain intensity to zero.
- virtual ∼No_Rain ()

  Destructor.

Public Member Functions inherited from Rain

- Rain (Parameters &)

  Constructor.
- virtual void rain_func (SCALAR, TAB &)=0

  Function to be specified in each case.
- virtual ∼Rain ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Rain

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.48.1 Detailed Description

No rain.

Sets the rain intensity to zero.

Definition at line 70 of file no_rain.hpp.

4.48.2 Constructor & Destructor Documentation

No_Rain()

```
No_Rain::No_Rain (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 59 of file no_rain.cpp.

~No_Rain()

```
No_Rain::~No_Rain ( )  [virtual]
```

Destructor.
Definition at line 85 of file no_rain.cpp.

4.48.3 Member Function Documentation

rain_func()

```
void No_Rain::rain_func (
            SCALAR time,
            TAB & Tab_rain )  [virtual]
```

Sets the rain intensity to zero.
No computation (rain intensity set to zero).

Parameters

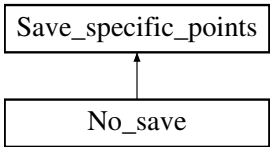
in	*time*	current time (unused).
in,out	*Tab_rain*	rain intensity at the current time on each cell.

Implements Rain.
Definition at line 68 of file no_rain.cpp.
The documentation for this class was generated from the following files:

- Headers/librain_infiltration/no_rain.hpp
- Sources/librain_infiltration/no_rain.cpp

4.49 No_save Class Reference

No output.
`#include <no_save.hpp>`
Inheritance diagram for No_save:



Public Member Functions

- No_save (Parameters &)

  Constructor.
- void save (const TAB &, const TAB &, const TAB &, const SCALAR)

  Nothing to do.
- virtual ~No_save ()

  Destructor.

Public Member Functions inherited from Save_specific_points

- Save_specific_points (Parameters &)

  Constructor.
- virtual void save (const TAB &, const TAB &, const TAB &, const SCALAR)=0

  Function which writes in a file the values at the specific points.
- virtual ~Save_specific_points ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Save_specific_points

- const SCALAR DX
- const SCALAR DY
- int row
- int column
- SCALAR T_output
- const SCALAR DT_SPECIFIC_POINTS

4.49.1 Detailed Description

No output.

No output files for specific points are created.

Definition at line 69 of file no_save.hpp.

4.49.2 Constructor & Destructor Documentation

No_save()

```
No_save::No_save (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 58 of file no_save.cpp.

~No_save()

```
No_save::~No_save ( )  [virtual]
```

Destructor.

Definition at line 85 of file no_save.cpp.

4.49.3 Member Function Documentation

save()

```
void No_save::save (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const SCALAR time )  [virtual]
```

Nothing to do.

Does nothing.

Parameters

in	*h*	the water height (unused).
in	*u*	first component of the velocity (unused).
in	*v*	second component of the velocity (unused).
in	*time*	the current time (unused).

Implements Save_specific_points.

Definition at line 67 of file no_save.cpp.

The documentation for this class was generated from the following files:

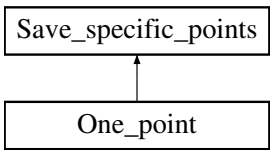
- Headers/libsave/no_save.hpp
- Sources/libsave/no_save.cpp

4.50 One_point Class Reference

One point output.

`#include <one_point.hpp>`

Inheritance diagram for One_point:



Public Member Functions

- One_point (Parameters &)

  Constructor.
- void save (const TAB &, const TAB &, const TAB &, const SCALAR)

  Saves the values at one point.
- virtual ~One_point ()

  Destructor.

Public Member Functions inherited from Save_specific_points

- Save_specific_points (Parameters &)

  Constructor.
- virtual void save (const TAB &, const TAB &, const TAB &, const SCALAR)=0

  Function which writes in a file the values at the specific points.
- virtual ~Save_specific_points ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Save_specific_points

- const SCALAR DX
- const SCALAR DY
- int row
- int column
- SCALAR T_output
- const SCALAR DT_SPECIFIC_POINTS

4.50.1 Detailed Description

One point output.

Class that writes the result in the output file with a structure optimized for Gnuplot.

Definition at line 73 of file one_point.hpp.

4.50.2 Constructor & Destructor Documentation

One_point()

```
One_point::One_point (
            Parameters & par )
```

Constructor.

Writes the header of the file 'hu_specific_points.dat'.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

***: ERROR: Impossible to open the *** file. Verify if the directory *** exists.

Note

If hu_specific_points.dat cannot be opened, the code will exit with failure termination code.

Definition at line 60 of file one_point.cpp.

~One_point()

```
One_point::~One_point ( )  [virtual]
```

Destructor.

Definition at line 132 of file one_point.cpp.

4.50.3 Member Function Documentation

save()

```
void One_point::save (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const SCALAR time )  [virtual]
```

Saves the values at one point.

Writes the values of Scheme::h, Scheme::u (=q1/h), Scheme::v (=q2/h), Scheme::h+ Scheme::z (free surface), Scheme::z, $|U| = \sqrt{u^2+v^2}$, the Froude number $\frac{|U|}{\sqrt{gh}}$, Scheme::q1, Scheme::q2, and $h|U|$ at the current time in "hu_specific_points.dat".

If the water height is too small, we replace it by 0, the velocity is null and the Froude number does not exist.

Parameters

in	*h*	the water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.
in	*time*	the current time.

Implements Save_specific_points.

Definition at line 91 of file one_point.cpp.

The documentation for this class was generated from the following files:

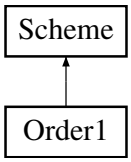
- Headers/libsave/one_point.hpp
- Sources/libsave/one_point.cpp

4.51 Order1 Class Reference

Order 1 scheme.

```
#include <order1.hpp>
```

Inheritance diagram for Order1:



Public Member Functions

- Order1 (Parameters &)

  Constructor.
- void calcul ()

Performs the numerical scheme.

- virtual ~Order1 ()

  Destructor.

Public Member Functions inherited from Scheme

- Scheme (Parameters &)

  Constructor.
- virtual void calcul ()=0

  Function to be specified in each numerical scheme.
- void allocation ()

  Allocation of spatialized variables.
- void deallocation ()

  Deallocation of variables.
- void maincalcflux (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR &)

  Main calculation of the flux.
- void maincalcscheme (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, SCALAR, SCALAR, int)

  Main calculation of the scheme.
- void boundary (TAB &, TAB &, TAB &, SCALAR, const int, const int)

  Calls the boundary conditions and affects the boundary values.
- SCALAR froude_number (TAB, TAB, TAB)

  Returns the Froude number.
- virtual ~Scheme ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Scheme

- const int NXCELL
- const int NYCELL
- const int ORDER
- const SCALAR T
- const int NBTIMES
- const int SCHEME_TYPE
- const SCALAR DX
- const SCALAR DY
- const SCALAR CFL_FIX
- SCALAR DT_FIX
- SCALAR tx
- SCALAR ty
- SCALAR T_output
- SCALAR dt_output
- const SCALAR FRICCOEF
- map< int, SCALAR > & L_IMP_Q
- map< int, SCALAR > & L_IMP_H
- map< SCALAR, string > left_times_files
- map< SCALAR, string >::const_iterator p_left_times_files
- map< int, int > L_choice_bound

- int Lbound_type
- bool is_Lbound_changed
- map< int, SCALAR > & R_IMP_Q
- map< int, SCALAR > & R_IMP_H
- map< SCALAR, string > right_times_files
- map< SCALAR, string >::const_iterator p_right_times_files
- map< int, int > R_choice_bound
- int Rbound_type
- bool is_Rbound_changed
- map< int, SCALAR > & B_IMP_Q
- map< int, SCALAR > & B_IMP_H
- map< SCALAR, string > bottom_times_files
- map< SCALAR, string >::const_iterator p_bottom_times_files
- map< int, int > B_choice_bound
- int Bbound_type
- bool is_Bbound_changed
- map< int, SCALAR > & T_IMP_Q
- map< int, SCALAR > & T_IMP_H
- map< SCALAR, string > top_times_files
- map< SCALAR, string >::const_iterator p_top_times_files
- map< int, int > T_choice_bound
- int Tbound_type
- bool is_Tbound_changed
- Parameters & fs2d_par
- TAB z
- TAB h
- TAB u
- TAB v
- TAB q1
- TAB q2
- TAB hs
- TAB us
- TAB vs
- TAB qs1
- TAB qs2
- TAB f1
- TAB f2
- TAB f3
- TAB g1
- TAB g2
- TAB g3
- TAB h1left
- TAB h1right
- TAB h2left
- TAB h2right
- TAB delz1
- TAB delz2
- TAB delzc1
- TAB delzc2

- TAB h1r
- TAB u1r
- TAB v1r
- TAB h1l
- TAB u1l
- TAB v1l
- TAB h2r
- TAB u2r
- TAB v2r
- TAB h2l
- TAB u2l
- TAB v2l
- TAB Rain
- TAB Vin_tot
- time_t start
- time_t end
- SCALAR timecomputation
- clock_t cpu_time
- int n
- SCALAR Fr
- SCALAR dt1
- SCALAR dt_max
- SCALAR cur_time
- SCALAR dt_first
- SCALAR Volrain_Tot
- SCALAR Total_volume_outflow
- SCALAR height_of_tot
- SCALAR height_Vinf_tot
- SCALAR Vol_inf_tot_cumul
- SCALAR Vol_of_tot
- Choice_condition * Lbound
- Choice_condition * Rbound
- Choice_condition * Bbound
- Choice_condition * Tbound
- Choice_output * out
- int verif
- Choice_save_specific_points * out_specific_points

4.51.1 Detailed Description

Order 1 scheme.

Class that computes the solution with a numerical scheme at order 1.

Definition at line 71 of file order1.hpp.

4.51.2 Constructor & Destructor Documentation

Order1()

```
Order1::Order1 (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file order1.cpp.

~Order1()

`Order1::~Order1 ( ) [virtual]`

Destructor.

Definition at line 257 of file order1.cpp.

4.51.3 Member Function Documentation

calcul()

`void Order1::calcul ( ) [virtual]`

Performs the numerical scheme.

Performs the first order numerical scheme.

Note

In DEBUG mode, the programme will save four other files with boundary fluxes and volumes of water.

Warning

order1: WARNING: the computation finished because the maximum number of time steps was reached (see MAX_ITER in misc.hpp)

Implements Scheme.

Definition at line 69 of file order1.cpp.

The documentation for this class was generated from the following files:

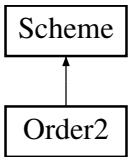
- Headers/libschemes/order1.hpp
- Sources/libschemes/order1.cpp

4.52 Order2 Class Reference

Order 2 scheme.

`#include <order2.hpp>`

Inheritance diagram for Order2:



Public Member Functions

- Order2 (Parameters &)

  Constructor.
- void calcul ()

  Performs the numerical scheme.
- virtual ~Order2 ()

  Destructor.

Public Member Functions inherited from Scheme

- Scheme (Parameters &)

  Constructor.
- virtual void calcul ()=0

  Function to be specified in each numerical scheme.
- void allocation ()

  Allocation of spatialized variables.
- void deallocation ()

  Deallocation of variables.
- void maincalcflux (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR &)

  Main calculation of the flux.
- void maincalcscheme (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, SCALAR, SCALAR, int)

  Main calculation of the scheme.
- void boundary (TAB &, TAB &, TAB &, SCALAR, const int, const int)

  Calls the boundary conditions and affects the boundary values.
- SCALAR froude_number (TAB, TAB, TAB)

  Returns the Froude number.
- virtual ~Scheme ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Scheme

- const int NXCELL
- const int NYCELL
- const int ORDER
- const SCALAR T
- const int NBTIMES
- const int SCHEME_TYPE
- const SCALAR DX
- const SCALAR DY
- const SCALAR CFL_FIX
- SCALAR DT_FIX
- SCALAR tx
- SCALAR ty
- SCALAR T_output
- SCALAR dt_output
- const SCALAR FRICCOEF

- map< int, SCALAR > & L_IMP_Q
- map< int, SCALAR > & L_IMP_H
- map< SCALAR, string > left_times_files
- map< SCALAR, string >::const_iterator p_left_times_files
- map< int, int > L_choice_bound
- int Lbound_type
- bool is_Lbound_changed
- map< int, SCALAR > & R_IMP_Q
- map< int, SCALAR > & R_IMP_H
- map< SCALAR, string > right_times_files
- map< SCALAR, string >::const_iterator p_right_times_files
- map< int, int > R_choice_bound
- int Rbound_type
- bool is_Rbound_changed
- map< int, SCALAR > & B_IMP_Q
- map< int, SCALAR > & B_IMP_H
- map< SCALAR, string > bottom_times_files
- map< SCALAR, string >::const_iterator p_bottom_times_files
- map< int, int > B_choice_bound
- int Bbound_type
- bool is_Bbound_changed
- map< int, SCALAR > & T_IMP_Q
- map< int, SCALAR > & T_IMP_H
- map< SCALAR, string > top_times_files
- map< SCALAR, string >::const_iterator p_top_times_files
- map< int, int > T_choice_bound
- int Tbound_type
- bool is_Tbound_changed
- Parameters & fs2d_par
- TAB z
- TAB h
- TAB u
- TAB v
- TAB q1
- TAB q2
- TAB hs
- TAB us
- TAB vs
- TAB qs1
- TAB qs2
- TAB f1
- TAB f2
- TAB f3
- TAB g1
- TAB g2
- TAB g3
- TAB h1left
- TAB h1right
- TAB h2left

- TAB h2right
- TAB delz1
- TAB delz2
- TAB delzc1
- TAB delzc2
- TAB h1r
- TAB u1r
- TAB v1r
- TAB h1l
- TAB u1l
- TAB v1l
- TAB h2r
- TAB u2r
- TAB v2r
- TAB h2l
- TAB u2l
- TAB v2l
- TAB Rain
- TAB Vin_tot
- time_t start
- time_t end
- SCALAR timecomputation
- clock_t cpu_time
- int n
- SCALAR Fr
- SCALAR dt1
- SCALAR dt_max
- SCALAR cur_time
- SCALAR dt_first
- SCALAR Volrain_Tot
- SCALAR Total_volume_outflow
- SCALAR height_of_tot
- SCALAR height_Vinf_tot
- SCALAR Vol_inf_tot_cumul
- SCALAR Vol_of_tot
- Choice_condition * Lbound
- Choice_condition * Rbound
- Choice_condition * Bbound
- Choice_condition * Tbound
- Choice_output * out
- int verif
- Choice_save_specific_points * out_specific_points

4.52.1 Detailed Description

Order 2 scheme.

Class that computes the solution with a numerical scheme at order 2.

Definition at line 71 of file order2.hpp.

4.52.2 Constructor & Destructor Documentation

Order2()

```
Order2::Order2 (
            Parameters & par )
```

Constructor.

Initializations, definition of the reconstruction and creation of 3 vectors for this reconstruction.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 59 of file order2.cpp.

~Order2()

```
Order2::~Order2 ( )  [virtual]
```

Destructor.

Definition at line 97 of file order2.cpp.

4.52.3 Member Function Documentation

calcul()

```
void Order2::calcul ( )  [virtual]
```

Performs the numerical scheme.

Performs the second order numerical scheme.

Note

In DEBUG mode, the programme will save another file with volumes of water.

Warning

order2: WARNING: the computation finished because the maximum number of time steps was reached (see MAX_ITER in misc.hpp)

Implements Scheme.

Definition at line 124 of file order2.cpp.

The documentation for this class was generated from the following files:

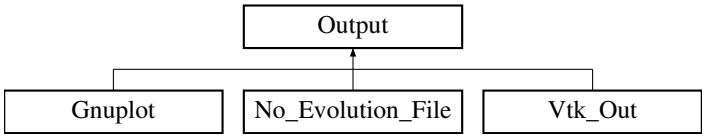
- Headers/libschemes/order2.hpp
- Sources/libschemes/order2.cpp

4.53 Output Class Reference

Output format

`#include <output.hpp>`

Inheritance diagram for Output:



Public Member Functions

- Output (Parameters &)

  Constructor.
- virtual void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)=0

  Function to be specified in each output format.
- void check_vol (const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &) const

  Saves the infiltrated and rain volumes.
- void result (const SCALAR &, const clock_t &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const int &, const SCALAR &) const

  Saves global values.
- void initial (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the initial time.
- void final (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the final time.
- SCALAR boundaries_flux (const SCALAR &, const TAB &, const TAB &, const SCALAR &, const SCALAR &, const int &, const int &)

  Saves the cumulated fluxes on the boundaries.
- void boundaries_flux_LR (const SCALAR &, const TAB &) const

  Saves the fluxes on the left and right boundaries.
- void boundaries_flux_BT (const SCALAR &, const TAB &) const

  Saves the fluxes on the bottom and top boundaries.
- SCALAR round5 (const SCALAR &) const

  Rounded value up to 10^-5.
- virtual ~Output ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- string outputDirectory
- string namefile_check_volume
- string namefile_res
- string namefile_init

- string namefile_final
- string namefile_Bound_flux
- string namefile_Bound_flux_BT
- string namefile_Bound_flux_LR

4.53.1 Detailed Description

Output format

Class that contains all the common declarations for the output formats.

Definition at line 71 of file output.hpp.

4.53.2 Constructor & Destructor Documentation

Output()

```
Output::Output (
            Parameters & par )
```

Constructor.

Defines the names of the outputs.
If run in DEBUG mode, writes the header of the file 'boundaries_flux.dat', 'check_vol.dat', 'flux_boundaries_↩ BT.dat' and 'flux_boundaries_LR.dat'.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

Impossible to open the *** file. Verify if the directory *** exists.

Note

If 'boundaries_flux.dat', 'check_vol.dat', 'flux_boundaries_BT.dat' or 'flux_boundaries_LR.dat' cannot be opened, the code will exit with failure termination code.

Definition at line 60 of file output.cpp.

~Output()

```
Output::~Output ( )  [virtual]
```

Destructor.

Definition at line 408 of file output.cpp.

4.53.3 Member Function Documentation

boundaries_flux()

```
SCALAR Output::boundaries_flux (
            const SCALAR & time,
```

```
            const TAB & flux_u,
            const TAB & flux_v,
            const SCALAR & dt,
            const SCALAR & dt_first,
            const int & ORDER,
            const int & verif )
```

Saves the cumulated fluxes on the boundaries.

Parameters

in	*time*	current time.
in	*flux_u*	flux on the left and right boundaries (m^2/s).
in	*flux_v*	flux on the bottom and top boundaries (m^2/s).
in	*dt*	current time step.
in	*dt_first*	previous time step.
in	*ORDER*	order of scheme.
in	*verif*	parameter to know if we removed the computation with the previous time step (dt_first).

Definition at line 275 of file output.cpp.

boundaries_flux_BT()

```
void Output::boundaries_flux_BT (
            const SCALAR & time,
            const TAB & BT_flux ) const
```

Saves the fluxes on the bottom and top boundaries.

Parameters

in	*time*	current time.
in	*BT_flux*	flux on the bottom and tom boundaries (m^2/s).

Definition at line 336 of file output.cpp.

boundaries_flux_LR()

```
void Output::boundaries_flux_LR (
            const SCALAR & time,
            const TAB & LR_flux ) const
```

Saves the fluxes on the left and right boundaries.

Parameters

in	*time*	current time.
in	*LR_flux*	flux on the left and right boundaries (m^2/s).

Definition at line 318 of file output.cpp.

check_vol()

```
void Output::check_vol (
```

```
            const SCALAR & time,
            const SCALAR & dt,
            const SCALAR & Vol_rain_tot,
            const SCALAR & Vol_inf,
            const SCALAR & Vol_of,
            const SCALAR & Vol_bound_tot ) const
```

Saves the infiltrated and rain volumes.

Parameters

in	*time*	current time.
in	*dt*	time step (unused).
in	*Vol_rain_tot*	total rain volume.
in	*Vol_inf*	volume of infiltrated water.
in	*Vol_of*	volume of overland flow.
in	*Vol_bound_tot*	total volume of water at the boundary.

Definition at line 202 of file output.cpp.

final()

```
void Output::final (
            const TAB & z,
            const TAB & h,
            const TAB & u,
            const TAB & v ) const
```

Saves the final time.

If the water height is too small, we replace it by 0, the velocities and discharge are null and the Froude number does not exist.

Parameters

in	*z*	topography.
in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.

Warning

Impossible to open the $***$ file. Verify if the directory $***$ exists.

Note

If huz_final.dat cannot be opened, the code will exit with failure termination code.

Definition at line 354 of file output.cpp.

initial()

```
void Output::initial (
            const TAB & z,
            const TAB & h,
```

```
            const TAB & u,
            const TAB & v ) const
```

Saves the initial time.

Parameters

in	*z*	topography.
in	*h*	water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.

Warning

Impossible to open the *** file. Verify if the directory *** exists.

Note

If huz_initial.dat cannot be opened, the code will exit with failure termination code.

Definition at line 167 of file output.cpp.

result()

```
void Output::result (
            const SCALAR & time,
            const clock_t & cpu,
            const SCALAR & Vol_rain,
            const SCALAR & Vol_inf,
            const SCALAR & Vol_of,
            const SCALAR & FROUDE,
            const int & NBITER,
            const SCALAR & vol_output ) const
```

Saves global values.

Parameters

in	*time*	elapsed time.
in	*cpu*	CPU time.
in	*Vol_rain*	total rain volume.
in	*Vol_inf*	total volume of infiltrated water.
in	*Vol_of*	total volume of overland flow.
in	*FROUDE*	mean Froude number (in space) at the final time.
in	*NBITER*	number of time steps.
in	*vol_output*	total outflow volume at the boundary.

Warning

Impossible to open the *** file. Verify if the directory *** exists.

Note

If results.dat cannot be opened, the code will exit with failure termination code.

Definition at line 223 of file output.cpp.

round5()

```
SCALAR Output::round5 (
            const SCALAR & x ) const
```

Rounded value up to 10^-5.

Computes the rounded value to 5 decimal places

Parameters

in	*x*	value to be rounded

Returns

y rounded value

Definition at line 397 of file output.cpp.

write()

```
virtual void Output::write (
            const TAB & ,
            const TAB & ,
            const TAB & ,
            const TAB & ,
            const SCALAR &  )  [pure virtual]
```

Function to be specified in each output format.

Implemented in Gnuplot, No_Evolution_File, and Vtk_Out.

4.53.4 Member Data Documentation

DX

```
const SCALAR Output::DX  [protected]
```

Space step in the first (x) direction.

Definition at line 114 of file output.hpp.

DY

```
const SCALAR Output::DY  [protected]
```

Space step in the second (y) direction.

Definition at line 116 of file output.hpp.

namefile_Bound_flux

`string Output::namefile_Bound_flux [protected]`

Name of the file where the cumulated boundary fluxes are saved.

Definition at line 128 of file output.hpp.

namefile_Bound_flux_BT

`string Output::namefile_Bound_flux_BT [protected]`

Name of the file where the bottom and top boundary fluxes are saved.

Definition at line 130 of file output.hpp.

namefile_Bound_flux_LR

`string Output::namefile_Bound_flux_LR [protected]`

Name of the file where the left and right boundary fluxes are saved.

Definition at line 132 of file output.hpp.

namefile_check_volume

`string Output::namefile_check_volume [protected]`

Name of the file where the verification of volumes is saved.

Definition at line 120 of file output.hpp.

namefile_final

`string Output::namefile_final [protected]`

Name of the file where the final time is saved.

Definition at line 126 of file output.hpp.

namefile_init

`string Output::namefile_init [protected]`

Name of the file where the initialization is saved.

Definition at line 124 of file output.hpp.

namefile_res

`string Output::namefile_res [protected]`

Name of the file where the global results are saved.

Definition at line 122 of file output.hpp.

NXCELL

`const int Output::NXCELL [protected]`

Number of cells in space in the first (x) direction.

Definition at line 110 of file output.hpp.

NYCELL

```
const int Output::NYCELL [protected]
```

Number of cells in space in the second (y) direction.
Definition at line 112 of file output.hpp.

outputDirectory

```
string Output::outputDirectory [protected]
```

Name of the output directory.
Definition at line 118 of file output.hpp.
The documentation for this class was generated from the following files:

- Headers/libsave/output.hpp
- Sources/libsave/output.cpp

4.54 Parameters Class Reference

Gets parameters.

```
#include <parameters.hpp>
```

Public Member Functions

- Parameters ()
  Constructor.
- void setparameters (const char *)
  Sets the parameters.
- virtual ~Parameters ()
  Destructor.
- int get_Nxcell () const
  Gives the number of cells in space along x.
- int get_Nycell () const
  Gives the number of cells in space along y.
- SCALAR get_T () const
  Gives the final time.
- int get_nbtimes () const
  Gives the number of times saved.
- int get_scheme_type () const
  Gives the choice of type of scheme (fixed cfl or fixed dt)
- SCALAR get_dtfix () const
  Gives the fixed time step from the parameters.txt file.
- SCALAR get_cflfix () const
  Gives the cfl of the scheme.
- SCALAR get_dx () const
  Gives the space step along x.
- SCALAR get_dy () const
  Gives the space step along y.
- SCALAR get_L () const
  Gives the length of the domain in the first (x) direction.

- SCALAR get_l () const

  Gives the length of the domain in the second (y) direction.
- map< int, int > & get_Lbound ()

  Gives the value corresponding to the left boundary condition.
- map< SCALAR, string > & get_times_files_Lbound ()

  Gives the list of files and time values corresponding to the left boundary condition.
- int get_type_Lbound () const

  Gives the type (constant or file) of the left boundary condition.
- map< int, SCALAR > & get_left_imp_discharge ()

  Gives the value of the imposed discharge in left bc.
- map< int, SCALAR > & get_left_imp_h ()

  Gives the value of the imposed water height in left bc.
- map< int, int > & get_Rbound ()

  Gives the value corresponding to the right boundary condition.
- map< SCALAR, string > & get_times_files_Rbound ()

  Gives the list of files and time values corresponding to the right boundary condition.
- int get_type_Rbound () const

  Gives the type (constant or file) of right boundary condition.
- map< int, SCALAR > & get_right_imp_discharge ()

  Gives the value of the imposed discharge in right bc.
- map< int, SCALAR > & get_right_imp_h ()

  Gives the value of the imposed water height in right bc.
- map< int, int > & get_Bbound ()

  Gives the value corresponding to the bottom boundary condition.
- map< SCALAR, string > & get_times_files_Bbound ()

  Gives the list of files and time values corresponding to the bottom boundary condition.
- int get_type_Bbound () const

  Gives the type (constant or file) of bottom boundary condition.
- map< int, SCALAR > & get_bottom_imp_discharge ()

  Gives the value of the imposed discharge in bottom bc.
- map< int, SCALAR > & get_bottom_imp_h ()

  Gives the value of the imposed water height in bottom bc.
- map< int, int > & get_Tbound ()

  Gives the value corresponding to the top boundary condition.
- map< SCALAR, string > & get_times_files_Tbound ()

  Gives the list of files and time values corresponding to the top boundary condition.
- int get_type_Tbound () const

  Gives the type (constant or file) of bottom boundary condition.
- map< int, SCALAR > & get_top_imp_discharge ()

  Gives the value of the imposed discharge in top bc.
- map< int, SCALAR > & get_top_imp_h ()

  Gives the value of the imposed water height in top bc.
- int get_flux () const

  Gives the value corresponding to the flux.
- int get_order () const

  Gives the order of the scheme.
- int get_rec () const

Gives the value corresponding to the reconstruction.

- int get_fric () const

  Gives the value corresponding to the friction law.

- int get_lim () const

  Gives the value corresponding to the limiter.

- int get_inf () const

  Gives the choice of infiltration model.

- SCALAR get_amortENO () const

  Gives the value of the amortENO parameter.

- SCALAR get_modifENO () const

  Gives the value of the modifENO parameter.

- SCALAR get_friccoef () const

  Gives the value of the friction coefficient.

- int get_fric_init () const

  Gives the value characterizing the spatialization (or not) of the friction coefficient.

- string get_KcNameFile (void) const

  Gives the full name of the file containing the hydraulic conductivity of the crust.

- string get_KcNameFileS () const

  Gives the name of the file containing the hydraulic conductivity of the crust.

- string get_KsNameFile (void) const

  Gives the full name of the file containing the hydraulic conductivity of the surface.

- string get_KsNameFileS () const

  Gives the name of the file containing the hydraulic conductivity of the surface.

- string get_dthetaNameFile (void) const

  Gives the full name of the file containing the initial water deficit.

- string get_dthetaNameFileS () const

  Gives the name of the file containing the initial water deficit.

- string get_PsiNameFile (void) const

  Gives the full name of the file containing the load pressure.

- string get_PsiNameFileS () const

  Gives the name of the file containing the load pressure.

- string get_zcrustNameFile (void) const

  Gives the full name of the file containing the thickness of the crust.

- string get_zcrustNameFileS () const

  Gives the name of the file containing the thickness of the crust.

- string get_imaxNameFile (void) const

  Gives the full name of the file containing the maximum infiltration rate.

- string get_imaxNameFileS () const

  Gives the name of the file containing the maximum infiltration rate.

- int get_Kc_init () const

  Gives the value characterizing the spatialization (or not) of the hydraulic conductivity of the crust.

- SCALAR get_Kc_coef () const

  Gives the value of the hydraulic conductivity of the crust.

- int get_Ks_init () const

  Gives the value characterizing the spatialization (or not) of the hydraulic conductivity of the soil.

- SCALAR get_Ks_coef () const

  Gives the value of the hydraulic conductivity of the soil.

- int get_dtheta_init () const

  Gives the value characterizing the spatialization (or not) of the initial water deficit.
- SCALAR get_dtheta_coef () const

  Gives the value of the initial water deficit.
- int get_Psi_init () const

  Gives the value characterizing the spatialization (or not) of the load pressure.
- SCALAR get_Psi_coef () const

  Gives the value of the load pressure.
- int get_zcrust_init () const

  Gives the value characterizing the spatialization (or not) of the thickness of the crust.
- SCALAR get_zcrust_coef () const

  Gives the value of the thickness of the crust.
- int get_imax_init () const

  Gives the value characterizing the spatialization (or not) of the maximum infiltration rate.
- SCALAR get_imax_coef () const

  Gives the value of the maximum infiltration rate.
- int get_topo () const

  Gives the value corresponding to the choice of topography.
- int get_huv () const

  Gives the value corresponding to the choice of initialization of h, u and v.
- int get_rain () const

  Gives the value corresponding to the choice of initialization of rain.
- string get_topographyNameFile (void) const

  Gives the full name of the file containing the topography.
- string get_topographyNameFileS () const

  Gives the name of the file containing the topography.
- string get_huvNameFile (void) const

  Gives the full name of the file containing the water height (h) and the velocities (u and v)
- string get_huvNameFileS (void) const

  Gives the name of the file containing the water height (h) and the velocities (u and v)
- string get_rainNameFile (void) const

  Gives the full name of the file containing the rain.
- string get_rainNameFileS (void) const

  Gives the name of the file containing the rain.
- string get_frictionNameFile (void) const

  Gives the full name of the file containing the friction coefficient.
- string get_frictionNameFileS (void) const

  Gives the name of the file containing the friction coefficient.
- string get_outputDirectory (void) const

  Gives the output directory with the suffix.
- string get_suffix (void) const

  Gives the suffix for the 'Outputs' directory.
- int get_output () const

  Gives the value corresponding to the choice of the format of the Output file.
- int get_choice_specific_point () const

  Gives the choice of specific points to be saved.
- int get_choice_dt_specific_points () const

Gives the choice of time step for specific points to be saved.

- SCALAR get_dt_specific_points () const

  Gives time step for specific points to be saved

- void fill_array (TAB &, const SCALAR) const

  Fills the TAB array with a SCALAR.

- void fill_array (TAB &, string) const

  Fills the TAB array with the values contained in a file

- bool is_coord_in_file_valid (const SCALAR &x, const SCALAR &y, int line_number, string namefile) const

  Verifies if the coordinates x and y exist in the mesh

- SCALAR get_x_coord (void) const

  Gives x coordinate of the specific point to be saved.

- SCALAR get_y_coord (void) const

  Gives y coordinate of the specific point to be saved.

- string get_list_pointNameFile (void) const

  Gives the full name of file containing the list of the specific points.

- string get_list_pointNameFileS (void) const

  Gives the name of the file containing the list of the specific points.

- map< int, int > fill_array_bc_inhomogeneous (string, string, char, map< int, SCALAR > &, map< int, SCALAR > &)

  Returns the vector containing the choice of boundary conditions. The two containers will contain the values of discharge and water heights imposed for each point.

- map< SCALAR, string > verif_file_bc_inhomogeneous (string, string, char, map< int, int > &, map< int, SCALAR > &, map< int, SCALAR > &)

  Returns the list of files and time values corresponding to the boundary condition. Moreover, fills the containers that will contain the choice of boundary conditions, the values of discharge and water heights imposed for each point.

- string get_path_input_directory (void) const

Protected Attributes

- SCALAR cfl_fix
- SCALAR dt_fix
- int scheme_type
- int Nxcell
- int Nycell
- int nbtimes
- SCALAR T
- SCALAR dx
- SCALAR dy
- SCALAR L
- SCALAR l
- map< int, int > Lbound
- map< int, SCALAR > left_imp_discharge
- map< int, SCALAR > left_imp_h
- map< SCALAR, string > left_times_files
- int Lbound_type
- map< int, int > Rbound

- map< int, SCALAR > right_imp_discharge
- map< int, SCALAR > right_imp_h
- map< SCALAR, string > right_times_files
- int Rbound_type
- map< int, int > Bbound
- map< int, SCALAR > bottom_imp_discharge
- map< int, SCALAR > bottom_imp_h
- map< SCALAR, string > bottom_times_files
- int Bbound_type
- map< int, int > Tbound
- map< int, SCALAR > top_imp_discharge
- map< int, SCALAR > top_imp_h
- map< SCALAR, string > top_times_files
- int Tbound_type
- int flux
- int order
- int rec
- int fric
- int lim
- int inf
- int topo
- int huv_init
- int rain
- int Kc_init
- int Ks_init
- int dtheta_init
- int Psi_init
- int zcrust_init
- int imax_init
- int output_format
- SCALAR amortENO
- SCALAR modifENO
- int fric_init
- SCALAR friccoef
- SCALAR Kc_coef
- SCALAR Ks_coef
- SCALAR dtheta_coef
- SCALAR Psi_coef
- SCALAR zcrust_coef
- SCALAR imax_coef
- string topography_namefile
- string topo_NF
- string huv_namefile
- string huv_NF
- string fric_namefile
- string fric_NF
- string rain_namefile
- string rain_NF
- string Kc_namefile

- string Kc_NF
- string Ks_namefile
- string Ks_NF
- string dtheta_namefile
- string dtheta_NF
- string Psi_namefile
- string Psi_NF
- string zcrust_namefile
- string zcrust_NF
- string imax_namefile
- string imax_NF
- string output_directory
- string suffix_outputs
- int Choice_points
- SCALAR x_coord
- SCALAR y_coord
- string list_point_NF
- string list_point_namefile
- VECT list_points_x
- VECT list_points_y
- int Choice_dt_specific_points
- SCALAR dt_specific_points
- string path_input_directory

4.54.1 Detailed Description

Gets parameters.

Class that reads the parameters, checks their values and contains all the common declarations to get the values of the parameters.

Definition at line 73 of file parameters.hpp.

4.54.2 Constructor & Destructor Documentation

Parameters()

```
Parameters::Parameters ( )
```

Constructor.

Definition at line 3228 of file parameters.cpp.

~Parameters()

```
Parameters::~Parameters ( ) [virtual]
```

Destructor.

Definition at line 3231 of file parameters.cpp.

4.54.3 Member Function Documentation

fill_array() [1/2]

```
void Parameters::fill_array (
            TAB & myarray,
            const SCALAR myvalue ) const
```

Fills the TAB array with a SCALAR.

Fills an array with a constant value.

Parameters

in,out	*myarray*	array to fill.
in	*myvalue*	value.

Definition at line 2651 of file parameters.cpp.

fill_array() [2/2]

```
void Parameters::fill_array (
            TAB & myarray,
            string namefile ) const
```

Fills the TAB array with the values contained in a file

Fills an array with the values given in the file

Parameters

in,out	*myarray*	array to fill.
in	*namefile*	name of the file containing the values to be inserted into the array.

Warning

***: ERROR: cannot open the file.

***: ERROR: the number of data in this file is too big/small.

***: ERROR: line ***.

***: ERROR: the value for the point x = *** y = *** is missing.

***: WARNING: line *** ; a commentary should begin with the # symbol.

Note

If the array cannot be filled correctly, the code will exit with failure termination code.

Definition at line 2665 of file parameters.cpp.

fill_array_bc_inhomogeneous()

```
map< int, int > Parameters::fill_array_bc_inhomogeneous (
            string Bc_NF,
            string path_input_directory,
            char BC,
            map< int, SCALAR > & imp_q,
            map< int, SCALAR > & imp_h )
```

Returns the vector containing the choice of boundary conditions. The two containers will contain the values of discharge and water heights imposed for each point.

Extracts from the Bc_NF file the type of boundary condition, discharge and water heights imposed for each point.

Parameters

in	*Bc_NF*	name of the file containing the values to be verified.
in	*path_input_directory*	path of directory containing Bc_NF file.
in	*BC*	represents the boundary condition which we handle
out	*imp_q*	container containing discharges [m3/s]
out	*imp_h*	container containing water heights [m]

Warning

***: ERROR: cannot open the file.

***: ERROR: it is necessary to specify a file for this choice of initialization of boundary condition.

***: ERROR: the file does not exist in the directory Inputs.

***: ERROR: the number of data in this file is not good.

***: ERROR: the format of the file is not good.

***: ERROR: line ***.

***: ERROR: the boundary does not exist!

Returns

the vector containing the choice of boundary conditions.

Note

If the containers cannot be filled correctly, the code will exit with failure termination code.

Definition at line 2934 of file parameters.cpp.

get_amortENO()

```
SCALAR Parameters::get_amortENO ( ) const
```

Gives the value of the amortENO parameter.

Returns

The value of the amortENO parameter Parameters::amortENO.

Definition at line 2219 of file parameters.cpp.

get_Bbound()

```
map< int, int > & Parameters::get_Bbound ( )
```

Gives the value corresponding to the bottom boundary condition.

Returns

The value corresponding to the bottom boundary condition Parameters::Bbound.

Definition at line 2075 of file parameters.cpp.

get_bottom_imp_discharge()

```
map< int, SCALAR > & Parameters::get_bottom_imp_discharge ( )
```

Gives the value of the imposed discharge in bottom bc.

Returns

The value of the imposed discharge per cell in the bottom boundary condition, that is Parameters::bottom_imp_discharge / Parameters::L.

Definition at line 2098 of file parameters.cpp.

get_bottom_imp_h()

```
map< int, SCALAR > & Parameters::get_bottom_imp_h ( )
```

Gives the value of the imposed water height in bottom bc.

Returns

The value of the imposed water height in the bottom boundary condition Parameters::bottom_imp_h.

Definition at line 2108 of file parameters.cpp.

get_cflfix()

```
SCALAR Parameters::get_cflfix ( ) const
```

Gives the cfl of the scheme.

Returns

The fixed cfl Parameters::cfl_fix.

Definition at line 1915 of file parameters.cpp.

get_choice_dt_specific_points()

```
int Parameters::get_choice_dt_specific_points ( ) const
```

Gives the choice of time step for specific points to be saved.

Returns

The choice of times saved for the specific points.

Definition at line 2914 of file parameters.cpp.

get_choice_specific_point()

```
int Parameters::get_choice_specific_point ( ) const
```

Gives the choice of specific points to be saved.

Returns

The choice for saving specific points.

Definition at line 2904 of file parameters.cpp.

get_dt_specific_points()

```
SCALAR Parameters::get_dt_specific_points ( ) const
```

Gives time step for specific points to be saved

Returns

The time step for saving specific points.

Definition at line 2924 of file parameters.cpp.

get_dtfix()

```
SCALAR Parameters::get_dtfix ( ) const
```

Gives the fixed time step from the parameters.txt file.

Returns

The fixed space step Parameters::dx_fix.

Definition at line 1905 of file parameters.cpp.

get_dtheta_coef()

```
SCALAR Parameters::get_dtheta_coef ( ) const
```

Gives the value of the initial water deficit.

Returns

The value of dtheta Parameters::dtheta_coef.

Definition at line 2479 of file parameters.cpp.

get_dtheta_init()

```
int Parameters::get_dtheta_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the initial water deficit.

Returns

The value corresponding to the initialization of dtheta Parameters::dtheta_init.

Definition at line 2469 of file parameters.cpp.

get_dthetaNameFile()

```
string Parameters::get_dthetaNameFile (
            void  ) const
```

Gives the full name of the file containing the initial water deficit.

Returns

The dtheta path for the initialization + Input directory Parameters::dtheta_namefile.

Definition at line 2489 of file parameters.cpp.

get_dthetaNameFileS()

```
string Parameters::get_dthetaNameFileS ( ) const
```

Gives the name of the file containing the initial water deficit.

Returns

The dtheta namefile for the initialization (inside the Input directory) Parameters::dtheta_NF.

Definition at line 2499 of file parameters.cpp.

get_dx()

```
SCALAR Parameters::get_dx ( ) const
```

Gives the space step along x.

Returns

The space step in the first (x) direction Parameters::dx.

Definition at line 1925 of file parameters.cpp.

get_dy()

```
SCALAR Parameters::get_dy ( ) const
```

Gives the space step along y.

Returns

The space step in the second (y) direction Parameters::dy.

Definition at line 1935 of file parameters.cpp.

get_flux()

```
int Parameters::get_flux ( ) const
```

Gives the value corresponding to the flux.

Returns

The value corresponding to the flux Parameters::flux.

Definition at line 2168 of file parameters.cpp.

get_fric()

```
int Parameters::get_fric ( ) const
```

Gives the value corresponding to the friction law.

Returns

The value corresponding to the friction law Parameters::fric.

Definition at line 2198 of file parameters.cpp.

get_fric_init()

```
int Parameters::get_fric_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the friction coefficient.

Returns

The value corresponding to the friction coefficient Parameters::fric_init.

Definition at line 2249 of file parameters.cpp.

get_friccoef()

```
SCALAR Parameters::get_friccoef ( ) const
```

Gives the value of the friction coefficient.

Returns

The value of the friction coefficient Parameters::friccoef.

Definition at line 2279 of file parameters.cpp.

get_frictionNameFile()

```
string Parameters::get_frictionNameFile (
            void  ) const
```

Gives the full name of the file containing the friction coefficient.

Returns

The friction coefficient path + Input directory Parameters::fric_namefile.

Definition at line 2259 of file parameters.cpp.

get_frictionNameFileS()

```
string Parameters::get_frictionNameFileS (
            void  ) const
```

Gives the name of the file containing the friction coefficient.

Returns

The friction coefficient namefile (inside the Input directory) Parameters::fric_NF.

Definition at line 2269 of file parameters.cpp.

get_huv()

```
int Parameters::get_huv ( ) const
```

Gives the value corresponding to the choice of initialization of h, u and v.

Returns

The value corresponding to the initialization of h and u,v Parameters::huv_init.

Definition at line 2319 of file parameters.cpp.

get_huvNameFile()

```
string Parameters::get_huvNameFile (
            void  ) const
```

Gives the full name of the file containing the water height (h) and the velocities (u and v)

Returns

The h and u,v path for the initialization + Input directory Parameters::huv_namefile.

Definition at line 2329 of file parameters.cpp.

get_huvNameFileS()

```
string Parameters::get_huvNameFileS (
            void  ) const
```

Gives the name of the file containing the water height (h) and the velocities (u and v)

Returns

The h and u namefile for the initialization (inside the Input directory) Parameters::huv_NF.

Definition at line 2339 of file parameters.cpp.

get_imax_coef()

```
SCALAR Parameters::get_imax_coef ( ) const
```

Gives the value of the maximum infiltration rate.

Returns

The value of imax Parameters::imax_coef.

Definition at line 2599 of file parameters.cpp.

get_imax_init()

```
int Parameters::get_imax_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the maximum infiltration rate.

Returns

The value corresponding to the initialization of imax Parameters::imax_init.

Definition at line 2589 of file parameters.cpp.

get_imaxNameFile()

```
string Parameters::get_imaxNameFile (
            void  ) const
```

Gives the full name of the file containing the maximum infiltration rate.

Returns

The imax path for the initialization + Input directory Parameters::imax_namefile.

Definition at line 2609 of file parameters.cpp.

get_imaxNameFileS()

```
string Parameters::get_imaxNameFileS ( ) const
```

Gives the name of the file containing the maximum infiltration rate.

Returns

The imax namefile for the initialization (inside the Input directory) Parameters::imax_NF.

Definition at line 2619 of file parameters.cpp.

get_inf()

```
int Parameters::get_inf ( ) const
```

Gives the choice of infiltration model.

Returns

The value corresponding to the infiltration Parameters::inf.

Definition at line 2239 of file parameters.cpp.

get_Kc_coef()

```
SCALAR Parameters::get_Kc_coef ( ) const
```

Gives the value of the hydraulic conductivity of the crust.

Returns

The value of Kc Parameters::Kc_coef.

Definition at line 2399 of file parameters.cpp.

get_Kc_init()

```
int Parameters::get_Kc_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the hydraulic conductivity of the crust.

Returns

The value corresponding to the initialization of Kc Parameters::Kc_init.

Definition at line 2389 of file parameters.cpp.

get_KcNameFile()

```
string Parameters::get_KcNameFile (
            void  ) const
```

Gives the full name of the file containing the hydraulic conductivity of the crust.

Returns

The Kc path for the initialization + Input directory Parameters::Kc_namefile.

Definition at line 2409 of file parameters.cpp.

get_KcNameFileS()

```
string Parameters::get_KcNameFileS ( ) const
```

Gives the name of the file containing the hydraulic conductivity of the crust.

Returns

The Kc namefile for the initialization (inside the Input directory) Parameters::Kc_NF.

Definition at line 2419 of file parameters.cpp.

get_Ks_coef()

```
SCALAR Parameters::get_Ks_coef ( ) const
```

Gives the value of the hydraulic conductivity of the soil.

Returns

The value of Ks Parameters::Ks_coef.

Definition at line 2439 of file parameters.cpp.

get_Ks_init()

```
int Parameters::get_Ks_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the hydraulic conductivity of the soil.

Returns

The value corresponding to the initialization of Ks Parameters::Ks_init.

Definition at line 2429 of file parameters.cpp.

get_KsNameFile()

```
string Parameters::get_KsNameFile (
            void  ) const
```

Gives the full name of the file containing the hydraulic conductivity of the surface.

Returns

The Ks path for the initialization + Input directory Parameters::Ks_namefile.

Definition at line 2449 of file parameters.cpp.

get_KsNameFileS()

```
string Parameters::get_KsNameFileS ( ) const
```

Gives the name of the file containing the hydraulic conductivity of the surface.

Returns

The Ks namefile for the initialization (inside the Input directory) Parameters::Ks_NF.

Definition at line 2459 of file parameters.cpp.

get_L()

`SCALAR Parameters::get_L ( ) const`

Gives the length of the domain in the first (x) direction.

Returns

the length of the domain in the first (x) direction.

Definition at line 1945 of file parameters.cpp.

get_l()

`SCALAR Parameters::get_l ( ) const`

Gives the length of the domain in the second (y) direction.

Returns

the length of the domain in the second (y) direction.

Definition at line 1955 of file parameters.cpp.

get_Lbound()

`map< int, int > & Parameters::get_Lbound ( )`

Gives the value corresponding to the left boundary condition.

Returns

The value corresponding to the left boundary condition Parameters::Lbound.

Definition at line 1976 of file parameters.cpp.

get_left_imp_discharge()

`map< int, SCALAR > & Parameters::get_left_imp_discharge ( )`

Gives the value of the imposed discharge in left bc.

Returns

The value of the imposed discharge per cell in the left boundary condition, that is Parameters::left_imp_discharge / Parameters::l.

Definition at line 1996 of file parameters.cpp.

get_left_imp_h()

`map< int, SCALAR > & Parameters::get_left_imp_h ( )`

Gives the value of the imposed water height in left bc.

Returns

The value of the imposed water height in the left boundary condition Parameters::left_imp_h.

Definition at line 2007 of file parameters.cpp.

get_lim()

```
int Parameters::get_lim ( ) const
```

Gives the value corresponding to the limiter.

Returns

The value corresponding to the limiter Parameters::lim.

Definition at line 2209 of file parameters.cpp.

get_list_pointNameFile()

```
string Parameters::get_list_pointNameFile (
            void  ) const
```

Gives the full name of file containing the list of the specific points.

Returns

The name of file containing the list of the specific points path for the initialization + Input directory Parameters::list_point_namefile.

Definition at line 2885 of file parameters.cpp.

get_list_pointNameFileS()

```
string Parameters::get_list_pointNameFileS (
            void  ) const
```

Gives the name of the file containing the list of the specific points.

Returns

The name of file containing list of the specific points for the initialization (inside the Input directory) Parameters::list_point_NF.

Definition at line 2894 of file parameters.cpp.

get_modifENO()

```
SCALAR Parameters::get_modifENO ( ) const
```

Gives the value of the modifENO parameter.

Returns

The value of the modifENO parameter Parameters::modifENO.

Definition at line 2229 of file parameters.cpp.

get_nbtimes()

```
int Parameters::get_nbtimes ( ) const
```

Gives the number of times saved.

Returns

The number of times saved Parameters::nbtimes.

Definition at line 1885 of file parameters.cpp.

get_Nxcell()

```
int Parameters::get_Nxcell ( ) const
```

Gives the number of cells in space along x.

Returns

The number of cells in space in the first (x) direction Parameters::Nxcell.

Definition at line 1855 of file parameters.cpp.

get_Nycell()

```
int Parameters::get_Nycell ( ) const
```

Gives the number of cells in space along y.

Returns

The number of cells in space in the second (y) direction Parameters::Nycell.

Definition at line 1865 of file parameters.cpp.

get_order()

```
int Parameters::get_order ( ) const
```

Gives the order of the scheme.

Returns

The order of the scheme Parameters::order.

Definition at line 2178 of file parameters.cpp.

get_output()

```
int Parameters::get_output ( ) const
```

Gives the value corresponding to the choice of the format of the Output file.

Returns

The type of output Parameters::output_format.

Definition at line 2639 of file parameters.cpp.

get_outputDirectory()

```
string Parameters::get_outputDirectory (
            void  ) const
```

Gives the output directory with the suffix.

Returns

The output directory with the suffix Parameters::output_directory.

Definition at line 2379 of file parameters.cpp.

get_path_input_directory()

```
string Parameters::get_path_input_directory (
            void  ) const
```

Returns

The Name of the input directory.

Definition at line 3216 of file parameters.cpp.

get_Psi_coef()

```
SCALAR Parameters::get_Psi_coef ( ) const
```

Gives the value of the load pressure.

Returns

The value of Psi Parameters::Psi_coef.

Definition at line 2519 of file parameters.cpp.

get_Psi_init()

```
int Parameters::get_Psi_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the load pressure.

Returns

The value corresponding to the initialization of Psi Parameters::Psi_init.

Definition at line 2509 of file parameters.cpp.

get_PsiNameFile()

```
string Parameters::get_PsiNameFile (
            void  ) const
```

Gives the full name of the file containing the load pressure.

Returns

The Psi path for the initialization + Input directory Parameters::Psi_namefile.

Definition at line 2529 of file parameters.cpp.

get_PsiNameFileS()

```
string Parameters::get_PsiNameFileS ( ) const
```

Gives the name of the file containing the load pressure.

Returns

The Psi namefile for the initialization (inside the Input directory) Parameters::Psi_NF.

Definition at line 2539 of file parameters.cpp.

get_rain()

```
int Parameters::get_rain ( ) const
```

Gives the value corresponding to the choice of initialization of rain.

Returns

The value corresponding to the initialization of the rain Parameters::rain.

Definition at line 2349 of file parameters.cpp.

get_rainNameFile()

```
string Parameters::get_rainNameFile (
            void  ) const
```

Gives the full name of the file containing the rain.

Returns

The rain path for the initialization + Input directory Parameters::rain_namefile.

Definition at line 2359 of file parameters.cpp.

get_rainNameFileS()

```
string Parameters::get_rainNameFileS (
            void  ) const
```

Gives the name of the file containing the rain.

Returns

The rain namefile for the initialization (inside the Input directory) Parameters::rain_NF.

Definition at line 2369 of file parameters.cpp.

get_Rbound()

```
map< int, int > & Parameters::get_Rbound ( )
```

Gives the value corresponding to the right boundary condition.

Returns

The value corresponding to the right boundary condition Parameters::Rbound.

Definition at line 2025 of file parameters.cpp.

get_rec()

```
int Parameters::get_rec ( ) const
```

Gives the value corresponding to the reconstruction.

Returns

The value corresponding to the reconstruction Parameters::rec.

Definition at line 2188 of file parameters.cpp.

get_right_imp_discharge()

```
map< int, SCALAR > & Parameters::get_right_imp_discharge ( )
```

Gives the value of the imposed discharge in right bc.

Returns

The value of the imposed discharge per cell in the right boundary condition, that is Parameters::right_imp_discharge / Parameters::l.

Definition at line 2045 of file parameters.cpp.

get_right_imp_h()

```
map< int, SCALAR > & Parameters::get_right_imp_h ( )
```

Gives the value of the imposed water height in right bc.

Returns

The value of the imposed water height in the right boundary condition Parameters::right_imp_h.

Definition at line 2055 of file parameters.cpp.

get_scheme_type()

```
int Parameters::get_scheme_type ( ) const
```

Gives the choice of type of scheme (fixed cfl or fixed dt)

Returns

The type of scheme Parameters::scheme_type.

Definition at line 1895 of file parameters.cpp.

get_suffix()

```
string Parameters::get_suffix (
            void  ) const
```

Gives the suffix for the 'Outputs' directory.

Returns

The suffix (for the output directory) Parameters::suffix_outputs.

Definition at line 2629 of file parameters.cpp.

get_T()

```
SCALAR Parameters::get_T ( ) const
```

Gives the final time.

Returns

The final time Parameters::T.

Definition at line 1875 of file parameters.cpp.

get_Tbound()

```
map< int, int > & Parameters::get_Tbound ( )
```

Gives the value corresponding to the top boundary condition.

Returns

The value corresponding to the top boundary condition Parameters::Tbound.

Definition at line 2128 of file parameters.cpp.

get_times_files_Bbound()

```
map< SCALAR, string > & Parameters::get_times_files_Bbound ( )
```

Gives the list of files and time values corresponding to the bottom boundary condition.

Returns

The value corresponding to the bottom boundary condition Parameters::Bbound.

Definition at line 2085 of file parameters.cpp.

get_times_files_Lbound()

```
map< SCALAR, string > & Parameters::get_times_files_Lbound ( )
```

Gives the list of files and time values corresponding to the left boundary condition.

Returns

The value corresponding to the bottom boundary condition Parameters::Bbound.

Definition at line 1986 of file parameters.cpp.

get_times_files_Rbound()

```
map< SCALAR, string > & Parameters::get_times_files_Rbound ( )
```

Gives the list of files and time values corresponding to the right boundary condition.

Returns

The value corresponding to the bottom boundary condition Parameters::Bbound.

Definition at line 2035 of file parameters.cpp.

get_times_files_Tbound()

```
map< SCALAR, string > & Parameters::get_times_files_Tbound ( )
```

Gives the list of files and time values corresponding to the top boundary condition.

Returns

The value corresponding to the bottom boundary condition Parameters::Bbound.

Definition at line 2138 of file parameters.cpp.

get_top_imp_discharge()

```
map< int, SCALAR > & Parameters::get_top_imp_discharge ( )
```

Gives the value of the imposed discharge in top bc.

Returns

The value of the imposed discharge per cell in the top boundary condition, that is Parameters::top_imp_discharge / Parameters::L.

Definition at line 2149 of file parameters.cpp.

get_top_imp_h()

```
map< int, SCALAR > & Parameters::get_top_imp_h ( )
```

Gives the value of the imposed water height in top bc.

Returns

The value of the imposed water height in the bottom boundary condition Parameters::top_imp_h.

Definition at line 2159 of file parameters.cpp.

get_topo()

```
int Parameters::get_topo ( ) const
```

Gives the value corresponding to the choice of topography.

Returns

The value corresponding to the topography Parameters::topo.

Definition at line 2309 of file parameters.cpp.

get_topographyNameFile()

```
string Parameters::get_topographyNameFile (
            void  ) const
```

Gives the full name of the file containing the topography.

Returns

The topography path + Input directory Parameters::topography_namefile.

Definition at line 2289 of file parameters.cpp.

get_topographyNameFileS()

```
string Parameters::get_topographyNameFileS (
            void  ) const
```

Gives the name of the file containing the topography.

Returns

The topography namefile (inside the Input directory) Parameters::topo_NF.

Definition at line 2299 of file parameters.cpp.

get_type_Bbound()

```
int Parameters::get_type_Bbound ( ) const
```

Gives the type (constant or file) of bottom boundary condition.

Returns

The value corresponding to the left boundary condition Parameters::Lbound.

Definition at line 2064 of file parameters.cpp.

get_type_Lbound()

```
int Parameters::get_type_Lbound ( ) const
```

Gives the type (constant or file) of the left boundary condition.

Returns

The value corresponding to the left boundary condition Parameters::Lbound.

Definition at line 1965 of file parameters.cpp.

get_type_Rbound()

```
int Parameters::get_type_Rbound ( ) const
```

Gives the type (constant or file) of right boundary condition.

Returns

The value corresponding to the left boundary condition Parameters::Lbound.

Definition at line 2016 of file parameters.cpp.

get_type_Tbound()

```
int Parameters::get_type_Tbound ( ) const
```

Gives the type (constant or file) of bottom boundary condition.

Returns

The value corresponding to the left boundary condition Parameters::Lbound.

Definition at line 2117 of file parameters.cpp.

get_x_coord()

```
SCALAR Parameters::get_x_coord (
            void  ) const
```

Gives x coordinate of the specific point to be saved.

Returns

x coordinate of the specific point to be saved.

Definition at line 2865 of file parameters.cpp.

get_y_coord()

```
SCALAR Parameters::get_y_coord (
            void  ) const
```

Gives y coordinate of the specific point to be saved.

Returns

y coordinate of the specific point to be saved.

Definition at line 2875 of file parameters.cpp.

get_zcrust_coef()

```
SCALAR Parameters::get_zcrust_coef ( ) const
```

Gives the value of the thickness of the crust.

Returns

The value of zcrust Parameters::zcrust_coef.

Definition at line 2559 of file parameters.cpp.

get_zcrust_init()

```
int Parameters::get_zcrust_init ( ) const
```

Gives the value characterizing the spatialization (or not) of the thickness of the crust.

Returns

The value corresponding to the initialization of zcrust Parameters::zcrust_init.

Definition at line 2549 of file parameters.cpp.

get_zcrustNameFile()

```
string Parameters::get_zcrustNameFile (
            void  ) const
```

Gives the full name of the file containing the thickness of the crust.

Returns

The zcrust path for the initialization + Input directory Parameters::zcrust_namefile.

Definition at line 2569 of file parameters.cpp.

get_zcrustNameFileS()

```
string Parameters::get_zcrustNameFileS ( ) const
```

Gives the name of the file containing the thickness of the crust.

Returns

The zcrust namefile for the initialization (inside the Input directory) Parameters::zcrust_NF.

Definition at line 2579 of file parameters.cpp.

is_coord_in_file_valid()

```
bool Parameters::is_coord_in_file_valid (
            const SCALAR & x,
            const SCALAR & y,
            int line_number,
            string namefile ) const
```

Verifies if the coordinates x and y exist in the mesh

Fills an array with the values given in the file

Parameters

in	*x*	coordinate of the specific point.
in	*y*	coordinate of the specific point.
in	*line_number*	the line number where the error appears.
in	*namefile*	name of the file containing the values to be verified.

Warning

***: ERROR: .

***: ERROR: x / y must be positive / lower than ***.

***: ERROR: line ***.

Note

-1 is used to not display the line number in the error message .

if the coordinates are not valid, the code will return with false.

Definition at line 2780 of file parameters.cpp.

setparameters()

```
void Parameters::setparameters (
            const char * FILENAME )
```

Sets the parameters.

Gets all the parameters from the file FILENAME, check and affect them. The values used by FullSWOF_2D are saved in the file parameters.dat. These values are also printed in the terminal when the code is run.

Parameters

in	*FILENAME*	name of the paramters file.

Warning

parameters.txt: ERROR: ***.

parameters.txt: WARNING: ***.

ERROR: the *** file *** does not exist in the directory Inputs.

Impossible to open the *** file. Verify if the directory *** exists.

Note

If a value cannot be affected correctly, the code will exit with failure termination code.
If parameters.dat cannot be opened, the code will exit with failure termination code.

Definition at line 63 of file parameters.cpp.

verif_file_bc_inhomogeneous()

```
map< double, string > Parameters::verif_file_bc_inhomogeneous (
            string Bc_NF,
            string path_input_directory,
            char BC,
            map< int, int > & vchoice,
            map< int, SCALAR > & imp_q,
            map< int, SCALAR > & imp_h )
```

Returns the list of files and time values corresponding to the boundary condition. Moreover, fills the containers that will contain the choice of boundary conditions, the values of discharge and water heights imposed for each point.

Extracts from the Bc_NF file the list of files and time values corresponding to the boundary condition.

Parameters

in	*Bc_NF*	name of the file containing the values to be verified.
in	*path_input_directory*	path of directory containing Bc_NF file.
in	*BC*	represents the boundary condition which we handle
out	*vchoice*	container containing the choice of boundary conditions at time equal to 0s.
out	*imp_q*	container containing discharges [m3/s] at time equal to 0s.
out	*imp_h*	container containing water heights [m] at time equal to 0s.

Warning

***: ERROR: cannot open the file.

***: ERROR: line ***.

***: ERROR: the first time must be t = 0.

***: ERROR: it is necessary to specify a file for this choice of initialization of boundary condition.

***: ERROR: the file does not exist in the directory Inputs.

Returns

the list of files and time values corresponding to the boundary condition.

Note

If the containers cannot be filled correctly, the code will exit with failure termination code.

Definition at line 3097 of file parameters.cpp.

4.54.4 Member Data Documentation

amortENO

`SCALAR Parameters::amortENO [protected]`

Parameter for eno.

Definition at line 463 of file parameters.hpp.

Bbound

`map<int,int> Parameters::Bbound [protected]`

Bottom boundary condition.

Definition at line 411 of file parameters.hpp.

Bbound_type

`int Parameters::Bbound_type [protected]`

Type (constant or file) of bottom boundary condition

Definition at line 419 of file parameters.hpp.

bottom_imp_discharge

`map<int,SCALAR> Parameters::bottom_imp_discharge [protected]`

Imposed discharge on the bottom boundary.

Definition at line 413 of file parameters.hpp.

bottom_imp_h

`map<int,SCALAR> Parameters::bottom_imp_h [protected]`

Imposed water height on the bottom boundary.

Definition at line 415 of file parameters.hpp.

bottom_times_files

`map<SCALAR,string> Parameters::bottom_times_files [protected]`

List of files and time values corresponding to the bottom boundary condition.

Definition at line 417 of file parameters.hpp.

cfl_fix

`SCALAR Parameters::cfl_fix [protected]`

Value of the fixed cfl.

Definition at line 369 of file parameters.hpp.

Choice_dt_specific_points

`int Parameters::Choice_dt_specific_points [protected]`

Choice between saving specific points at each time or only for a given time step.

Definition at line 539 of file parameters.hpp.

Choice_points

`int Parameters::Choice_points [protected]`

Choice of the number of specific points to be saved.
Definition at line 527 of file parameters.hpp.

dt_fix

`SCALAR Parameters::dt_fix [protected]`

Value of the fixed time step.
Definition at line 371 of file parameters.hpp.

dt_specific_points

`SCALAR Parameters::dt_specific_points [protected]`

Time step to save the list of the specific points.
Definition at line 541 of file parameters.hpp.

dtheta_coef

`SCALAR Parameters::dtheta_coef [protected]`

Value of dtheta.
Definition at line 475 of file parameters.hpp.

dtheta_init

`int Parameters::dtheta_init [protected]`

Type of initialization of dtheta.
Definition at line 453 of file parameters.hpp.

dtheta_namefile

`string Parameters::dtheta_namefile [protected]`

Name of the file for dtheta: Inputs/file.
Definition at line 507 of file parameters.hpp.

dtheta_NF

`string Parameters::dtheta_NF [protected]`

Name of the file for dtheta without 'Inputs'.
Definition at line 509 of file parameters.hpp.

dx

`SCALAR Parameters::dx [protected]`

Space step in the first (x) direction.
Definition at line 383 of file parameters.hpp.

dy

`SCALAR Parameters::dy [protected]`

Space step in the second (y) direction.

Definition at line 385 of file parameters.hpp.

flux

`int Parameters::flux [protected]`

Numerical flux.

Definition at line 431 of file parameters.hpp.

fric

`int Parameters::fric [protected]`

Friction.

Definition at line 437 of file parameters.hpp.

fric_init

`int Parameters::fric_init [protected]`

Type of friction coefficient.

Definition at line 467 of file parameters.hpp.

fric_namefile

`string Parameters::fric_namefile [protected]`

Name of the file for the friction coefficient: Inputs/file.

Definition at line 491 of file parameters.hpp.

fric_NF

`string Parameters::fric_NF [protected]`

Name of the file for the friction coefficient without 'Inputs'.

Definition at line 493 of file parameters.hpp.

friccoef

`SCALAR Parameters::friccoef [protected]`

Friction coefficient.

Definition at line 469 of file parameters.hpp.

huv_init

`int Parameters::huv_init [protected]`

Type of initial conditions for h and u,v.

Definition at line 445 of file parameters.hpp.

huv_namefile

`string Parameters::huv_namefile [protected]`

Name of the file for the initialization of h and u,v: Inputs/file.

Definition at line 487 of file parameters.hpp.

huv_NF

`string Parameters::huv_NF [protected]`

Name of the file for the initialization of h and u,v without 'Inputs'.

Definition at line 489 of file parameters.hpp.

imax_coef

`SCALAR Parameters::imax_coef [protected]`

Value of imax.

Definition at line 481 of file parameters.hpp.

imax_init

`int Parameters::imax_init [protected]`

Type of initialization of imax.

Definition at line 459 of file parameters.hpp.

imax_namefile

`string Parameters::imax_namefile [protected]`

Name of the file for imax: Inputs/file.

Definition at line 519 of file parameters.hpp.

imax_NF

`string Parameters::imax_NF [protected]`

Name of the file for imax without 'Inputs'.

Definition at line 521 of file parameters.hpp.

inf

`int Parameters::inf [protected]`

Type of infiltration.

Definition at line 441 of file parameters.hpp.

Kc_coef

`SCALAR Parameters::Kc_coef [protected]`

Value of Kc.

Definition at line 471 of file parameters.hpp.

Kc_init

`int Parameters::Kc_init [protected]`

Type of initialization of Kc.
Definition at line 449 of file parameters.hpp.

Kc_namefile

`string Parameters::Kc_namefile [protected]`

Name of the file for Kc: Inputs/file.
Definition at line 499 of file parameters.hpp.

Kc_NF

`string Parameters::Kc_NF [protected]`

Name of the file for Kc without 'Inputs'.
Definition at line 501 of file parameters.hpp.

Ks_coef

`SCALAR Parameters::Ks_coef [protected]`

Value of Ks.
Definition at line 473 of file parameters.hpp.

Ks_init

`int Parameters::Ks_init [protected]`

Type of initialization of Ks.
Definition at line 451 of file parameters.hpp.

Ks_namefile

`string Parameters::Ks_namefile [protected]`

Name of the file for Ks: Inputs/file.
Definition at line 503 of file parameters.hpp.

Ks_NF

`string Parameters::Ks_NF [protected]`

Name of the file for Ks without 'Inputs'.
Definition at line 505 of file parameters.hpp.

L

`SCALAR Parameters::L [protected]`

Length of the domain in the first (x) direction.
Definition at line 387 of file parameters.hpp.

l

`SCALAR Parameters::l [protected]`

Length of the domain in the second (y) direction.

Definition at line 389 of file parameters.hpp.

Lbound

`map<int,int> Parameters::Lbound [protected]`

Left boundary condition.

Definition at line 391 of file parameters.hpp.

Lbound_type

`int Parameters::Lbound_type [protected]`

Type (constant or file) of left boundary condition

Definition at line 399 of file parameters.hpp.

left_imp_discharge

`map<int,SCALAR> Parameters::left_imp_discharge [protected]`

Imposed discharge on the left boundary.

Definition at line 393 of file parameters.hpp.

left_imp_h

`map<int,SCALAR> Parameters::left_imp_h [protected]`

Imposed water height on the left boundary.

Definition at line 395 of file parameters.hpp.

left_times_files

`map<SCALAR,string> Parameters::left_times_files [protected]`

List of files and time values corresponding to the left boundary condition.

Definition at line 397 of file parameters.hpp.

lim

`int Parameters::lim [protected]`

Slope limiter.

Definition at line 439 of file parameters.hpp.

list_point_namefile

`string Parameters::list_point_namefile [protected]`

Name of file containing the list of the specific points without 'Inputs'.

Definition at line 535 of file parameters.hpp.

list_point_NF

`string Parameters::list_point_NF [protected]`

Name of file containing the list of the specific points.

Definition at line 533 of file parameters.hpp.

list_points_x

`VECT Parameters::list_points_x [protected]`

The list of the specific points.

Definition at line 537 of file parameters.hpp.

list_points_y

`VECT Parameters::list_points_y [protected]`

Definition at line 537 of file parameters.hpp.

modifENO

`SCALAR Parameters::modifENO [protected]`

Parameter for eno_modif.

Definition at line 465 of file parameters.hpp.

nbtimes

`int Parameters::nbtimes [protected]`

Number of times saved.

Definition at line 379 of file parameters.hpp.

Nxcell

`int Parameters::Nxcell [protected]`

Number of cells in space in the first (x) direction.

Definition at line 375 of file parameters.hpp.

Nycell

`int Parameters::Nycell [protected]`

Number of cells in space in the second (y) direction.

Definition at line 377 of file parameters.hpp.

order

`int Parameters::order [protected]`

Order of the numerical scheme.

Definition at line 433 of file parameters.hpp.

output_directory

```
string Parameters::output_directory [protected]
```

Name of the output directory Outputs+suffix.
Definition at line 523 of file parameters.hpp.

output_format

```
int Parameters::output_format [protected]
```

Type of output.
Definition at line 461 of file parameters.hpp.

path_input_directory

```
string Parameters::path_input_directory [protected]
```

Name of the input directory
Definition at line 543 of file parameters.hpp.

Psi_coef

```
SCALAR Parameters::Psi_coef [protected]
```

Value of Psi.
Definition at line 477 of file parameters.hpp.

Psi_init

```
int Parameters::Psi_init [protected]
```

Type of initialization of Psi.
Definition at line 455 of file parameters.hpp.

Psi_namefile

```
string Parameters::Psi_namefile [protected]
```

Name of the file for Psi: Inputs/file.
Definition at line 511 of file parameters.hpp.

Psi_NF

```
string Parameters::Psi_NF [protected]
```

Name of the file for Psi without 'Inputs'.
Definition at line 513 of file parameters.hpp.

rain

```
int Parameters::rain [protected]
```

Type of rain.
Definition at line 447 of file parameters.hpp.

rain_namefile

`string Parameters::rain_namefile [protected]`

Name of the file for the rain: Inputs/file.

Definition at line 495 of file parameters.hpp.

rain_NF

`string Parameters::rain_NF [protected]`

Name of the file for the rain without 'Inputs'.

Definition at line 497 of file parameters.hpp.

Rbound

`map<int,int> Parameters::Rbound [protected]`

Right boundary condition.

Definition at line 401 of file parameters.hpp.

Rbound_type

`int Parameters::Rbound_type [protected]`

Type (constant or file) of right boundary condition

Definition at line 409 of file parameters.hpp.

rec

`int Parameters::rec [protected]`

Reconstruction.

Definition at line 435 of file parameters.hpp.

right_imp_discharge

`map<int,SCALAR> Parameters::right_imp_discharge [protected]`

Imposed discharge on the right boundary.

Definition at line 403 of file parameters.hpp.

right_imp_h

`map<int,SCALAR> Parameters::right_imp_h [protected]`

Imposed water height on the right boundary.

Definition at line 405 of file parameters.hpp.

right_times_files

`map<SCALAR,string> Parameters::right_times_files [protected]`

List of files and time values corresponding to the right boundary condition.

Definition at line 407 of file parameters.hpp.

scheme_type

```
int Parameters::scheme_type [protected]
```

Type of scheme (fixed cfl or time step).
Definition at line 373 of file parameters.hpp.

suffix_outputs

```
string Parameters::suffix_outputs [protected]
```

Suffix for the output directory.
Definition at line 525 of file parameters.hpp.

T

```
SCALAR Parameters::T [protected]
```

Final time.
Definition at line 381 of file parameters.hpp.

Tbound

```
map<int,int> Parameters::Tbound [protected]
```

Top boundary condition.
Definition at line 421 of file parameters.hpp.

Tbound_type

```
int Parameters::Tbound_type [protected]
```

Type (constant or file) of top boundary condition
Definition at line 429 of file parameters.hpp.

top_imp_discharge

```
map<int,SCALAR> Parameters::top_imp_discharge [protected]
```

Imposed discharge on the top boundary.
Definition at line 423 of file parameters.hpp.

top_imp_h

```
map<int,SCALAR> Parameters::top_imp_h [protected]
```

Imposed water height on the top boundary.
Definition at line 425 of file parameters.hpp.

top_times_files

```
map<SCALAR,string> Parameters::top_times_files [protected]
```

List of files and time values corresponding to the top boundary condition.
Definition at line 427 of file parameters.hpp.

topo

```
int Parameters::topo  [protected]
```

Type of topography.
Definition at line 443 of file parameters.hpp.

topo_NF

```
string Parameters::topo_NF  [protected]
```

Name of the file for the topography without 'Inputs'.
Definition at line 485 of file parameters.hpp.

topography_namefile

```
string Parameters::topography_namefile  [protected]
```

Name of the file for the topography: Inputs/file.
Definition at line 483 of file parameters.hpp.

x_coord

```
SCALAR Parameters::x_coord  [protected]
```

x coordinate of the specific point to be saved.
Definition at line 529 of file parameters.hpp.

y_coord

```
SCALAR Parameters::y_coord  [protected]
```

y coordinate of the specific point to be saved.
Definition at line 531 of file parameters.hpp.

zcrust_coef

```
SCALAR Parameters::zcrust_coef  [protected]
```

Value of zcrust.
Definition at line 479 of file parameters.hpp.

zcrust_init

```
int Parameters::zcrust_init  [protected]
```

Type of initialization of zcrust.
Definition at line 457 of file parameters.hpp.

zcrust_namefile

```
string Parameters::zcrust_namefile  [protected]
```

Name of the file for zcrust: Inputs/file.
Definition at line 515 of file parameters.hpp.

zcrust_NF

```
string Parameters::zcrust_NF  [protected]
```

Name of the file for zcrust without 'Inputs'.

Definition at line 517 of file parameters.hpp.

The documentation for this class was generated from the following files:

- Headers/libparameters/parameters.hpp
- Sources/libparameters/parameters.cpp

4.55 Parser Class Reference

Parser to read the entries

```
#include <parser.hpp>
```

Public Member Functions

- Parser (const char *)

  Constructor.
- string getValue (const char *)

  Returns the value of the variable.
- virtual ~Parser ()

  Destructor.

4.55.1 Detailed Description

Parser to read the entries

Class that reads the input file writen as description <variable>:: value # comment and keep the values after the "::" ignoring the comments that begin with a "#".

Definition at line 80 of file parser.hpp.

4.55.2 Constructor & Destructor Documentation

Parser()

```
Parser::Parser (
            const char * FILENAME )
```

Constructor.

Constructor: reads the input parameter and copy the data in a tabular.

Parameters

in	*FILENAME*	name of the paramters file.

Warning

Impossible to open the *** file.

Note

If the parameters file cannot be opened, the code will exit with failure termination code.

Definition at line 57 of file parser.cpp.

~Parser()

```
Parser::~Parser ( )  [virtual]
```

Destructor.
Definition at line 161 of file parser.cpp.

4.55.3 Member Function Documentation

getValue()

```
string Parser::getValue (
            const char * TAG )
```

Returns the value of the variable.
Return the value corresponding to the tag.

Parameters

in	*TAG*	name of the variable with delimiters.

Warning

No entry for the variable ***.

Bad syntax for ***. The syntax is: description <variable>:: value

Returns

Value of the variable as a string

Note

If the value cannot be read correctly, the code will exit with failure termination code.

Definition at line 113 of file parser.cpp.
The documentation for this class was generated from the following files:

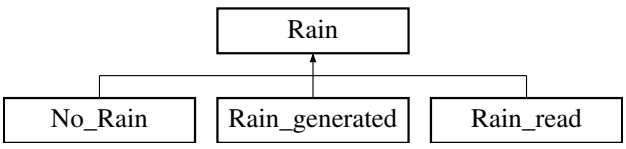
- Headers/libparser/parser.hpp
- Sources/libparser/parser.cpp

4.56 Rain Class Reference

Initialization of the rain.

```
#include <rain.hpp>
```

Inheritance diagram for Rain:



Public Member Functions

- Rain (Parameters &)

  Constructor.
- virtual void rain_func (SCALAR, TAB &)=0

  Function to be specified in each case.
- virtual ∼Rain ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.56.1 Detailed Description

Initialization of the rain.

Class that contains all the common declarations for the initialization of the rain.

Definition at line 70 of file rain.hpp.

4.56.2 Constructor & Destructor Documentation

Rain()

```
Rain::Rain (
            Parameters & par )
```

Constructor.

Defines the number of cells and the space steps.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 58 of file rain.cpp.

∼Rain()

```
Rain::∼Rain ( )  [virtual]
```

Destructor.

Definition at line 68 of file rain.cpp.

4.56.3 Member Function Documentation

rain_func()

```
virtual void Rain::rain_func (
            SCALAR ,
            TAB &  )  [pure virtual]
```

Function to be specified in each case.
Implemented in No_Rain, Rain_generated, and Rain_read.

4.56.4 Member Data Documentation

DX

`const SCALAR Rain::DX [protected]`
Space step in the first (x) direction (unused).
Definition at line 89 of file rain.hpp.

DY

`const SCALAR Rain::DY [protected]`
Space step in the second (y) direction (unused).
Definition at line 91 of file rain.hpp.

NXCELL

`const int Rain::NXCELL [protected]`
Number of cells in space in the first (x) direction.
Definition at line 85 of file rain.hpp.

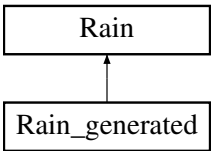
NYCELL

`const int Rain::NYCELL [protected]`
Number of cells in space in the second (y) direction.
Definition at line 87 of file rain.hpp.
The documentation for this class was generated from the following files:

- Headers/librain_infiltration/rain.hpp
- Sources/librain_infiltration/rain.cpp

4.57 Rain_generated Class Reference

Constant rain configuration.
`#include <rain_generated.hpp>`
Inheritance diagram for Rain_generated:



Public Member Functions

- Rain_generated (Parameters &)
  Constructor.
- void rain_func (SCALAR, TAB &)
  Performs the constant initialization.
- virtual ~Rain_generated ()
  Destructor.

Public Member Functions inherited from Rain

- Rain (Parameters &)
  Constructor.
- virtual void rain_func (SCALAR, TAB &)=0
  Function to be specified in each case.
- virtual ~Rain ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Rain

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.57.1 Detailed Description

Constant rain configuration.
Class that initializes a constant rain, with value 0.00001 m/s = 36 mm/h.
Definition at line 73 of file rain_generated.hpp.

4.57.2 Constructor & Destructor Documentation

Rain_generated()

```
Rain_generated::Rain_generated (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 59 of file rain_generated.cpp.

~Rain_generated()

```
Rain_generated::~Rain_generated ( )  [virtual]
```

Destructor.

Definition at line 85 of file rain_generated.cpp.

4.57.3 Member Function Documentation

rain_func()

```
void Rain_generated::rain_func (
            SCALAR time,
            TAB & Tab_rain ) [virtual]
```

Performs the constant initialization.
Initializes the rain to 0.00001 m/s = 36 mm/h.

Parameters

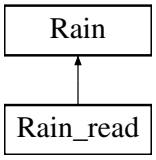
in	*time*	the current time (unused).
in,out	*Tab_rain*	rain intensity at the current time on each cell.

Implements Rain.
Definition at line 67 of file rain_generated.cpp.
The documentation for this class was generated from the following files:

- Headers/librain_infiltration/rain_generated.hpp
- Sources/librain_infiltration/rain_generated.cpp

4.58 Rain_read Class Reference

File configuration.
`#include <rain_read.hpp>`
Inheritance diagram for Rain_read:



Public Member Functions

- Rain_read (Parameters &)
  Constructor.
- void rain_func (SCALAR, TAB &)
  Performs the initialization.
- virtual ~Rain_read ()
  Destructor.

Public Member Functions inherited from Rain

- Rain (Parameters &)
  Constructor.
- virtual void rain_func (SCALAR, TAB &)=0
  Function to be specified in each case.

- virtual ∼Rain ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Rain

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.58.1 Detailed Description

File configuration.

Class that initializes the rain to the values read in a file.

Definition at line 71 of file rain_read.hpp.

4.58.2 Constructor & Destructor Documentation

Rain_read()

```
Rain_read::Rain_read (
            Parameters & par )
```

Constructor.

Defines the name of the file for the initialization and creates two tables (times and intensity) from the data read in the file.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

(rain_namefile): ERROR: cannot open the rain file.

(rain_namefile): ERROR: line ***.

(rain_namefile): ERROR: the first time must be t = 0.

Definition at line 59 of file rain_read.cpp.

∼Rain_read()

```
Rain_read::~Rain_read ( )  [virtual]
```

Destructor.

Definition at line 151 of file rain_read.cpp.

4.58.3 Member Function Documentation

rain_func()

```
void Rain_read::rain_func (
            SCALAR time,
            TAB & Tab_rain ) [virtual]
```

Performs the initialization.
Initializes the rain to the values read in the corresponding file.

Parameters

in	*time*	current time.
in,out	*Tab_rain*	rain intensity at the current time on each cell.

Note

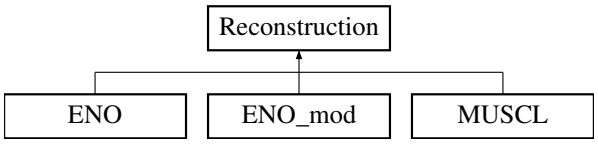
As the times read in the file must start with t = 0, Tab_rain is initialized.

Implements Rain.
Definition at line 131 of file rain_read.cpp.
The documentation for this class was generated from the following files:

- Headers/librain_infiltration/rain_read.hpp
- Sources/librain_infiltration/rain_read.cpp

4.59 Reconstruction Class Reference

Reconstruction of the variables
`#include <reconstruction.hpp>`
Inheritance diagram for Reconstruction:



Public Member Functions

- Reconstruction (Parameters &, TAB &)

  Constructor.
- virtual void calcul (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &)=0

  Function to be specified in each reconstruction.
- virtual ~Reconstruction ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- TAB z1r
- TAB z1l
- TAB z2r

- TAB z2l
- TAB delta_z1
- TAB delta_z2
- Choice_limiter * limiter

4.59.1 Detailed Description

Reconstruction of the variables

Class that contains all the common declarations for the second order reconstruction in space.

Definition at line 74 of file reconstruction.hpp.

4.59.2 Constructor & Destructor Documentation

Reconstruction()

```
Reconstruction::Reconstruction (
            Parameters & par,
            TAB & z )
```

Constructor.

Defines the number of cells, the slope limiter, and initializes Reconstruction::z1l, Reconstruction::z2l, Reconstruction::z1r, Reconstruction::z2r, Reconstruction::delta_z1, Reconstruction::delta_z2.

Parameters

in	*par*	parameter, contains all the values from the parameters file.
in	*z*	topography.

Definition at line 59 of file reconstruction.cpp.

~Reconstruction()

```
Reconstruction::~Reconstruction ( )  [virtual]
```

Destructor.

Definition at line 109 of file reconstruction.cpp.

4.59.3 Member Function Documentation

calcul()

```
virtual void Reconstruction::calcul (
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
```

```
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB & ,
            TAB &  ) [pure virtual]
```

Function to be specified in each reconstruction.

Implemented in ENO, ENO_mod, and MUSCL.

4.59.4 Member Data Documentation

delta_z1

```
TAB Reconstruction::delta_z1 [protected]
```

Difference between the values of the topography on two adjacent cells (on the right) in the first direction

Definition at line 101 of file reconstruction.hpp.

delta_z2

```
TAB Reconstruction::delta_z2 [protected]
```

Difference between the values of the topography on two adjacent cells (on the right) in the second direction

Definition at line 103 of file reconstruction.hpp.

limiter

```
Choice_limiter* Reconstruction::limiter [protected]
```

Slope limiter

Definition at line 105 of file reconstruction.hpp.

NXCELL

```
const int Reconstruction::NXCELL [protected]
```

Number of cells in space in the first (x) direction.

Definition at line 89 of file reconstruction.hpp.

NYCELL

```
const int Reconstruction::NYCELL [protected]
```

Number of cells in space in the second (y) direction.

Definition at line 91 of file reconstruction.hpp.

z1l

`TAB Reconstruction::z1l [protected]`

Reconstructed topography on the left boundary in the first direction

Definition at line 95 of file reconstruction.hpp.

z1r

`TAB Reconstruction::z1r [protected]`

Reconstructed topography on the right boundary in the first direction

Definition at line 93 of file reconstruction.hpp.

z2l

`TAB Reconstruction::z2l [protected]`

Reconstructed topography on the left boundary in the second direction

Definition at line 99 of file reconstruction.hpp.

z2r

`TAB Reconstruction::z2r [protected]`

Reconstructed topography on the right boundary in the second direction

Definition at line 97 of file reconstruction.hpp.

The documentation for this class was generated from the following files:

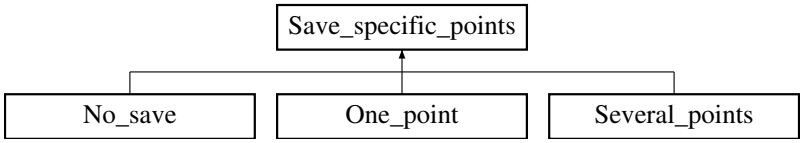
- Headers/libreconstructions/reconstruction.hpp
- Sources/libreconstructions/reconstruction.cpp

4.60 Save_specific_points Class Reference

Specific points to save.

`#include <save_specific_points.hpp>`

Inheritance diagram for Save_specific_points:



Public Member Functions

- Save_specific_points (Parameters &)

  Constructor.

- virtual void save (const TAB &, const TAB &, const TAB &, const SCALAR)=0

  Function which writes in a file the values at the specific points.

- virtual ~Save_specific_points ()

  Destructor.

Protected Attributes

- const SCALAR DX
- const SCALAR DY
- int row
- int column
- SCALAR T_output
- const SCALAR DT_SPECIFIC_POINTS

4.60.1 Detailed Description

Specific points to save.

Class that contains all the common declarations for the saving of specific points.

Definition at line 70 of file save_specific_points.hpp.

4.60.2 Constructor & Destructor Documentation

Save_specific_points()

```
Save_specific_points::Save_specific_points (
            Parameters & par )
```

Constructor.

Virtual class that defines the common parts for saving one or several specific points.

Definition at line 59 of file save_specific_points.cpp.

~Save_specific_points()

```
Save_specific_points::~Save_specific_points ( )  [virtual]
```

Destructor.

Definition at line 73 of file save_specific_points.cpp.

4.60.3 Member Function Documentation

save()

```
virtual void Save_specific_points::save (
            const TAB & ,
            const TAB & ,
            const TAB & ,
            const SCALAR )  [pure virtual]
```

Function which writes in a file the values at the specific points.

Implemented in No_save, One_point, and Several_points.

4.60.4 Member Data Documentation

column

`int Save_specific_points::column [protected]`

Second index of the array from the space variable.

Definition at line 91 of file save_specific_points.hpp.

DT_SPECIFIC_POINTS

`const SCALAR Save_specific_points::DT_SPECIFIC_POINTS [protected]`

Time step to save the list of the specific points.

Definition at line 95 of file save_specific_points.hpp.

DX

`const SCALAR Save_specific_points::DX [protected]`

Space step in the first (x) direction.

Definition at line 85 of file save_specific_points.hpp.

DY

`const SCALAR Save_specific_points::DY [protected]`

Space step in the second (y) direction.

Definition at line 87 of file save_specific_points.hpp.

row

`int Save_specific_points::row [protected]`

First index of the array from the space variable.

Definition at line 89 of file save_specific_points.hpp.

T_output

`SCALAR Save_specific_points::T_output [protected]`

Value of the fixed time step.

Definition at line 93 of file save_specific_points.hpp.

The documentation for this class was generated from the following files:

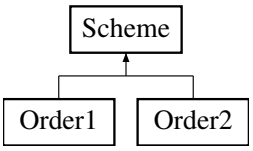
- Headers/libsave/save_specific_points.hpp
- Sources/libsave/save_specific_points.cpp

4.61 Scheme Class Reference

Numerical scheme.

`#include <scheme.hpp>`

Inheritance diagram for Scheme:



Public Member Functions

- Scheme (Parameters &)

  Constructor.
- virtual void calcul ()=0

  Function to be specified in each numerical scheme.
- void allocation ()

  Allocation of spatialized variables.
- void deallocation ()

  Deallocation of variables.
- void maincalcflux (SCALAR, SCALAR, SCALAR, SCALAR, SCALAR, SCALAR &)

  Main calculation of the flux.
- void maincalcscheme (TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, TAB &, SCALAR, SCALAR, int)

  Main calculation of the scheme.
- void boundary (TAB &, TAB &, TAB &, SCALAR, const int, const int)

  Calls the boundary conditions and affects the boundary values.
- SCALAR froude_number (TAB, TAB, TAB)

  Returns the Froude number.
- virtual ∼Scheme ()

  Destructor.

Protected Attributes

- const int NXCELL
- const int NYCELL
- const int ORDER
- const SCALAR T
- const int NBTIMES
- const int SCHEME_TYPE
- const SCALAR DX
- const SCALAR DY
- const SCALAR CFL_FIX
- SCALAR DT_FIX
- SCALAR tx
- SCALAR ty
- SCALAR T_output
- SCALAR dt_output
- const SCALAR FRICCOEF
- map< int, SCALAR > & L_IMP_Q
- map< int, SCALAR > & L_IMP_H
- map< SCALAR, string > left_times_files
- map< SCALAR, string >::const_iterator p_left_times_files

- map< int, int > L_choice_bound
- int Lbound_type
- bool is_Lbound_changed
- map< int, SCALAR > & R_IMP_Q
- map< int, SCALAR > & R_IMP_H
- map< SCALAR, string > right_times_files
- map< SCALAR, string >::const_iterator p_right_times_files
- map< int, int > R_choice_bound
- int Rbound_type
- bool is_Rbound_changed
- map< int, SCALAR > & B_IMP_Q
- map< int, SCALAR > & B_IMP_H
- map< SCALAR, string > bottom_times_files
- map< SCALAR, string >::const_iterator p_bottom_times_files
- map< int, int > B_choice_bound
- int Bbound_type
- bool is_Bbound_changed
- map< int, SCALAR > & T_IMP_Q
- map< int, SCALAR > & T_IMP_H
- map< SCALAR, string > top_times_files
- map< SCALAR, string >::const_iterator p_top_times_files
- map< int, int > T_choice_bound
- int Tbound_type
- bool is_Tbound_changed
- Parameters & fs2d_par
- TAB z
- TAB h
- TAB u
- TAB v
- TAB q1
- TAB q2
- TAB hs
- TAB us
- TAB vs
- TAB qs1
- TAB qs2
- TAB f1
- TAB f2
- TAB f3
- TAB g1
- TAB g2
- TAB g3
- TAB h1left
- TAB h1right
- TAB h2left
- TAB h2right
- TAB delz1
- TAB delz2
- TAB delzc1

- TAB delzc2
- TAB h1r
- TAB u1r
- TAB v1r
- TAB h1l
- TAB u1l
- TAB v1l
- TAB h2r
- TAB u2r
- TAB v2r
- TAB h2l
- TAB u2l
- TAB v2l
- TAB Rain
- TAB Vin_tot
- time_t start
- time_t end
- SCALAR timecomputation
- clock_t cpu_time
- int n
- SCALAR Fr
- SCALAR dt1
- SCALAR dt_max
- SCALAR cur_time
- SCALAR dt_first
- SCALAR Volrain_Tot
- SCALAR Total_volume_outflow
- SCALAR height_of_tot
- SCALAR height_Vinf_tot
- SCALAR Vol_inf_tot_cumul
- SCALAR Vol_of_tot
- Choice_condition * Lbound
- Choice_condition * Rbound
- Choice_condition * Bbound
- Choice_condition * Tbound
- Choice_output * out
- int verif
- Choice_save_specific_points * out_specific_points

4.61.1 Detailed Description

Numerical scheme.

Class that contains all the common declarations for the numerical schemes.

Definition at line 116 of file scheme.hpp.

4.61.2 Constructor & Destructor Documentation

Scheme()

```
Scheme::Scheme (
            Parameters & par )
```

Constructor.

Initializations and allocations.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 61 of file scheme.cpp.

~Scheme()

```
Scheme::~Scheme ( ) [virtual]
```

Destructor.

Definition at line 860 of file scheme.cpp.

4.61.3 Member Function Documentation

allocation()

```
void Scheme::allocation ( )
```

Allocation of spatialized variables.

Allocation of Scheme::z, Scheme::h, Scheme::u, Scheme::v, Scheme::q1, Scheme::q2, Scheme::Vin_tot, Scheme::hs, Scheme::us, Scheme::vs, Scheme::qs1, Scheme::qs2, Scheme::f1, Scheme::f2, Scheme::f3, Scheme::g1, Scheme::g2, Scheme::g3 Scheme::h1left, Scheme::h1l, Scheme::u1l, Scheme::v1l, Scheme::h1right, Scheme::h1r, Scheme::u1r, Scheme::v1r, Scheme::h2left, Scheme::h2l, Scheme::u2l, Scheme::v2l, Scheme::h2right, Scheme::h2r, Scheme::u2r, Scheme::v2r, Scheme::delz1, Scheme::delz2, Scheme::delzc1, Scheme::delzc2, Scheme::Rain.

Definition at line 585 of file scheme.cpp.

boundary()

```
void Scheme::boundary (
            TAB & h_tmp,
            TAB & u_tmp,
            TAB & v_tmp,
            SCALAR time_tmp,
            const int NODEX,
            const int NODEY )
```

Calls the boundary conditions and affects the boundary values.

Parameters

in,out	*h_tmp*	water height.
in,out	*u_tmp*	first component of the velocity.
in,out	*v_tmp*	second component of the velocity.
in	*time_tmp*	current time.
in	*NODEX*	number of space cells in the first (x) direction.
in	*NODEY*	number of space cells in the second (y) direction.

Definition at line 386 of file scheme.cpp.

calcul()

```
virtual void Scheme::calcul ( ) [pure virtual]
```

Function to be specified in each numerical scheme.

Implemented in Order1, and Order2.

deallocation()

```
void Scheme::deallocation ( )
```

Deallocation of variables.

Deallocation of Scheme::z, Scheme::h, Scheme::u, Scheme::v, Scheme::q1, Scheme::q2, Scheme::Vin_tot, Scheme::hs, Scheme::us, Scheme::vs, Scheme::qs1, Scheme::qs2, Scheme::f1, Scheme::f2, Scheme::f3, Scheme::g1, Scheme::g2, Scheme::g3 Scheme::h1left, Scheme::h1l, Scheme::u1l, Scheme::v1l, Scheme::h1right, Scheme::h1r, Scheme::u1r, Scheme::v1r, Scheme::h2left, Scheme::h2l, Scheme::u2l, Scheme::v2l, Scheme::h2right, Scheme::h2r, Scheme::u2r, Scheme::v2r, Scheme::delz1, Scheme::delz2, Scheme::delzc1, Scheme::delzc2, Scheme::Rain.

Definition at line 723 of file scheme.cpp.

froude_number()

```
SCALAR Scheme::froude_number (
            TAB h_s,
            TAB u_s,
            TAB v_s )
```

Returns the Froude number.

Mean value in space of the Froude number at the final time.

Parameters

in	*h_s*	water height.
in	*u_s*	first component of the velocity.
in	*v_s*	second component of the velocity.

Returns

The mean Froude number $\frac{\sqrt{u_s^2+v_s^2}}{\sqrt{gh_s}}$.

Definition at line 548 of file scheme.cpp.

maincalcflux()

```
void Scheme::maincalcflux (
            SCALAR cflfix,
            SCALAR T,
            SCALAR curtime,
            SCALAR dt_max,
```

```
            SCALAR dt,
            SCALAR & dt_cal )
```

Main calculation of the flux.

First part. Construction of variables for hydrostatic reconstruction. Fluxes in the two directions. Computation of the time step for the fixed cfl. This calculation is called once at the order 1, and twice at the second order.

Parameters

in	*cflfix*	fixed cfl.
in	*T*	final time (unused).
in	*curtime*	current time.
in	*dt_max*	maximum value of the time step.
in	*dt*	time step.
out	*dt_cal*	effective time step.

Warning

the CFL condition is not satisfied: CFL $>$ ***

Definition at line 173 of file scheme.cpp.

maincalcscheme()

```
void Scheme::maincalcscheme (
            TAB & he,
            TAB & ve1,
            TAB & ve2,
            TAB & qe1,
            TAB & qe2,
            TAB & hes,
            TAB & ves1,
            TAB & ves2,
            TAB & qes1,
            TAB & qes2,
            TAB & Vin,
            SCALAR curtime,
            SCALAR dt,
            int n )
```

Main calculation of the scheme.

Second part. Computation of h, u and v. This calculation is called once at the order 1, and twice at the second order.

Parameters

in	*he*	water height.
in	*ve1*	first component of the velocity.
in	*ve2*	second component of the velocity.
in	*qe1*	first component of the discharge (unused).
in	*qe2*	second component of the discharge (unused).
out	*hes*	water height.
out	*ves1*	first component of the velocity.
out	*ves2*	second component of the velocity.

Parameters

out	*qes1*	first component of the discharge.
out	*qes2*	second component of the discharge.
out	*Vin*	infiltrated volume
in	*curtime*	current time.
in	*dt*	time step.
in	*n*	number of iterations (unused).

Note

In DEBUG mode, the programme will save three other files with boundaries fluxes.

Definition at line 253 of file scheme.cpp.

4.61.4 Member Data Documentation

B_choice_bound

```
map<int,int> Scheme::B_choice_bound [protected]
```

Bottom boundary condition.
Definition at line 216 of file scheme.hpp.

B_IMP_H

```
map<int,SCALAR>& Scheme::B_IMP_H [protected]
```

Imposed water height on the bottom boundary.
Definition at line 210 of file scheme.hpp.

B_IMP_Q

```
map<int,SCALAR>& Scheme::B_IMP_Q [protected]
```

Imposed discharge on the bottom boundary.
Definition at line 208 of file scheme.hpp.

Bbound

```
Choice_condition* Scheme::Bbound [protected]
```

The choice of the bottom boundary condition.
Definition at line 352 of file scheme.hpp.

Bbound_type

```
int Scheme::Bbound_type [protected]
```

Type (constant or file) of bottom boundary condition
Definition at line 218 of file scheme.hpp.

bottom_times_files

`map<SCALAR,string> Scheme::bottom_times_files [protected]`

List of files and time values corresponding to the bottom boundary condition.

Definition at line 212 of file scheme.hpp.

CFL_FIX

`const SCALAR Scheme::CFL_FIX [protected]`

Value of the fixed cfl.

Definition at line 166 of file scheme.hpp.

cpu_time

`clock_t Scheme::cpu_time [protected]`

CPU time.

Definition at line 322 of file scheme.hpp.

cur_time

`SCALAR Scheme::cur_time [protected]`

The current simulation time.

Definition at line 332 of file scheme.hpp.

delz1

`TAB Scheme::delz1 [protected]`

Variations of the topography along x.

Definition at line 280 of file scheme.hpp.

delz2

`TAB Scheme::delz2 [protected]`

Variations of the topography along y.

Definition at line 282 of file scheme.hpp.

delzc1

`TAB Scheme::delzc1 [protected]`

Difference between the reconstructed topographies on the left and on the right boundary of a cell along x.

Definition at line 284 of file scheme.hpp.

delzc2

`TAB Scheme::delzc2 [protected]`

Difference between the reconstructed topographies on the left and on the right boundary of a cell along y.

Definition at line 286 of file scheme.hpp.

dt1

`SCALAR Scheme::dt1 [protected]`

Time step in case of fixed cfl.

Definition at line 328 of file scheme.hpp.

dt_first

`SCALAR Scheme::dt_first [protected]`

Space step in the first step in the method Heun.

Definition at line 334 of file scheme.hpp.

DT_FIX

`SCALAR Scheme::DT_FIX [protected]`

Value of the fixed time step.

Definition at line 168 of file scheme.hpp.

dt_max

`SCALAR Scheme::dt_max [protected]`

Maximum time step in case of fixed cfl.

Definition at line 330 of file scheme.hpp.

dt_output

`SCALAR Scheme::dt_output [protected]`

Time step to save the data (evolution file).

Definition at line 176 of file scheme.hpp.

DX

`const SCALAR Scheme::DX [protected]`

Space step in the first (x) direction.

Definition at line 162 of file scheme.hpp.

DY

`const SCALAR Scheme::DY [protected]`

Space step in the second (y) direction.

Definition at line 164 of file scheme.hpp.

end

`time_t Scheme::end [protected]`

End of timer.

Definition at line 318 of file scheme.hpp.

f1

`TAB Scheme::f1 [protected]`

First component of the numerical flux along x.
Definition at line 260 of file scheme.hpp.

f2

`TAB Scheme::f2 [protected]`

Second component of the numerical flux along x.
Definition at line 262 of file scheme.hpp.

f3

`TAB Scheme::f3 [protected]`

Third component of the numerical flux along x.
Definition at line 264 of file scheme.hpp.

Fr

`SCALAR Scheme::Fr [protected]`

Mean Froude number.
Definition at line 326 of file scheme.hpp.

FRICCOEF

`const SCALAR Scheme::FRICCOEF [protected]`

Friction coefficient.
Definition at line 178 of file scheme.hpp.

fs2d_par

`Parameters& Scheme::fs2d_par [protected]`

Parameters object to read the files of boundary conditions
Definition at line 236 of file scheme.hpp.

g1

`TAB Scheme::g1 [protected]`

First component of the numerical flux along y.
Definition at line 266 of file scheme.hpp.

g2

`TAB Scheme::g2 [protected]`

Second component of the numerical flux along y.
Definition at line 268 of file scheme.hpp.

g3

`TAB Scheme::g3 [protected]`

Third component of the numerical flux along y.

Definition at line 270 of file scheme.hpp.

h

`TAB Scheme::h [protected]`

Water height.

Definition at line 240 of file scheme.hpp.

h1l

`TAB Scheme::h1l [protected]`

Water height on the cell located at the left of the boundary along x.

Definition at line 294 of file scheme.hpp.

h1left

`TAB Scheme::h1left [protected]`

Hydrostatic reconstruction on the left along x.

Definition at line 272 of file scheme.hpp.

h1r

`TAB Scheme::h1r [protected]`

Water height on the cell located at the right of the boundary along x.

Definition at line 288 of file scheme.hpp.

h1right

`TAB Scheme::h1right [protected]`

Hydrostatic reconstruction on the right along x.

Definition at line 274 of file scheme.hpp.

h2l

`TAB Scheme::h2l [protected]`

Water height on the cell located at the left of the boundary along y.

Definition at line 306 of file scheme.hpp.

h2left

`TAB Scheme::h2left [protected]`

Hydrostatic reconstruction on the left along y.

Definition at line 276 of file scheme.hpp.

h2r

`TAB Scheme::h2r [protected]`

Water height on the cell located at the right of the boundary along y.
Definition at line 300 of file scheme.hpp.

h2right

`TAB Scheme::h2right [protected]`

Hydrostatic reconstruction on the right along y.
Definition at line 278 of file scheme.hpp.

height_of_tot

`SCALAR Scheme::height_of_tot [protected]`

Cumulative water height on the whole domain
Definition at line 340 of file scheme.hpp.

height_Vinf_tot

`SCALAR Scheme::height_Vinf_tot [protected]`

Cumulative height of infiltrated water on the whole domain
Definition at line 342 of file scheme.hpp.

hs

`TAB Scheme::hs [protected]`

Water height after one step of the scheme.
Definition at line 250 of file scheme.hpp.

is_Bbound_changed

`bool Scheme::is_Bbound_changed [protected]`

This variable is true if the boundary has been updated
Definition at line 220 of file scheme.hpp.

is_Lbound_changed

`bool Scheme::is_Lbound_changed [protected]`

This variable is true if the boundary has been updated
Definition at line 192 of file scheme.hpp.

is_Rbound_changed

`bool Scheme::is_Rbound_changed [protected]`

This variable is true if the boundary has been updated
Definition at line 206 of file scheme.hpp.

is_Tbound_changed

`bool Scheme::is_Tbound_changed [protected]`

This variable is true if the boundary has been updated

Definition at line 234 of file scheme.hpp.

L_choice_bound

`map<int,int> Scheme::L_choice_bound [protected]`

Right boundary condition.

Definition at line 188 of file scheme.hpp.

L_IMP_H

`map<int,SCALAR>& Scheme::L_IMP_H [protected]`

Imposed water height on the left boundary.

Definition at line 182 of file scheme.hpp.

L_IMP_Q

`map<int,SCALAR>& Scheme::L_IMP_Q [protected]`

Imposed discharge on the left boundary.

Definition at line 180 of file scheme.hpp.

Lbound

`Choice_condition* Scheme::Lbound [protected]`

The choice of the left boundary condition.

Definition at line 348 of file scheme.hpp.

Lbound_type

`int Scheme::Lbound_type [protected]`

Type (constant or file) of left boundary condition

Definition at line 190 of file scheme.hpp.

left_times_files

`map<SCALAR,string> Scheme::left_times_files [protected]`

List of files and time values corresponding to the left boundary condition.

Definition at line 184 of file scheme.hpp.

n

`int Scheme::n [protected]`

Iterator for the loop in time.

Definition at line 324 of file scheme.hpp.

NBTIMES

```
const int Scheme::NBTIMES [protected]
```

Number of times saved.

Definition at line 158 of file scheme.hpp.

NXCELL

```
const int Scheme::NXCELL [protected]
```

Number of cells in space in the first (x) direction.

Definition at line 150 of file scheme.hpp.

NYCELL

```
const int Scheme::NYCELL [protected]
```

Number of cells in space in the second (y) direction.

Definition at line 152 of file scheme.hpp.

ORDER

```
const int Scheme::ORDER [protected]
```

Order of the numerical scheme.

Definition at line 154 of file scheme.hpp.

out

```
Choice_output* Scheme::out [protected]
```

The choice of output.

Definition at line 356 of file scheme.hpp.

out_specific_points

```
Choice_save_specific_points* Scheme::out_specific_points [protected]
```

The choice of output for the specific points.

Definition at line 360 of file scheme.hpp.

p_bottom_times_files

```
map<SCALAR,string>::const_iterator Scheme::p_bottom_times_files [protected]
```

Iterator pointing to a file and time value corresponding to the bottom boundary condition

Definition at line 214 of file scheme.hpp.

p_left_times_files

```
map<SCALAR,string>::const_iterator Scheme::p_left_times_files [protected]
```

Iterator pointing to a file and time value corresponding to the left boundary condition

Definition at line 186 of file scheme.hpp.

p_right_times_files

`map<SCALAR,string>::const_iterator Scheme::p_right_times_files [protected]`

Iterator pointing to a file and time value corresponding to the right boundary condition

Definition at line 200 of file scheme.hpp.

p_top_times_files

`map<SCALAR,string>::const_iterator Scheme::p_top_times_files [protected]`

Iterator pointing to a file and time value corresponding to the top boundary condition

Definition at line 228 of file scheme.hpp.

q1

`TAB Scheme::q1 [protected]`

First component of the discharge.

Definition at line 246 of file scheme.hpp.

q2

`TAB Scheme::q2 [protected]`

Second component of the discharge.

Definition at line 248 of file scheme.hpp.

qs1

`TAB Scheme::qs1 [protected]`

First component of the discharge after one step of the scheme.

Definition at line 256 of file scheme.hpp.

qs2

`TAB Scheme::qs2 [protected]`

Second component of the discharge after one step of the scheme.

Definition at line 258 of file scheme.hpp.

R_choice_bound

`map<int,int> Scheme::R_choice_bound [protected]`

Right boundary condition.

Definition at line 202 of file scheme.hpp.

R_IMP_H

`map<int,SCALAR>& Scheme::R_IMP_H [protected]`

Imposed water height on the right boundary.

Definition at line 196 of file scheme.hpp.

R_IMP_Q

`map<int,SCALAR>& Scheme::R_IMP_Q [protected]`

Imposed discharge on the right boundary.

Definition at line 194 of file scheme.hpp.

Rain

`TAB Scheme::Rain [protected]`

Rain intensity.

Definition at line 312 of file scheme.hpp.

Rbound

`Choice_condition* Scheme::Rbound [protected]`

The choice of the right boundary condition.

Definition at line 350 of file scheme.hpp.

Rbound_type

`int Scheme::Rbound_type [protected]`

Type (constant or file) of right boundary condition

Definition at line 204 of file scheme.hpp.

right_times_files

`map<SCALAR,string> Scheme::right_times_files [protected]`

List of files and time values corresponding to the right boundary condition.

Definition at line 198 of file scheme.hpp.

SCHEME_TYPE

`const int Scheme::SCHEME_TYPE [protected]`

Type of scheme (fixed cfl or time step).

Definition at line 160 of file scheme.hpp.

start

`time_t Scheme::start [protected]`

Beginning of timer.

Definition at line 316 of file scheme.hpp.

T

`const SCALAR Scheme::T [protected]`

Final time.

Definition at line 156 of file scheme.hpp.

T_choice_bound

```
map<int,int> Scheme::T_choice_bound [protected]
```

Top boundary condition.
Definition at line 230 of file scheme.hpp.

T_IMP_H

```
map<int,SCALAR>& Scheme::T_IMP_H [protected]
```

Imposed water height on the top boundary.
Definition at line 224 of file scheme.hpp.

T_IMP_Q

```
map<int,SCALAR>& Scheme::T_IMP_Q [protected]
```

Imposed discharge on the top boundary.
Definition at line 222 of file scheme.hpp.

T_output

```
SCALAR Scheme::T_output [protected]
```

Time to save the data (evolution file).
Definition at line 174 of file scheme.hpp.

Tbound

```
Choice_condition* Scheme::Tbound [protected]
```

The choice of the top boundary condition.
Definition at line 354 of file scheme.hpp.

Tbound_type

```
int Scheme::Tbound_type [protected]
```

Type (constant or file) of top boundary condition
Definition at line 232 of file scheme.hpp.

timecomputation

```
SCALAR Scheme::timecomputation [protected]
```

Duration of the computation.
Definition at line 320 of file scheme.hpp.

top_times_files

```
map<SCALAR,string> Scheme::top_times_files [protected]
```

List of files and time values corresponding to the top boundary condition.
Definition at line 226 of file scheme.hpp.

Total_volume_outflow

`SCALAR Scheme::Total_volume_outflow [protected]`

Cumulative outflow volume at the boundary.

Definition at line 338 of file scheme.hpp.

tx

`SCALAR Scheme::tx [protected]`

Ratio dt/dx.

Definition at line 170 of file scheme.hpp.

ty

`SCALAR Scheme::ty [protected]`

Ratio dt/dy.

Definition at line 172 of file scheme.hpp.

u

`TAB Scheme::u [protected]`

First component of the velocity.

Definition at line 242 of file scheme.hpp.

u1l

`TAB Scheme::u1l [protected]`

First component of the velocity on the cell located at the left of the boundary along x.

Definition at line 296 of file scheme.hpp.

u1r

`TAB Scheme::u1r [protected]`

First component of the velocity on the cell located at the right of the boundary along x.

Definition at line 290 of file scheme.hpp.

u2l

`TAB Scheme::u2l [protected]`

First component of the velocity on the cell located at the left of the boundary along y.

Definition at line 308 of file scheme.hpp.

u2r

`TAB Scheme::u2r [protected]`

First component of the velocity on the cell located at the right of the boundary along y.

Definition at line 302 of file scheme.hpp.

us

`TAB Scheme::us [protected]`

First component of the velocity after one step of the scheme.

Definition at line 252 of file scheme.hpp.

v

`TAB Scheme::v [protected]`

Second component of the velocity.

Definition at line 244 of file scheme.hpp.

v1l

`TAB Scheme::v1l [protected]`

Second component of the velocity on the cell located at the left of the boundary along x.

Definition at line 298 of file scheme.hpp.

v1r

`TAB Scheme::v1r [protected]`

Second component of the velocity on the cell located at the right of the boundary along x.

Definition at line 292 of file scheme.hpp.

v2l

`TAB Scheme::v2l [protected]`

Second component of the velocity on the cell located at the left of the boundary along y.

Definition at line 310 of file scheme.hpp.

v2r

`TAB Scheme::v2r [protected]`

Second component of the velocity on the cell located at the right of the boundary along y.

Definition at line 304 of file scheme.hpp.

verif

`int Scheme::verif [protected]`

Flag for the time step.

Definition at line 358 of file scheme.hpp.

Vin_tot

`TAB Scheme::Vin_tot [protected]`

Cumulative volume of infiltrated water (at each point).

Definition at line 314 of file scheme.hpp.

Vol_inf_tot_cumul

`SCALAR Scheme::Vol_inf_tot_cumul [protected]`

Cumulative volume of water infiltrated.

Definition at line 344 of file scheme.hpp.

Vol_of_tot

`SCALAR Scheme::Vol_of_tot [protected]`

Cumulative streammed volume.

Definition at line 346 of file scheme.hpp.

Volrain_Tot

`SCALAR Scheme::Volrain_Tot [protected]`

Cumulative volume of rain on the whole domain.

Definition at line 336 of file scheme.hpp.

vs

`TAB Scheme::vs [protected]`

Second component of the velocity after one step of the scheme.

Definition at line 254 of file scheme.hpp.

z

`TAB Scheme::z [protected]`

Topography.

Definition at line 238 of file scheme.hpp.

The documentation for this class was generated from the following files:

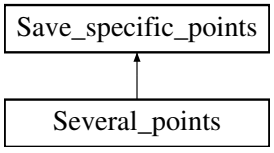
- Headers/libschemes/scheme.hpp
- Sources/libschemes/scheme.cpp

4.62 Several_points Class Reference

Several points output.

`#include <several_points.hpp>`

Inheritance diagram for Several_points:



Public Member Functions

- Several_points (Parameters &)

  Constructor.
- void save (const TAB &, const TAB &, const TAB &, const SCALAR)

  Saves the values at the specific points.
- virtual ~Several_points ()

  Destructor.

Public Member Functions inherited from Save_specific_points

- Save_specific_points (Parameters &)

  Constructor.
- virtual void save (const TAB &, const TAB &, const TAB &, const SCALAR)=0

  Function which writes in a file the values at the specific points.
- virtual ~Save_specific_points ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Save_specific_points

- const SCALAR DX
- const SCALAR DY
- int row
- int column
- SCALAR T_output
- const SCALAR DT_SPECIFIC_POINTS

4.62.1 Detailed Description

Several points output.

Class that writes the result in the output file with a structure optimized for Gnuplot.

Definition at line 73 of file several_points.hpp.

4.62.2 Constructor & Destructor Documentation

Several_points()

```
Several_points::Several_points (
            Parameters & par )
```

Constructor.

Writes the header of the file 'hu_specific_points.dat'.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Warning

***: ERROR: Impossible to open the *** file. Verify if the directory *** exists.

***: WARNING: line *** ; a commentary should begin with the # symbol.

Note

If hu_specific_points.dat cannot be opened, the code will exit with failure termination code.

x coordinate of the specific point to be saved.
y coordinate of the specific point to be saved.
Name of file containing the list of the specific points.
Definition at line 60 of file several_points.cpp.

~Several_points()

```
Several_points::~Several_points ( )  [virtual]
```

Destructor.
Definition at line 188 of file several_points.cpp.

4.62.3 Member Function Documentation

save()

```
void Several_points::save (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const SCALAR time )  [virtual]
```

Saves the values at the specific points.
Writes the values of Scheme::h, Scheme::u (=q1/h), Scheme::v (=q2/h), Scheme::h+ Scheme::z (free surface), Scheme::z, $|U| = \sqrt{u^2 + v^2}$, the Froude number $\frac{|U|}{\sqrt{gh}}$, Scheme::q1, Scheme::q2, and $h|U|$ at the current time in "hu_specific_points.dat".
If the water height is too small, we replace it by 0, the velocity is null and the Froude number does not exist.

Parameters

in	*h*	the water height.
in	*u*	first component of the velocity.
in	*v*	second component of the velocity.
in	*time*	the current time.

Implements Save_specific_points.
Definition at line 141 of file several_points.cpp.
The documentation for this class was generated from the following files:

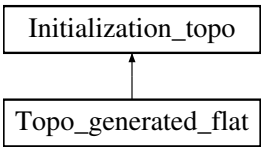
- Headers/libsave/several_points.hpp
- Sources/libsave/several_points.cpp

4.63 Topo_generated_flat Class Reference

Flat configuration.

`#include <topo_generated_flat.hpp>`

Inheritance diagram for Topo_generated_flat:



Public Member Functions

- Topo_generated_flat (Parameters &)

  Constructor.
- void initialization (TAB &)

  Performs the initialization.
- virtual ~Topo_generated_flat ()

  Destructor.

Public Member Functions inherited from Initialization_topo

- Initialization_topo (Parameters &)

  Constructor.
- virtual void initialization (TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_topo ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_topo

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.63.1 Detailed Description

Flat configuration.

Class that initializes a flat topography, with value 0.

Definition at line 73 of file topo_generated_flat.hpp.

4.63.2 Constructor & Destructor Documentation

Topo_generated_flat()

```
Topo_generated_flat::Topo_generated_flat (
            Parameters & par )
```

Constructor.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 60 of file topo_generated_flat.cpp.

~Topo_generated_flat()

```
Topo_generated_flat::~Topo_generated_flat ( )  [virtual]
```

Destructor.
Definition at line 84 of file topo_generated_flat.cpp.

4.63.3 Member Function Documentation

initialization()

```
void Topo_generated_flat::initialization (
            TAB & topo )  [virtual]
```

Performs the initialization.
Initializes the topography to 0.

Parameters

in,out	*topo*	topography.

Implements Initialization_topo.
Definition at line 69 of file topo_generated_flat.cpp.
The documentation for this class was generated from the following files:

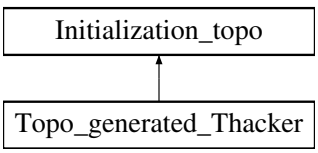
- Headers/libinitializations/topo_generated_flat.hpp
- Sources/libinitializations/topo_generated_flat.cpp

4.64 Topo_generated_Thacker Class Reference

Thacker configuration.

```
#include <topo_generated_thacker.hpp>
```

Inheritance diagram for Topo_generated_Thacker:



Public Member Functions

- Topo_generated_Thacker (Parameters &)

  Constructor.
- void initialization (TAB &)

  Performs the initialization.

- virtual ~Topo_generated_Thacker ()

  Destructor.

Public Member Functions inherited from Initialization_topo

- Initialization_topo (Parameters &)

  Constructor.
- virtual void initialization (TAB &)=0

  Function to be specified in each initialization.
- virtual ~Initialization_topo ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_topo

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.64.1 Detailed Description

Thacker configuration.

Class that initializes a topography for Thacker's benchmark (shape of a paraboloid of revolution).

Definition at line 73 of file topo_generated_thacker.hpp.

4.64.2 Constructor & Destructor Documentation

Topo_generated_Thacker()

```
Topo_generated_Thacker::Topo_generated_Thacker (
            Parameters & par )
```

Constructor.

Defines the parameters of the paraboloid.

Parameters

in	*par*	parameter, contains all the values from the parameters file (unused).

Definition at line 60 of file topo_generated_thacker.cpp.

~Topo_generated_Thacker()

```
Topo_generated_Thacker::~Topo_generated_Thacker ( )  [virtual]
```

Destructor.

Definition at line 98 of file topo_generated_thacker.cpp.

4.64.3 Member Function Documentation

initialization()

```
void Topo_generated_Thacker::initialization (
            TAB & topo ) [virtual]
```

Performs the initialization.
Initializes the topography to $h_0 \left(\frac{(x-Lx/2)^2+(y-Ly/2)^2}{a^2} - 1 \right)$, see Thacker [1981].

Parameters

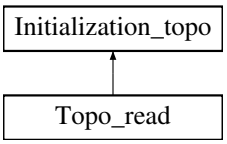
in,out	*topo*	topography.

Implements Initialization_topo.
Definition at line 78 of file topo_generated_thacker.cpp.
The documentation for this class was generated from the following files:

- Headers/libinitializations/topo_generated_thacker.hpp
- Sources/libinitializations/topo_generated_thacker.cpp

4.65 Topo_read Class Reference

File configuration.
`#include <topo_read.hpp>`
Inheritance diagram for Topo_read:



Public Member Functions

- Topo_read (Parameters &)
  Constructor.
- void initialization (TAB &)
  Performs the initialization.
- virtual ~Topo_read ()
  Destructor.

Public Member Functions inherited from Initialization_topo

- Initialization_topo (Parameters &)
  Constructor.
- virtual void initialization (TAB &)=0
  Function to be specified in each initialization.
- virtual ~Initialization_topo ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Initialization_topo

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY

4.65.1 Detailed Description

File configuration.

Class that initializes the topography to the values read in a file.

Definition at line 72 of file topo_read.hpp.

4.65.2 Constructor & Destructor Documentation

Topo_read()

```
Topo_read::Topo_read (
            Parameters & par )
```

Constructor.

Defines the name of the file for the initialization.

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 60 of file topo_read.cpp.

~Topo_read()

```
Topo_read::~Topo_read ( )  [virtual]
```

Destructor.

Definition at line 187 of file topo_read.cpp.

4.65.3 Member Function Documentation

initialization()

```
void Topo_read::initialization (
            TAB & topo )  [virtual]
```

Performs the initialization.

Initializes the topography to the values read in the corresponding file.

Parameters

in,out	*topo*	topography.

Warning

(huv_namefile): ERROR: cannot open the topography file.

(huv_namefile): ERROR: the number of data in this file is too big

(huv_namefile): ERROR: line ***.

(huv_namefile): WARNING: line ***.

(huv_namefile): ERROR: the number of data in this file is too small

(huv_namefile): ERROR: the value for the point x *** y *** is missing

Note

If the file cannot be opened or if the data are not correct, the code will exit with failure termination code.

Implements Initialization_topo.
Definition at line 72 of file topo_read.cpp.
The documentation for this class was generated from the following files:

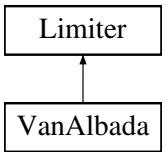
- Headers/libinitializations/topo_read.hpp
- Sources/libinitializations/topo_read.cpp

4.66 VanAlbada Class Reference

Van Albada slope limiter.

`#include <vanalbada.hpp>`

Inheritance diagram for VanAlbada:



Public Member Functions

- VanAlbada ()

  Constructor.
- void calcul (SCALAR, SCALAR)

  Calculates the value of the slope limiter.
- virtual ~VanAlbada ()

  Destructor.

Public Member Functions inherited from Limiter

- Limiter ()

  Constructor.
- virtual void calcul (SCALAR, SCALAR)=0

  Function to be specified in each slope limiter.
- SCALAR get_rec () const

  Gives the reconstructed value.
- virtual ~Limiter ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Limiter

- SCALAR rec

4.66.1 Detailed Description

Van Albada slope limiter.

Class that calculates Van Albada slope limiter.

Definition at line 70 of file vanalbada.hpp.

4.66.2 Constructor & Destructor Documentation

VanAlbada()

```
VanAlbada::VanAlbada ( )
```

Constructor.

Definition at line 59 of file vanalbada.cpp.

~VanAlbada()

```
VanAlbada::~VanAlbada ( )  [virtual]
```

Destructor.

Definition at line 85 of file vanalbada.cpp.

4.66.3 Member Function Documentation

calcul()

```
void VanAlbada::calcul (
            SCALAR a,
            SCALAR b )  [virtual]
```

Calculates the value of the slope limiter.

Van Albada function:

$$\text{VA}(x,y) = \begin{cases} 0 & \text{if } \text{sign}(x) \neq \text{sign}(y), \\ \dfrac{x(y^2+\varepsilon)+y(x^2+\varepsilon)}{x^2+y^2+2\varepsilon} & \text{else,} \end{cases}$$

with $0 \leq \varepsilon \ll 1$.

Parameters

in	*a*	slope on the left of the cell.
in	*b*	slope on the right of the cell.

Modifies

Limiter::rec recontructed value.

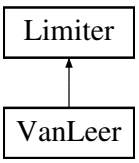
Implements Limiter.

Definition at line 62 of file vanalbada.cpp.
The documentation for this class was generated from the following files:

- Headers/liblimitations/vanalbada.hpp
- Sources/liblimitations/vanalbada.cpp

4.67 VanLeer Class Reference

Van Leer slope limiter.
`#include <vanleer.hpp>`
Inheritance diagram for VanLeer:



Public Member Functions

- VanLeer ()
  Constructor.
- void calcul (SCALAR, SCALAR)
  Calculates the value of the slope limiter.
- virtual ∼VanLeer ()
  Destructor.

Public Member Functions inherited from Limiter

- Limiter ()
  Constructor.
- virtual void calcul (SCALAR, SCALAR)=0
  Function to be specified in each slope limiter.
- SCALAR get_rec () const
  Gives the reconstructed value.
- virtual ∼Limiter ()
  Destructor.

Additional Inherited Members

Protected Attributes inherited from Limiter

- SCALAR rec

4.67.1 Detailed Description

Van Leer slope limiter.
Class that calculates Van Leer slope limiter.
Definition at line 70 of file vanleer.hpp.

4.67.2 Constructor & Destructor Documentation

VanLeer()

```
VanLeer::VanLeer ( )
```

Constructor.

Definition at line 59 of file vanleer.cpp.

~VanLeer()

```
VanLeer::~VanLeer ( )  [virtual]
```

Destructor.

Definition at line 84 of file vanleer.cpp.

4.67.3 Member Function Documentation

calcul()

```
void VanLeer::calcul (
            SCALAR a,
            SCALAR b )  [virtual]
```

Calculates the value of the slope limiter.

Van Leer function:

$$\mathrm{VL}(x,y) = \begin{cases} 0 & \text{if } xy \leq 0, \\ \dfrac{2xy}{x+y} & \text{else.} \end{cases}$$

Parameters

in	*a*	slope on the left of the cell.
in	*b*	slope on the right of the cell.

Modifies

Limiter::rec recontructed value.

Implements Limiter.

Definition at line 62 of file vanleer.cpp.

The documentation for this class was generated from the following files:

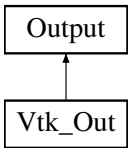
- Headers/liblimitations/vanleer.hpp
- Sources/liblimitations/vanleer.cpp

4.68 Vtk_Out Class Reference

VTK output.

`#include <vtk_out.hpp>`

Inheritance diagram for Vtk_Out:



Public Member Functions

- Vtk_Out (Parameters &)

  Constructor.
- void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)

  Saves one time step.
- virtual ~Vtk_Out ()

  Destructor.

Public Member Functions inherited from Output

- Output (Parameters &)

  Constructor.
- virtual void write (const TAB &, const TAB &, const TAB &, const TAB &, const SCALAR &)=0

  Function to be specified in each output format.
- void check_vol (const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &) const

  Saves the infiltrated and rain volumes.
- void result (const SCALAR &, const clock_t &, const SCALAR &, const SCALAR &, const SCALAR &, const SCALAR &, const int &, const SCALAR &) const

  Saves global values.
- void initial (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the initial time.
- void final (const TAB &, const TAB &, const TAB &, const TAB &) const

  Saves the final time.
- SCALAR boundaries_flux (const SCALAR &, const TAB &, const TAB &, const SCALAR &, const SCALAR &, const int &, const int &)

  Saves the cumulated fluxes on the boundaries.
- void boundaries_flux_LR (const SCALAR &, const TAB &) const

  Saves the fluxes on the left and right boundaries.
- void boundaries_flux_BT (const SCALAR &, const TAB &) const

  Saves the fluxes on the bottom and top boundaries.
- SCALAR round5 (const SCALAR &) const

  Rounded value up to 10^-5.
- virtual ~Output ()

  Destructor.

Additional Inherited Members

Protected Attributes inherited from Output

- const int NXCELL
- const int NYCELL
- const SCALAR DX
- const SCALAR DY
- string outputDirectory
- string namefile_check_volume
- string namefile_res
- string namefile_init
- string namefile_final

- string namefile_Bound_flux
- string namefile_Bound_flux_BT
- string namefile_Bound_flux_LR

4.68.1 Detailed Description

VTK output.

Class that writes the result in the output file with a structure optimized for VTK.

Definition at line 71 of file vtk_out.hpp.

4.68.2 Constructor & Destructor Documentation

Vtk_Out()

```
Vtk_Out::Vtk_Out (
            Parameters & par )
```

Constructor.

Defines the output name huz_evolution.dat

Parameters

in	*par*	parameter, contains all the values from the parameters file.

Definition at line 61 of file vtk_out.cpp.

~Vtk_Out()

```
Vtk_Out::~Vtk_Out ( )  [virtual]
```

Destructor.

Definition at line 260 of file vtk_out.cpp.

4.68.3 Member Function Documentation

write()

```
void Vtk_Out::write (
            const TAB & h,
            const TAB & u,
            const TAB & v,
            const TAB & z,
            const SCALAR & time )  [virtual]
```

Saves one time step.

Writes the values of Scheme::z, Scheme::h, Scheme::u (=q1/h), Scheme::v (=q2/h), Scheme::h+ Scheme::z (free surface), $|U| = \sqrt{u^2 + v^2}$, the Froude number $\frac{|U|}{\sqrt{gh}}$, Scheme::q1, Scheme::q2, and $h|U|$ at the current time in huz_evolution.dat***.vtk.

If the water height is too small, we replace it by 0, the velocity is null and the Froude number does not exist.

Parameters

in	*h*	the water height.

Parameters

in	*u*	first component of the velocity.
in	*v*	second component of the velocity.
in	*z*	the topography.
in	*time*	the current time.

Note

If huz_evolution.dat∗∗∗.vtk cannot be opened, the code will exit with failure termination code.

Implements Output.
Definition at line 77 of file vtk_out.cpp.
The documentation for this class was generated from the following files:

- Headers/libsave/vtk_out.hpp
- Sources/libsave/vtk_out.cpp

Bibliography

E. Audusse, M.-O. Bristeau, and B. Perthame. Kinetic schemes for Saint-Venant equations with source terms on unstructured grids. Technical Report 3989, INRIA, 2000. 97, 99

E. Audusse, F. Bouchut, M.-O. Bristeau, R. Klein, and B. Perthame. A fast and stable well-balanced scheme with hydrostatic reconstruction for shallow water flows. *SIAM J. Sci. Comput.*, 25(6):2050–2065, 2004. doi:10.1137/S1064827503431090. 104

P. Batten, N. Clarke, C. Lambert, and D. M. Causon. On the choice of wavespeeds for the HLLC Riemann solver. *SIAM Journal on Scientific Computing*, 18(6):1553–1570, 1997. doi:10.1137/S1064827593260140. 63, 68

F. Bouchut. *Nonlinear stability of finite volume methods for hyperbolic conservation laws, and well-balanced schemes for sources*, volume 2/2004. Birkhäuser Basel, 2004. doi:10.1007/b93802. 55, 58, 61, 66, 71

F. Bouchut. Chapter 4 efficient numerical finite volume schemes for shallow water models. In V. Zeitlin, editor, *Nonlinear Dynamics of Rotating Shallow Water: Methods and Advances*, volume 2 of *Edited Series on Advances in Nonlinear Science and Complexity*, pages 189 – 256. Elsevier Science, 2007. doi:10.1016/S1574-6909(06)02004-1. URL http://www.sciencedirect.com/science/article/B8JG3-4PS6TNS-6/2/f4c3dbcf476626c1cb6f353e9bc66cc9. 55, 58, 118

M.-O. Bristeau and B. Coussin. Boundary conditions for the shallow water equations solved by kinetic schemes. Technical Report RR-4282, INRIA, 2001. 1, 25

N. Goutal and F. Maurel. Proceedings of the 2^{nd} workshop on dam-break wave simulation. Technical Report HE-43/97/016/B, Electricité de France, Direction des études et recherches, 1997. 97, 99

A. Harten, S. Osher, B. Engquist, and S. R. Chakravarthy. Some results on uniformly high-order accurate essentially nonoscillatory schemes. *Applied Numerical Mathematics*, 2(3-5):347 – 377, 1986. ISSN 0168-9274. doi:10.1016/0168-9274(86)90039-5. URL http://www.sciencedirect.com/science/article/B6TYD-45GVV8T-J/2/830ca2dce5e3fb34ace9fa53ae5ab76b. Special Issue in Honor of Milt Rose's Sixtieth Birthday. 55

A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy. Uniformly High Order Accurate Essentially Non-oscillatory Schemes, III. *Journal of Computational Physics*, 71:231–303, 1987. 55

A. Y. Le Roux. Conditions aux limites et problèmes hyperboliques : un point de vue numérique. Available in the document of the Conférence at IHP-Paris, 2001. 1, 25

C.-W. Shu and S. Osher. Efficient implementation of essentially non-oscillatory shock-capturing schemes. *Journal of Computational Physics*, 77(2):439 – 471, 1988. ISSN 0021-9991. doi:10.1016/0021-9991(88)90177-5. URL http://www.sciencedirect.com/science/article/B6WHY-4DD1T6W-MM/2/bef99e4b67bd7132c8af1984c34ce57e. 55

M. W. Smith, N. J. Cox, and L. J. Bracken. Applying flow resistance equations to overland flows. *Progress in Physical Geography*, 31(4):363–387, 2007. doi:10.1177/0309133307081289. 77, 83

W. C. Thacker. Some exact solutions to the nonlinear shallow-water wave equations. *Journal of Fluid Mechanics*, 107:499–508, 1981. doi:10.1017/S0022112081001882. URL http://journals.cambridge.org/action/displayAbstract?fromPage=online&aid=389055&fulltextType=RA&fileId=S0022112081001882. 101, 225

E. F. Toro. *Shock-Capturing Methods for Free-Surface Shallow Flows*. John Wiley & Sons, 2001. 66, 68

B. van Leer. Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov's method. *Journal of Computational Physics*, 32(1):101 – 136, 1979. ISSN 0021-9991. doi:10.1016/0021-9991(79)90145-1. URL http://www.sciencedirect.com/science/article/B6WHY-4DD1N8T-C5/2/9b051d1cfcff715a3d0f4b7b7b0397cc. 118

Index